

Sıfırdan İşlemciye

Sayısal Donanım Tasarımı: Sayı Sistemlerinden Basit Bir CPU
Mimarisine

Şafak Şahin - Dr. Batuhan Hangün

Sıfırdan İşlemciye — Sayısal Donanım Tasarımı

Şafak Şahin - Dr. Batuhan Hangün

Sürüm 0.1.0, 2026

Kişisel çalışma notu olarak hazırlanmıştır; yayıncı yoktur.

Bu belgenin LaTeX altyapısı (preamble/renk-modu/tikz-sekiller-makrolar/sürüm-ayarları), Oğuz Ergin'in *Bilgisayar Mimarisi* kitabından (<https://github.com/prof-oguzergin/bilgisayar-mimarisi>) uyarlanmıştır; bkz. o projenin lisansı. Bu belgedeki özgün metin ve teknik içerik yazarına aittir.

İlk sürüm, Temmuz 2026

İçindekiler

Şekil Listesi	viii
Tablo Listesi	ix
Bilgi Notları Listesi	xi
Detaylı Bilgi Listesi	xiii
1 Sayı Sistemleri ve Kodlama	1
1.1 Giriş	1
1.2 Sayı Tabanları ve Dönüşümler	2
1.2.1 Pozisyonel Sayı Sistemi	2
1.2.2 İkiliden Ondalığa Dönüşüm	2
1.2.3 Ondalıktan İkiliye Dönüşüm	2
1.2.4 Sekizlik ve Onaltılık Sistemler: Sembol Sorunu	4
1.2.5 Sekizlik ve Onaltılıktan Ondalığa Dönüşüm	4
1.2.6 Ondalıktan Sekizliğe ve Onaltılığa Dönüşüm	5
1.2.7 İkili, Sekizlik ve Onaltılık Arasında Gruplama	6
1.2.8 Tabanı Bilinmeyen İşlemlerde Taban Bulma	7
1.2.9 Kesirli Sayı Dönüşümleri	8
1.3 İkili Aritmetik: Toplama, Çıkarma, Çarpma ve Bölme	10
1.3.1 İkili Toplama	11
1.3.2 İkili Çıkarma	12
1.3.3 İkili Çarpma	12
1.3.4 İkili Bölme	13
1.4 Tümleyenler	14
1.4.1 Genel Tümleyen Tanımı	15
1.4.2 Ondalık Örneklerle Tümleyen Hesabı	15
1.4.3 Kısasol Yöntemi ve İspatı	16
1.4.4 İkilide Tümleyenler	16
1.4.5 Sekizlik ve Onaltılıkta $(r-1)$ 'in Tümleyeni	17
1.4.6 Kesirli (Noktalı) Sayılarda Tümleyen Alma	17
1.4.7 r 'nin Tümleyeniyle Çıkarma İşlemi	18
1.4.8 $(r-1)$ 'in Tümleyeniyle Çıkarma: Uçtan-Dolaşan Elde	20
1.5 İşaretili İkili Sayılar	21
1.5.1 İşaret Biti ve Yorumlama	21

1.5.2	İşaret-Büyüklik Gösterimi	21
1.5.3	İşaretli Tümlleyen Sistemi: 1'in ve 2'nin Tümlayeni	21
1.5.4	Neden 2'nin Tümlayeni Tercih Edilir	23
1.5.5	İşaretli Toplama	24
1.5.6	İşaretli Çıkarma	25
1.6	IEEE 754 ile Kayan Noktalı Sayı Gösterimi	27
1.6.1	Neden Sabit Nokta Yetmez?	28
1.6.2	Normalizasyon: Bedava Bir Bit	28
1.6.3	IEEE 754 Formatları ve Bias	30
1.6.4	Ondalıktan IEEE 754'e Dönüşüm	31
1.6.5	IEEE 754'ten Ondalığa Dönüşüm	31
1.6.6	Özel Değerler ve Kapanış	32
1.7	İkili Kodlama Sistemleri	34
1.7.1	BCD (Binary-Coded Decimal) Kodu	34
1.7.2	BCD Toplama	35
1.7.3	Diğer Ondalık Kodlar	37
1.7.4	BCD'de İşaretli Ondalık Aritmetik (10'un Tümlayeni)	40
1.7.5	Gray Kodu	42
1.7.6	ASCII Karakter Kodu	45
1.7.7	Hata Sezme: Parity Biti	46
2	Bool Cebiri ve Mantık Kapıları	49
2.1	Giriş	49
2.1.1	Bool Cebirinin Kısa Tarihçesi	49
2.1.2	İkili Değişkenler ve Mantık Fonksiyonları	50
2.2	Cebirsel Yapı Kavramı	53
2.2.1	Postülat Nedir?	53
2.2.2	Küme ve İkili İşlem	54
2.2.3	Cebirsel Yapı İçin Altı Genel Postülat	54
2.3	Bool Cebirinin Aksiyomları	55
2.3.1	Sıradan Cebirle Karşılaştırma	56
2.3.2	İki-Değerli Bool Cebirinin Doğrulanması	58
2.4	Düallite İlkesi ve Temel Teoremler	59
2.4.1	Operatör Önceliği	61
2.5	De Morgan Teoremi	62
2.5.1	Doğruluk Tablosuyla Doğrulama	63
2.5.2	n -Değişkenli Genelleme	64
2.5.3	Bir Fonksiyonun Tümlayenini Bulmanın İki Yolu	65
2.5.4	Diğer Mantık İşlemleri	66
2.6	Sayısal Mantık Kapıları	67
2.6.1	Sekiz Temel Kapı	67
2.6.2	Sadeleştirmenin Önemi: Devre Üzerinden Gösterim	71
2.6.3	Çok Girişli Genişletme	73
2.6.4	Pozitif ve Negatif Mantık	75
2.7	Kanonik ve Standart Formlar	77
2.7.1	Minterm ve Maxterm	77

2.7.2	Minterm Toplamı ve Maxterm Çarpımı Simgeleri	78
2.7.3	Cebirsel İfadeyi Minterm Toplamına Genişletme	79
2.7.4	Standart Formlar: Çarpımların Toplamı ve Toplamların Çarpımı	80
2.8	Entegre Devreler	82
2.8.1	Entegrasyon Seviyeleri	82
2.8.2	Dijital Lojik Aileleri	83
Terimler Sözlüğü		85

Şekil Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

- 1.1 Frank Gray, 1930’da geliştirdiği mekanik televizyon aygıtını incelerken. 42
- 1.2 Gray’in 2,632,058 numaralı patentinin ön sayfasından bir bölüm; yansıtılmış ikili kodu taşıyan plakayı (15) içeren PCM tüpü (10) görülüyor. 43

Bölüm 2 – Bool Cebiri ve Mantık Kapıları

- 2.1 Boole’un portresi, *The Illustrated London News*, 21 Ocak 1865. 50
- 2.2 Claude Shannon, y. 1950’ler (Tekniska Museet). 50
- 2.3 Beş bölgeli sinyal seviyesi diyagramı (şematik, ölçekli değildir). 51
- 2.4 Gönderen ve alan kapı arasındaki gürültü payı. 52
- 2.5 Edward V. Huntington (1874–1952). 56
- 2.6 Paralel $(x + y)$ ve seri $(x \cdot y)$ anahtar bağlantısı. 57
- 2.7 Postülat 2’nin $(x + x' = 1, x \cdot x' = 0)$ anahtarlama devresi doğrulaması. 57
- 2.8 Postülat 1’in $(x + 0 = x, x \cdot 1 = x)$ anahtarlama devresi doğrulaması. 58
- 2.9 Augustus De Morgan (1806–1871). 62
- 2.10 AND kapısı: sembol ve doğruluk tablosu. 68
- 2.11 OR kapısı: sembol ve doğruluk tablosu. 68
- 2.12 NOT (tersleyici) kapısı: sembol ve doğruluk tablosu. 68
- 2.13 Tampon (buffer) kapısı: sembol ve doğruluk tablosu. 68
- 2.14 NAND kapısı: sembol ve doğruluk tablosu. 69
- 2.15 NOR kapısı: sembol ve doğruluk tablosu. 69
- 2.16 XOR (özel-VEYA) kapısı: sembol ve doğruluk tablosu. 69
- 2.17 XNOR (denklik) kapısı: sembol ve doğruluk tablosu. 69
- 2.18 Sadeleştirilmemiş devre: $F = x'y'z + xyz + x'yz + xy'z$ (4 AND + 1 OR; 2 NOT şekle dahil edilmedi). 71
- 2.19 Sadeleştirilmiş devre: $F = z$, tek bir tel. 71
- 2.20 NOR işleminin birleşmeli olmadığının gösterimi: $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$ 74
- 2.21 Üç girişli NOR ve NAND kapılarının grafik sembolleri. 74
- 2.22 Ardı ardına bağlı NAND kapılarıyla $F = [(ABC)'(DE)']' = ABC + DE$ gerçekleştirilmesi. 74
- 2.23 Üç girişli XOR’un iki eşdeğer gösterimi (üstte: ardı ardına bağlantı; altta: tek kapı sembolü). 75
- 2.24 Mantıksal değerin sinyal değerine ataması. 76
- 2.25 Aynı fiziksel kapının pozitif ve negatif mantıktaki sembolleri. 76
- 2.26 $F_1 = z + xy + x'y'z'$ ifadesinin çarpımların toplamı (SOP) biçiminde, iki-seviyeli gerçekleştirilmesi. 80

2.27	$F_2 = x(y + z')(x' + y' + z)$ ifadesinin toplamaların çarpımı (POS) biçiminde, iki-seviyeli gerçekleştirilmesi.	81
2.28	$F_3 = A(B + C) + DE$: standart olmayan, üç seviyeli devre.	81
2.29	$F_3 = AB + AC + DE$: aynı fonksiyonun standart (SOP), iki seviyeli gerçekleştirilmesi.	82

Tablo Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

1.1	Ondalık–Onaltılık–İkili Karşılık Tablosu	4
1.2	4-Bit İşaretli Sayılar: Üç Gösterimin Karşılaştırması	24
1.3	IEEE 754 Tek ve Çift Duyarlıklı Formatlar	30
1.4	Dört Farklı Ondalık Kodun Karşılaştırılması	38
1.5	0’dan 15’e kadar ikili ve Gray kodu karşılaştırması.	44
1.6	Standart ASCII karakter tablosu (7 bit, $b_6 \dots b_0$).	46

Bölüm 2 – Bool Cebiri ve Mantık Kapıları

2.1	Gerilim eşiği simgelerinin açılımı.	51
2.2	Örnek gerilim eşikleri (5 V’luk bir sistem örneği).	51
2.3	Huntington Postülatları	56
2.4	İki-değerli Bool cebrinin işlem tabloları.	58
2.5	Dağılma postülatının ($x \cdot (y+z) = xy+xz$) mükemmel tümevarımla doğrulanması.	59
2.6	Temel Teoremler	60
2.7	$(x + y)' = x' y'$ özdeşliğinin doğruluk tablosuyla doğrulanması.	64
2.8	İki ikili değişkenin 16 fonksiyonu için doğruluk tabloları	66
2.9	İki değişkenin 16 fonksiyonu için Bool ifadeleri	67
2.10	Üç girişli XOR’un doğruluk tablosu.	75
2.11	Aynı fiziksel kapının pozitif mantıkta AND, negatif mantıkta OR olarak yorumlanması.	76
2.12	Üç ikili değişken için minterm ve maxtermler.	78
2.13	$F = x + y'z$ fonksiyonunun doğruluk tablosu.	78

Bilgi Notları Listesi

Bölüm 1 – Sayı Sistemleri ve Kodlama

1.1	Neden Bilgisayarlar İkili Kullanır?	2
1.2	Neden Kalan, Bölüm Değil?	3
1.3	Neden "10" Değil de Yeni Bir Sembol?	4
1.4	Tuzak: Yön Simetrik Değildir	8
1.5	Her Kesir Sonlanmaz	9
1.6	Neden Çıkarmayı Toplamaya Çeviriyoruz?	14
1.7	Pratik Kısayol: r 'nin Tümleyenini Doğrudan Almak	16
1.8	Kısayol Neden İşliyor? Borç Yayılımı İspatı	16
1.9	Önce Basamak Sayılarını Eşitleyin	19
1.10	Aynı Bitler, Farklı Anlam	21
1.11	İpucu: Ağırlıklı Toplamla Doğrudan Değer Okuma	22
1.12	Sebeplere 1: Tek Sıfır	23
1.13	Sebeplere 2 (Asıl Önemli Olan): Aritmetik Kolaylığı	23
1.14	Taşma (Overflow)	25
1.15	Büyük Resim: Tek Bir Devre Yeter	26
1.16	Üç Terimin Kökeni: Mantissa, Bias, Üs	27
1.17	Örtük Baştaki 1	28
1.18	Neden İşaretli Değil, Biaslı Üs?	30
1.19	Artık Biliyoruz: $0.1 + 0.2 \neq 0.3$	34
1.20	Öz-Tümleyen Kodlarla Bağlantı	40
1.21	İkiliden Gray Koduna Dönüşüm	44
1.22	Tarihi Bir Not: ASCII'nin Orijinal Bit Numaralandırması	45
1.23	Neden 8 Bit (Byte) ile Saklanır?	46
1.24	Parity'nin Kritik Sınırlaması: Çift Sayıda Hatayı Yakalayamaz	47

Bölüm 2 – Bool Cebiri ve Mantık Kapıları

2.1	Bir Devre 0 ve 1'i Fiziksel Olarak Nasıl Ayırt Eder?	50
2.2	Neden Üç Eşdeğer Gösterim?	53
2.3	Kelimenin Kökeni: Kabul mü, İspat mı?	53
2.4	Neden İspatsız Kabul Ederiz?	53
2.5	Bool Cebiri Nereye Oturuyor?	55
2.6	Anahtarlama Devresi Yorumu	57
2.7	Düalitenin Getirisi: İkinci Teorem Bedava	59
2.8	Teorem 3: Çifte Tümleme Neden Farklı İspatlanır?	60

2.9	İspatların İkinci Yolu: Doğruluk Tablosu	61
2.10	İkinci Yarı Düallite ile Bedava	63
2.11	Kanıt Yöntemi: Matematiksel Tümevarım (Induction)	64
2.12	Tampon Bir Mantık İşlemi mi?	68
2.13	NAND ve NOR Neden Bu Kadar Yaygın?	69
2.14	Sadeleştirme Neden Bu Kadar Önemli?	71
2.15	Bu Kitapta Hangi Mantık Kullanılıyor?	77

Detaylı Bilgi Listesi

2.1	Yasak Bölge Gerçekten Ne Anlama Gelir?	52
2.2	Gürültü Payı (Noise Margin)	52
2.3	CMOS Transistör Düzeyinde NAND ve NOR Neden Daha Az Eleman Gerektirir?	70

Sayı Sistemleri ve Kodlama

Öğrenme Hedefleri

Bu bölümü tamamladığınızda aşağıdakileri yapabileceksiniz:

- Pozisyonel sayı sistemlerinin genel yapısını ve herhangi bir tabandaki sayıyı ondalığa çevirmeyi uygulamak.
- Ondalık bir sayıyı ardışık bölme yöntemiyle istenen tabana çevirmek.
- İkili, sekizlik ve onaltılık sistemler arasında doğrudan bit gruplamasıyla dönüşüm yapmak.
- İkili sayılarda toplama, çıkarma, çarpma ve bölme işlemlerini genel r -tabanlı formüllerleriyle birlikte elle yapmak.
- $(r-1)$ 'in ve r 'nin tümleyenlerini (9'un/10'un, 1'in/2'nin) hesaplamak ve çıkarma işlemini toplamaya çevirmek için kullanmak.
- İşaretli tam sayıların (işaret-büyüklik, 1'in ve 2'nin tümleyeni) gösterimlerini karşılaştırmak ve 2'nin tümleyeninin neden tercih edildiğini gerekçelendirmek.
- 2'nin tümleyeni sistemiyle işaretli toplama/çıkarma yapmak ve taşma (overflow) kavramını tanımlamak.
- IEEE 754 standardıyla kayan noktalı (ondalıklı) sayıları normalize edip işaret/üs/mantissa alanlarına ayırmak, ve $0.1+0.2 \neq 0.3$ gibi yuvarlama tuhaflıklarının kaynağını açıklamak.
- BCD, Gray kodu ve ASCII gibi temel ikili kodların kullanım amaçlarını açıklamak.

1.1 Giriş

Bir bilgisayarın belleğinde tuttuğu her şey (bir tam sayı, bir metin karakteri, bir fotoğrafın bir pikseli, bir ses örneği) sonunda aynı hammaddeye indirgenir: 0 ve 1'lerden oluşan bir dizi. Bu indirgemenin nasıl yapıldığını, yani sayıların ve sembollerin ikili düzende nasıl *kodlandığını* anlamadan, bu bitleri işleyecek devreleri tasarlamaya başlayamayız. Bu yüzden yolculuğumuza, tüm sayısal donanım tasarımının en alt katmanını olan sayı sistemleriyle başlıyoruz: önce herhangi bir tabandaki bir sayıyı nasıl okuyup dönüştüreceğimizi, ardından negatif sayıları ve metni/sembolleri ikili düzende nasıl temsil edeceğimizi göreceğiz.

1.2 Sayı Tabanları ve Dönüşümler

1.2.1 Pozisyonel Sayı Sistemi

Pozisyonel Sayı Sistemi: Bir sayı sisteminde her basamağın değeri, o basamağın *konumuna* bağlıysa buna *pozisyonel (ağırlıklı) sayı sistemi* denir. r tabanında (radix) yazılmış bir sayı, aşağıdaki ağırlıklı toplamla ifade edilir:

$$(d_n d_{n-1} \dots d_1 d_0)_r = \sum_{i=0}^n d_i \cdot r^i, \quad 0 \leq d_i < r$$

Günlük kullandığımız ondalık (decimal) sistem, $r = 10$ özel durumudur; örneğin $137 = 1 \cdot 10^2 + 3 \cdot 10^1 + 7 \cdot 10^0$.

Bilgi Notu 1.1: Neden Bilgisayarlar İkili Kullanır?

İkili (binary, $r = 2$) sistemin seçimi matematiksel değil, *fiziksel* bir mühendislik kararıdır. Bir devrede onluk sistem için gereken 10 farklı gerilim seviyesini güvenilir biçimde ayırt etmek pahalı ve gürültüye karşı kırılgan olurdu. Seviyeler arası fark küçük kalır, en ufak bir parazit yanlış okumaya yol açar. Yalnızca *iki* durumu (akım var/yok, yüksek/düşük gerilim) ayırt etmek çok daha güvenilir ve ucuzdur; seviyeler arasında büyük bir güvenlik payı (gürültü marjı) bırakılabilir. İkili değişkenler üzerine kurulu Bool cebirini Bölüm 2’de işleyeceğiz.

1.2.2 İkiliden Ondalığa Dönüşüm

Bir tabandan ondalığa dönüşüm, pozisyonel sistem tanımındaki ağırlıklı toplamın doğrudan uygulanmasıyla yapılır. Zorluğu artan üç örnekle pekiştirelim.

Örnek 1.1: $(101)_2$ ’nin Ondalık Karşılığı

$$(101)_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1 = (5)_{10}$$

Örnek 1.2: $(1011)_2$ ’nin Ondalık Karşılığı

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 8 + 0 + 2 + 1 = (11)_{10}$$

Örnek 1.3: $(110101)_2$ ’nin Ondalık Karşılığı

$$(110101)_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 16 + 0 + 4 + 0 + 1 = (53)_{10}$$

1.2.3 Ondalıktan İkiliye Dönüşüm

Ondalıktan bir tabana geçiş, ağırlıklı toplamın tersine çevrilmesiyle yapılır: tam sayı kısmı hedef tabana *ardışık olarak bölünür*; her adımın kalanı, *sondan başa* doğru okunarak sonuç

elde edilir. Şimdi bunu, zorluğu artan üç örnekle, hep ikilik tabanda pekiştirelim.

Örnek 1.4: $(6)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$6 \div 2$	3	0 (en düşük anlamlı bit)
$3 \div 2$	1	1
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(6)_{10} = (110)_2$.

Bilgi Notu 1.2: Neden Kalan, Bölüm Değil?

$N = d_n r^n + \dots + d_1 r + d_0$ olsun. d_0 dışındaki her terim r 'nin katı olduğundan, $N \bmod r = d_0$; yani **kalan, her zaman tam olarak en düşük anlamlı basamaktır**. Buna karşılık $\lfloor N/r \rfloor = d_n r^{n-1} + \dots + d_1$: **bölüm, aynı sayının son basamağı silinmiş hâlidir**, kendisi asla bir basamak değildir. İşlemi bölüm 0'a düşene kadar tekrarlarız (0'ı basamak olarak yazmayız); her adımda yazılan basamak o adımın *kalanıdır*, bölümü değil. Yukarıdaki örneğin ikinci satırında ($3 \div 2 = 1$ kalan 1) bölüm (1) ile kalan (1) aynı değeri taşıyor; bu, satırı yanlış okuyup bölümü basamak sanmanın fark edilmeyeceği bir durum yaratır. Aşağıdaki iki örnekte bölüm ve kalanın genelde *birbirinden bağımsız, farklı* sayılar olduğunu göreceğiz.

Örnek 1.5: $(25)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$25 \div 2$	12	1 (en düşük anlamlı bit)
$12 \div 2$	6	0
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(25)_{10} = (11001)_2$.

Örnek 1.6: $(45)_{10}$ 'un İkili Karşılığı

İşlem	Bölüm	Kalan
$45 \div 2$	22	1 (en düşük anlamlı bit)
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı okunduğunda: $(45)_{10} = (101101)_2$. Doğrulama: $32 + 8 + 4 + 1 = 45$. ✓

1.2.4 Sekizlik ve Onaltılık Sistemler: Sembol Sorunu

Onluk sistemde tam 10 rakam sembolümüz vardır: $0, 1, \dots, 9$. Bu, $r \leq 10$ olan her taban için yeterlidir. Sekizlik sistemde yalnızca 0-7 kullanılır, 8 ve 9'a hiç dokunulmaz. Ama $r = 16$ olduğunda her basamak 0'dan 15'e kadar **16 farklı değer** alabilmelidir; elimizdeki 10 rakam sembolü 6 değer için (10, 11, ..., 15) yetersiz kalır.

Bilgi Notu 1.3: Neden "10" Değil de Yeni Bir Sembol?

Akla gelen ilk çözüm, 10'u iki karakterle ("10") yazmak gibi görünebilir; ama bu, pozisyonel gösterimin temel kuralını bozar: **her basamak tek bir karakter olmalıdır**, yoksa gösterim belirsizleşir. $(110)_{16}$ dizisini gördüğünüzde bunun "1, 1, 0" mı yoksa "1, 10" mı olduğunu ayırt edemezsiniz. Çözüm, onluk rakamların bittiği yerden alfabeyle devam etmektir: $A = 10, B = 11, C = 12, D = 13, E = 14, F = 15$. Bu, matematiksel bir zorunluluk değil, *yalnızca* her basamağı tek karakterde tutmak için benimsenmiş bir gösterim kuralıdır.

Ondalık	Onaltılık	4-bit İkili
0–9	0–9	0000–1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Tablo 1.1: Ondalık–Onaltılık–İkili Karşılık Tablosu

Bu tablo (Tablo 1.1), donanım tasarımında sürekli kullanılacağı için ezberlenmeye değer.

1.2.5 Sekizlik ve Onaltılıktan Ondalığa Dönüşüm

Bu dönüşüm, Bölüm 1.2.1'de kurduğumuz ağırlıklı toplam formülünün doğrudan uygulanmasıdır. Yeni bir yöntem gerekmez. Onaltılıkta tek fark, bir harf basamağıyla karşılaştığınızda onu Tablo 1.1'daki sayısal karşılığıyla değiştirmenizdir.

Örnek 1.7: $(247)_8$ 'in Ondalık Karşılığı

$$(247)_8 = 2 \cdot 8^2 + 4 \cdot 8^1 + 7 \cdot 8^0 = 128 + 32 + 7 = (167)_{10}$$

Örnek 1.8: $(17)_8$ 'in Ondalık Karşılığı

$$(17)_8 = 1 \cdot 8^1 + 7 \cdot 8^0 = 8 + 7 = (15)_{10}$$

Örnek 1.9: $(3A)_{16}$ 'nın Ondalık Karşılığı

Harf basamağı önce sayısal değerine çevrilir: $A = 10$.

$$(3A)_{16} = 3 \cdot 16^1 + 10 \cdot 16^0 = 48 + 10 = (58)_{10}$$

Örnek 1.10: $(2FC)_{16}$ 'nın Ondalık Karşılığı

İki harf basamağı birden çevrilir: $F = 15$, $C = 12$.

$$(2FC)_{16} = 2 \cdot 16^2 + 15 \cdot 16^1 + 12 \cdot 16^0 = 512 + 240 + 12 = (764)_{10}$$

1.2.6 Ondalıktan Sekizliğe ve Onaltılığa Dönüşüm

Bu dönüşüm de yeni bir yöntem gerektirmez: Bölüm 1.2.3'de öğrendiğimiz ardışık bölme yöntemi, taban 2 yerine 8 veya 16 ile aynen uygulanır. Onaltılıkta tek fark, bir kalan 10 veya üzerine çıktığında Tablo 1.1 ile harfe çevrilmesidir.

Örnek 1.11: $(100)_{10}$ 'un Sekizlik Karşılığı

İşlem	Bölüm	Kalan
$100 \div 8$	12	4 (en düşük anlamlı basamak)
$12 \div 8$	1	4
$1 \div 8$	0	1 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(100)_{10} = (144)_8$. Doğrulama: $1 \cdot 8^2 + 4 \cdot 8^1 + 4 \cdot 8^0 = 64 + 32 + 4 = 100$. ✓

Örnek 1.12: $(45)_{10}$ 'un Sekizlik Karşılığı

İşlem	Bölüm	Kalan
$45 \div 8$	5	5 (en düşük anlamlı basamak)
$5 \div 8$	0	5 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(45)_{10} = (55)_8$. Doğrulama: $5 \cdot 8^1 + 5 \cdot 8^0 = 40 + 5 = 45$. ✓

Örnek 1.13: $(202)_{10}$ 'un Onaltılık Karşılığı: Bir Tutarlılık Sınaması

Bu örneği daha önce, Bölüm 1.2.7'da bit gruplamasıyla da çözüp $(CA)_{16}$ bulacağız. Burada *başka bir yöntemle* aynı sonuca ulaşarak iki yöntemi çapraz doğrulamış olacağız.

İşlem	Bölüm	Kalan
$202 \div 16$	12	10 = A (en düşük anlamlı basamak)
$12 \div 16$	0	12 = C (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(202)_{10} = (CA)_{16}$.

Örnek 1.14: $(500)_{10}$ 'un Onaltılık Karşılığı

İşlem	Bölüm	Kalan
$500 \div 16$	31	4 (en düşük anlamlı basamak)
$31 \div 16$	1	15 = F
$1 \div 16$	0	1 (en yüksek anlamlı basamak)

Kalanlar aşağıdan yukarı okunduğunda: $(500)_{10} = (1F4)_{16}$. Doğrulama: $1 \cdot 16^2 + 15 \cdot 16^1 + 4 \cdot 16^0 = 256 + 240 + 4 = 500$. ✓

1.2.7 İkili, Sekizlik ve Onaltılık Arasında Gruplama

İkili-Sekizlik-Onaltılık Arası Gruplama: İkili sayılar bilgisayar için idealdir ama insan gözüyle takip edilmesi zahmetlidir. Sekizlik (octal, $r = 8$) ve onaltılık (hexadecimal, $r = 16$) sistemler tam bu yüzden kullanılır: $8 = 2^3$ ve $16 = 2^4$ olduğu için, ikili ile bu tabanlar arasındaki dönüşüm bir bölme/çarpma *gerektirmez*. Yalnızca bitlerin gruplanmasıyla yapılır: her **3 bit** bir sekizlik basamağa, her **4 bit** bir onaltılık basamağa karşılık gelir.

Örnek 1.15: İkiliden Onaltılığa Gruplama

$(11001010)_2$ sayısını onaltılığa çevirelim; sağdan başlayarak 4'erli gruplarız:

$$\underbrace{1100}_C \underbrace{1010}_A \Rightarrow (CA)_{16}$$

Örnek 1.16: Onaltılıdan İkiliye Açma: Ters Yön

Gruplama tersine de çevrilebilir: her onaltılık basamak, kendi 4-bit ikili karşılığıyla değiştirilir. $(CA)_{16}$ 'yı ikiliye açalım:

$$\underbrace{C}_{1100} \underbrace{A}_{1010} \Rightarrow (11001010)_2$$

Bu, bir önceki örneğin tam tersidir; aynı sayı çifti arasında gidip gelebildiğimizi gösterir.

Örnek 1.17: İkiliden Sekizliğe Grublama

$(101101110)_2$ sayısını sekizliğe çevirelim; bu kez bit sayısı (9) tam olarak 3'e bölündüğü için baştan sıfır eklemeye gerek yok, doğrudan sağdan 3'erli gruplanır:

$$\underbrace{101}_5 \underbrace{101}_5 \underbrace{110}_6 \Rightarrow (556)_8$$

Doğrulama, her iki tarafı da onluk tabana çevirip karşılaştırarak yapılır: $(556)_8 = 5 \cdot 8^2 + 5 \cdot 8^1 + 6 \cdot 8^0 = 320 + 40 + 6 = (366)_{10}$, ve $(101101110)_2 = 1 \cdot 256 + 1 \cdot 64 + 1 \cdot 32 + 1 \cdot 8 + 1 \cdot 4 + 1 \cdot 2 = (366)_{10}$. İki taraf da aynı onluk değere (366) eşit olduğundan sonuç doğrulanmış olur. ✓

Alıştırma 1.1 ★ $(11001010)_2$ sayısını, yukarıdaki örnekteki gibi ama bu kez **sekizlik** sisteme, sağdan 3'erli gruplayarak çeviriniz. (İpucu: 8 bit 3'e tam bölünmez; en sola gerektiği kadar 0 ekleyerek grupları tamamlayınız.)

Çözüm: 8 biti 3'e tam bölmek için sola bir 0 eklenir, 9 bit elde edilir; sağdan 3'erli gruplanır:

$$\underbrace{011}_3 \underbrace{001}_1 \underbrace{010}_2 \Rightarrow (312)_8$$

Doğrulama: $(312)_8 = 3 \cdot 8^2 + 1 \cdot 8^1 + 2 \cdot 8^0 = 192 + 8 + 2 = 202$, ve $(11001010)_2 = (202)_{10}$ — iki sonuç örtüşüyor.

1.2.8 Tabanı Bilinmeyen İşlemlerde Taban Bulma

Şimdiye kadar hep taban r 'yi bilip bir sayıyı çeviriyorduk. Bu alt bölümde işi tersine çevireceğiz: pozisyonel sistem tanımındaki ağırlıklı toplam formülünü ($\sum d_i r^i$), tabanın kendisini **bilinmeyen** kabul ederek bir denklem olarak kuracak ve r için çözeceğiz. Tek kısıtlama akılda tutulmalı: bir ifadede kullanılan her basamak, tabandan *küçük* olmak zorundadır ($d_i < r$); bulunan çözüm bu kısıtla sınanmalıdır.

Örnek 1.18: $(23)_x$ Sayısının Ondalık Karşılığı 19 İse x Nedir?

Ağırlıklı toplam formülü, üsler açık yazılarak kurulur; $x^0 = 1$ olduğu için birler basamağı x ile çarpılmaz, sadece kendisi kalır:

$$(23)_x = 2 \cdot x^1 + 3 \cdot x^0 = 2x + 3$$

Bu ifade 19'a eşitlenip x için çözülür:

$$2x + 3 = 19 \Rightarrow 2x = 16 \Rightarrow x = 8$$

Kısıt kontrolü: kullanılan basamaklar $\{2, 3\}$, $x = 8$ bunların ikisinden de büyük. ✓

Örnek 1.19: $(34)_x + (21)_x = (105)_x$ İse x Nedir?

Üç tarafı da önce üsler açık yazılarak kuralım:

$$(3 \cdot x^1 + 4 \cdot x^0) + (2 \cdot x^1 + 1 \cdot x^0) = 1 \cdot x^2 + 0 \cdot x^1 + 5 \cdot x^0$$

Sadeleştirilince:

$$(3x + 4) + (2x + 1) = x^2 + 5 \Rightarrow 5x + 5 = x^2 + 5 \Rightarrow x^2 - 5x = 0 \Rightarrow x(x - 5) = 0$$

İki kök: $x = 0$ ve $x = 5$. $x = 0$ geçerli bir taban olamaz (ayrıca kısıt $x > 4$ 'ü de sağlamaz); $x = 5$ kısıtı sağlar ($5 > 4$). ✓Doğrulama: $(34)_5=19$, $(21)_5=11$, toplam $30 = (105)_5$.

Örnek 1.20: $(32)_x + (24)_x = (100)_x$ İse x Nedir?

Bu kez eşitliğin sağ tarafı (right-hand side, RHS) üç basamaklı olduğu için denklem *ikinci dereceden* çıkar. Üsler açık:

$$(3 \cdot x^1 + 2 \cdot x^0) + (2 \cdot x^1 + 4 \cdot x^0) = 1 \cdot x^2 + 0 \cdot x^1 + 0 \cdot x^0$$

Sadeleştirilince:

$$(3x + 2) + (2x + 4) = x^2 \Rightarrow x^2 - 5x - 6 = 0 \Rightarrow (x - 6)(x + 1) = 0$$

İki kök: $x = 6$ ve $x = -1$. Negatif taban fiziksel olarak anlamsız olduğundan $x = -1$ reddedilir; $x = 6$ kısıtı da sağlar ($6 > 4$). ✓Doğrulama: $(32)_6=20$, $(24)_6=16$, toplam $36 = (100)_6$.

1.2.9 Kesirli Sayı Dönüşümleri

Pozisyonel gösterim yalnızca tam sayılarla sınırlı değildir; noktadan sonraki basamaklar *negatif* kuvvetler taşır:

$$(0.d_{-1}d_{-2} \dots d_{-m})_r = \sum_{i=1}^m d_{-i} \cdot r^{-i}$$

Tabandan ondalığa dönüşüm yine doğrudan bu toplamla yapılır; örneğin $(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.5 + 0 + 0.125 = (0.625)_{10}$. Ondalıktan bir tabana geçiş ise, tam sayı kısmından **farklı ve ters işleyen** bir yöntem gerektirir.

Örnek 1.21: Ondalıktan Bir Tabana Dönüşüm: Ardışık Çarpma Yöntemi

Kesirli kısım hedef tabanla ardışık olarak çarpılır; her adımda taşan tam sayı basamağı sonucun bir hanesi olur ve kalan kesirle işleme devam edilir. $(0.6875)_{10}$ 'u ikiliye çevirelim:

İşlem	Sonuç	Taşan Basamak
0.6875×2	1.375	1 (en yüksek anlamlı bit)
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1 (en düşük anlamlı bit)

Kesir 0'a düştüğü için işlem sonlanır. Taşan basamaklar **yukarıdan aşağı** okunur: $(0.6875)_{10} = (0.1011)_2$.

Bilgi Notu 1.4: Tuzak: Yön Simetrik Değildir

Tam sayı dönüşümünde (ardışık bölme) sonuç *aşağıdan yukarı* okunurken, kesir dönüşümünde (ardışık çarpma) sonuç *yukarıdan aşağı* okunur. Bu asimetri sık yapılan bir hatanın kaynağıdır. Bir sayının tam ve kesirli kısımlarını ayrı ayrı çevirip doğru yönde birleştirmeyi unutmayın.

Bilgi Notu 1.5: Her Kesir Sonlanmaz

Her ondalık kesir, hedef tabanda sonlu basamakla ifade edilemez. Bunu somut görmek için $(0.1)_{10}$ 'u ardışık çarpma yöntemiyle ikiliye çevirmeyi deneyelim:

İşlem	Sonuç	Taşan Basamak
0.1×2	0.2	0
0.2×2	0.4	0
0.4×2	0.8	0
0.8×2	1.6	1
0.6×2	1.2	1
0.2×2	0.4	0

Altıncı satırda kesir tekrar 0.2'ye döner; bu değeri ikinci satırda zaten görmüştük. Yani işlem hiçbir zaman 0'a düşmeyen bir **döngüye** girmiştir: kesir sonsuza kadar $0.2 \rightarrow 0.4 \rightarrow 0.8 \rightarrow 0.6 \rightarrow 0.2 \rightarrow \dots$ sırasını tekrar eder ve her turda aynı 0, 0, 1, 1 basamak dörtlüsünü üretir. Sonuç, sonsuz tekrar eden bir ikili dizidir: $(0.1)_{10} = (0.0001100110011 \dots)_2$. Bu olgu, kayan noktalı sayı gösteriminde karşılaşılan $0.1 + 0.2 \neq 0.3$ gibi tanıdık tuhaflıkların temel nedenidir: sonlu bit genişliğinde bu sonsuz dizi bir yerde kesilmek zorunda kalır, ve kesilen kısım küçük bir yuvarlama hatası olarak kalır. Konuya kayan nokta gösterimini işlerken geri döneceğiz.

Alıştırma 1.2 ★ $(0.375)_{10}$ 'u ardışık çarpma yöntemiyle ikiliye çeviriniz.

Çözüm:

İşlem	Sonuç	Taşan Basamak
0.375×2	0.75	0
0.75×2	1.5	1
0.5×2	1.0	1

Yukarıdan aşağı okunduğunda: $(0.375)_{10} = (0.011)_2$. Doğrulama: $0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.25 + 0.125 = 0.375$. ✓

Örnek 1.22: Sekizlik Kesir Dönüşümü

Ardışık çarpma yöntemi herhangi bir tabana genelleşir; 8'lik tabana dönüşümde kesirli kısım 2 yerine 8 ile çarpılır. $(0.513)_{10}$ 'u sekizliğe çevirelim:

İşlem	Sonuç	Taşan Basamak
0.513×8	4.104	4
0.104×8	0.832	0
0.832×8	6.656	6
0.656×8	5.248	5
0.248×8	1.984	1
0.984×8	7.872	7

Kesir hiç 0'a düşmüyor (beklendiği gibi — her kesir sonlanmaz); 6 basamak hassasiyetle kesersek: $(0.513)_{10} \approx (0.406517)_8$.

Örnek 1.23: Karma Sayı Dönüşümü: Tam ve Kesirli Kısım Birlikte

Hem tam sayı hem kesir içeren bir sayıyı çevirirken, **iki kısım ayrı ayrı çevrilip sonra birleştirilir**; tam kısım için ardışık bölme, kesirli kısım için ardışık çarpma. $(41.6875)_{10}$ 'u ikiliye çevirelim.

Tam kısım (41), ardışık bölme ile:

İşlem	Bölüm	Kalan
$41 \div 2$	20	1 (en düşük anlamlı bit)
$20 \div 2$	10	0
$10 \div 2$	5	0
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1 (en yüksek anlamlı bit)

Kalanlar aşağıdan yukarı: $(41)_{10} = (101001)_2$.

Kesirli kısım (0.6875), ardışık çarpma ile bulunur (bunu daha önce hesaplamıştık): $(0.6875)_{10} = (0.1011)_2$.

İki sonuç birleştirilir:

$$(41.6875)_{10} = (101001.1011)_2$$

1.3 İkili Aritmetik: Toplama, Çıkarma, Çarpma ve Bölme

Şimdiye kadar sayıları temsil etmeyi ve tabanlar arasında çevirmeyi öğrendik. Bu bölümde bu sayılar üzerinde temel aritmetik işlemleri (toplama, çıkarma, çarpma, bölme) nasıl yapacağımızı ele alıyoruz. r tabanındaki aritmetik işlemler, ilkokulda öğrendiğimiz ondalık kurallarla **birebir aynı mantıkla** işler; tek fark, her basamağın 0'dan $r-1$ 'e kadar sınırlı olması ve elde/borç eşiğinin 10 değil r olmasıdır. Bir sonraki bölümde (Tümleyenler) çıkarmayı toplamaya çeviren bir "trik" öğreneceğiz. Ama önce, optimize edeceğimiz ham işlemi elle nasıl yaptığımızı bilmemiz gerekiyor. Her işlem için önce genel r -tabanlı matematiksel formülü kuracağız, sonra $r = 2$ özel durumunu (ikili) örneklerle pekiştireceğiz; bu formüller, ileride mantık kapıları ile gerçek toplayıcı/çarpıcı/bölücü *devreleri* tasarlarken birebir kullanılacak.

1.3.1 İkili Toplama

Genel Toplama Formülü: n basamaklı, r tabanındaki $A = (a_{n-1} \dots a_1 a_0)_r$ ile $B = (b_{n-1} \dots b_1 b_0)_r$ toplanırken, her basamak i için (sağdan sola, $i=0$ 'dan başlayarak):

$$s_i = (a_i + b_i + c_i) \bmod r, \quad c_{i+1} = \left\lfloor \frac{a_i + b_i + c_i}{r} \right\rfloor$$

Burada c_i , bir önceki basamaktan gelen elde ($c_0=0$). $a_i, b_i, c_i \leq r-1$ olduğundan toplamaları en fazla $2(r-1) + 1 = 2r-1 < 2r$ 'dir; bu yüzden c_{i+1} her zaman 0 veya 1'dir. Hiçbir zaman 2 veya daha büyük bir elde oluşmaz.

İkilide ($r=2$) bu formül $s_i = (a_i + b_i + c_i) \bmod 2$, $c_{i+1} = \lfloor (a_i + b_i + c_i)/2 \rfloor$ 'ye indirgenir; bu, ileride kuracağımız bir "tam toplayıcı" (full adder) mantık kapısı devresinin giriş-çıkış ilişkisinin ta kendisidir. Ondalıkta iki basamağın toplamı 10'u geçince elde doğar; ikilide eşik $r=2$ 'dir: $1+1 = (10)_2$ — yani bu basamağa 0 yazılır, bir üst basamağa 1 elde gider.

Örnek 1.24: $(101101)_2 + (100111)_2$

Sağdan sola, sütun sütun:

Basamak	İşlem	Yaz	Elde
0	$1 + 1$	0	1
1	$0 + 1 + 1$	0	1
2	$1 + 1 + 1$	1	1
3	$1 + 0 + 1$	0	1
4	$0 + 0 + 1$	1	0
5	$1 + 1 + 0$	0	1

Son elde (1) yeni bir basamak olarak yazılır: $(1010100)_2$. Doğrulama: $(101101)_2 = 45$, $(100111)_2 = 39$, $45 + 39 = 84 = (1010100)_2$. ✓ Bu bit-bit prosedür, donanımda birebir bir *tam toplayıcı* (full adder) devresine dönüşür. Devresini ilerideki Kombinasyonel Mantık bölümünde kuracağız.

Alıştırma 1.3 ★ $(1101)_2 + (1011)_2$ işlemini sütun sütun (elde takip ederek) yapınız.

Çözüm:

Basamak	İşlem	Yaz	Elde
0	$1 + 1$	0	1
1	$0 + 1 + 1$	0	1
2	$1 + 0 + 1$	0	1
3	$1 + 1 + 1$	1	1

Son elde (1) yeni basamak olur: $(11000)_2$. Doğrulama: $13 + 11 = 24 = (11000)_2$. ✓

1.3.2 İkili Çıkarma

Genel Çıkarma Formülü: n basamaklı $A - B$ işlemi için, her basamak i 'de:

$$d_i = \begin{cases} a_i - k_i - b_i & \text{eğer } a_i - k_i \geq b_i \\ (a_i - k_i - b_i) + r & \text{eğer } a_i - k_i < b_i \end{cases} \quad k_{i+1} = \begin{cases} 0 & \text{eğer } a_i - k_i \geq b_i \\ 1 & \text{eğer } a_i - k_i < b_i \end{cases}$$

Burada k_i , bir önceki basamaktan gelen borç ($k_0=0$). İkinci durumda ($a_i - k_i < b_i$), mevcut basamağa taban r **eklenerek** (yani "komşudan r birim ödünç alınarak") fark hesaplanır. Ondalıkta bu eklenen değer 10, ikilide 2'dir.

Ondalıkta borç alırken basamağa 10 eklenir; ikilide borç alırken basamağa sadece 2 eklenir. Genel kural: **borcun değeri, her zaman kullanılan tabana eşittir.**

Örnek 1.25: $(101101)_2 - (100111)_2$

Basamak	İşlem	Yaz	Borç?
0	$1 - 1$	0	hayır
1	$0 - 1 \rightarrow (0 + 2) - 1$	1	evet, üstten alındı
2	$(1 - 1) - 1 \rightarrow (0 + 2) - 1$	1	evet
3	$(1 - 1) - 0$	0	hayır
4	$0 - 0$	0	hayır
5	$1 - 1$	0	hayır

Sonuç: $(000110)_2$. Doğrulama: $45 - 39 = 6 = (000110)_2$. ✓

Alıştırma 1.4 ★ $(1101)_2 - (1011)_2$ işlemini sütun sütun (borç takip ederek) yapınız.

Çözüm:

Basamak	İşlem	Yaz	Borç?
0	$1 - 1$	0	hayır
1	$0 - 1 \rightarrow (0 + 2) - 1$	1	evet
2	$(1 - 1) - 0$	0	hayır
3	$(1 - 0) - 1$	0	hayır

Sonuç: $(0010)_2$. Doğrulama: $13 - 11 = 2 = (0010)_2$. ✓

1.3.3 İkili Çarpma

Genel Çarpma Formülü: A (çarpılan) ile n basamaklı $B = (b_{n-1} \dots b_1 b_0)_r$ (çarpan) çarpımı, çarpanın her basamağının ürettiği bir *kısmi çarpımın*, doğru yere kaydırılıp toplanmasıyla bulunur:

$$A \times B = \sum_{i=0}^{n-1} b_i \cdot A \cdot r^i$$

İkilide ($r=2$) çarpan basamakları b_i yalnızca 0 veya 1 olabileceğinden, her terim ($b_i \cdot A \cdot 2^i$) ya tamamen **sıfır** ya da A 'nın i basamak **sola kaydırılmış bir kopyasıdır**. Çarpma işlemi hiçbir zaman gerçek bir çarpım gerektirmez, sadece kaydırma ve toplama. Bu yüzden ikili çarpmaya *kaydır-ve-topla* (shift-and-add) da denir.

Örnek 1.26: $(1011)_2 \times (101)_2$

Çarpanın basamakları (sağdan sola) $b_0=1$, $b_1=0$, $b_2=1$. Her biri için kısmi çarpımı, **za-ten kaydırılmış tam genişliğiyle** yazalım (böylece toplama sıradan basamak-hizalı toplamaya döner):

Terim	Değer	Açıklama
$b_0 \cdot A \cdot 2^0$	001011	A , kaydırmaz
$b_1 \cdot A \cdot 2^1$	000000	$b_1=0$ olduğu için tamamen sıfır
$b_2 \cdot A \cdot 2^2$	101100	A , 2 basamak sola kaydırılmış

Üç terim de artık aynı genişlikte (6 bit); Bölüm 1.3.1'deki sıradan ikili toplama ile eklenir:

$$001011 + 000000 + 101100 = 110111$$

Doğrulama: $(1011)_2 = 11$, $(101)_2 = 5$, $11 \times 5 = 55 = (110111)_2$. ✓

1.3.4 İkili Bölme

Bölme, diğer üç işlemten farklıdır: basamak başına kapalı bir formülle değil, adım adım işleyen bir **algoritma**yla tanımlanır (uzun bölme). Önce ne aradığımızı netleştirelim:

Bölmenin Tanımı: A (bölünen) ve $B \neq 0$ (bölen) verildiğinde, bölme işlemi

$$A = Q \cdot B + R, \quad 0 \leq R < B$$

eşitliğini sağlayan Q (bölüm) ve R (kalan) değerlerini bulmaktır.

Genel Bölme Algoritması (Kaydır-ve-Çıkar): A 'nın basamakları en anlamlıdan (soldan) başlayarak tek tek işlenir. Bir çalışma kaydı R (0'dan başlar) her adımda önce tabana göre kaydırılıp bir sonraki basamak eklenir, sonra bölmele karşılaştırılır:

$$R \leftarrow r \cdot R + a_i \quad \text{ardından} \quad q_i = \max\{q \in \{0, 1, \dots, r-1\} : q \cdot B \leq R\}, \quad R \leftarrow R - q_i \cdot B$$

Yani her adımda, B 'nin kaydırılmış R 'yi aşmayan en büyük katı bulunur; bu katın kaç kez alındığı (q_i) o adımın bölüm basamağıdır, ve o kat R 'den çıkarılır. Onluk gibi $r > 2$ tabanlarda q_i genelde 0 ile $r-1$ arasında herhangi bir değer olabileceği için bu en büyük katı bulmak bir miktar deneme gerektirir (aşağıda somut bir örnekle göreceğiz). İkilide ($r=2$) ise q_i yalnızca 0 veya 1 olabileceğinden bu arayış tek bir soruya indirgenir: **"şu anki R , bölenden büyük veya eşit mi?"** Cevap evetse bölüm biti 1 ve B çıkarılır, hayırsa bölüm biti 0. Bu basitlik (kaydır, karşılaştır, gerekirse çıkar), donanımda *ardışık bölücü* devrelerinin temelini oluşturur.

Önce genel algoritmayı, alışık olduğumuz onluk tabanda (uzun bölme) somutlaştıralım.

Örnek 1.27: $(138)_{10} \div (6)_{10}$: Genel Algoritma Onluk Tabanda

$A = 138$, $B = 6$. Basamaklar soldan sağa (1, 3, 8) tek tek işlenir; her adımda q_i , $q_i \cdot 6$ R 'yi aşmayacak şekilde *deneme yoluyla* bulunur:

Adım	Basamak	$R \leftarrow 10R + a_i$	En büyük $q \cdot 6 \leq R$	Bölüm biti, yeni R
1	1	1	$0 \cdot 6 \leq 1$	$q=0, R=1$
2	3	13	$2 \cdot 6 \leq 13$	$q=2, R=1$
3	8	18	$3 \cdot 6 \leq 18$	$q=3, R=0$

Bölüm basamakları sırayla 0, 2, 3: $Q = (023)_{10} = 23$. Son $R = 0$, kalan yok. Doğrulama: $23 \times 6 = 138$. ✓

Şimdi aynı algoritmayı, karşılaştırmanın tek bir soruya indirildiği $r = 2$ özel durumunda uygulayalım:

Örnek 1.28: $(1011)_2 \div (11)_2$

$A = 1011$ (11), $B = 11$ (3). Bitler soldan sağa işlenir:

Adım	Getirilen bit	$R \leftarrow 2R + a_i$	$R \geq B$ mi?	Bölüm biti, yeni R
1	1	1	$1 < 3$, hayır	$q=0, R=1$
2	0	2	$2 < 3$, hayır	$q=0, R=2$
3	1	5	$5 \geq 3$, evet	$q=1, R=5 - 3 = 2$
4	1	5	$5 \geq 3$, evet	$q=1, R=5 - 3 = 2$

Bölüm bitleri sırayla 0, 0, 1, 1: $Q = (0011)_2 = 3$. Son $R = 2 = (10)_2$, kalan. Doğrulama: $11 = 3 \times 3 + 2$. ✓

Alıştırma 1.5 ★★ $(1110)_2 \div (11)_2$ işlemini yukarıdaki tabloyla aynı yöntemle yapınız.

Çözüm: $A = 1110$ (14), $B = 11$ (3):

Adım	Getirilen bit	$R \leftarrow 2R + a_i$	$R \geq B$ mi?	Bölüm biti, yeni R
1	1	1	$1 < 3$, hayır	$q=0, R=1$
2	1	3	$3 \geq 3$, evet	$q=1, R=3 - 3 = 0$
3	1	1	$1 < 3$, hayır	$q=0, R=1$
4	0	2	$2 < 3$, hayır	$q=0, R=2$

$Q = (0100)_2 = 4$, kalan $R = (10)_2 = 2$. Doğrulama: $14 = 4 \times 3 + 2$. ✓

1.4 Tümleyenler

Şimdiye kadar sayıları bir tabandan diğerine çevirdik. Bu bölümde farklı bir amaca hizmet eden bir araç kuracağız: *tümleyenler*.

Bilgi Notu 1.6: Neden Çıkarmayı Toplamaya Çeviriyoruz?

Bir bilgisayarın işlemcisi, aslında milyonlarca küçük anahtardan (transistörden) örülmüş bir ağıdır. Bu transistörler, ikili girişlerden ikili çıkış üreten *mantık kapılarına* (AND, OR, NOT vb.; Bölüm 2.6’de göreceğimiz kapılara) gruplanır, kapılar da bir araya gelerek *toplayıcı* (adder) gibi aritmetik devreleri oluşturur. Sorun şu: toplama işlemini yapan bir devre ile çıkarma işlemini yapan bir devre, **birbirinden farklı** kapı düzenlemeleri gerektirir; yani ikisini de donanımda gerçeklemek isterseniz iki ayrı devre kurmanız, dolayısıyla iki katı transistör harcamanız gerekir. Ama eğer $M - N$ işlemini $M + (\text{tümleyen}(N))$ olarak yeniden yazabilirsek, çıkarma işlemi de aynı toplayıcı devre üzerinden yürür. Ayrı bir ”çıkarcı” devre tasarlamaya hiç gerek kalmaz. Tümleyenlerin donanım tasarımındaki asıl değeri budur: transistör/kapı sayısını neredeyse yarıya indiren köklü bir basitleştirme. Bu devrenin somut kapı şemasını, toplayıcı devreleri işlediğimizde (ilerideki Toplayıcı Devreler bölümünde) göreceğiz.

1.4.1 Genel Tümleyen Tanımı

Tümleyen Türleri: n basamaklı, r tabanında bir N sayısı için iki tür tümleyen tanımlanır:

- **$(r-1)$ 'in tümleyeni** (diminished radix complement – ikilide ”1’in tümleyeni”, ondalıkta ”9’un tümleyeni”):

$$(r-1)\text{'in tümleyeni} = (r^n - 1) - N$$

- **r 'nin tümleyeni** (radix complement – ikilide ”2’nin tümleyeni”, ondalıkta ”10’un tümleyeni”):

$$r\text{'nin tümleyeni} = r^n - N = [(r-1)\text{'in tümleyeni}] + 1$$

İkinci tanımın ilk tanıma bağlı olması pratik açıdan önemlidir: $(r-1)$ 'in tümleyenini hesaplamak çok kolaydır (aşağıda göreceğiz), r 'nin tümleyeni de ona sadece 1 eklenerek bulunur.

1.4.2 Ondalık Örneklerle Tümleyen Hesabı

Kavramı önce aşına olduğumuz ondalık sistemde ($r=10$) somutlaştıralım. $(r-1)$ 'in tümleyenini, yani 9’un tümleyenini, hesaplamanın pratik yolu: **her basamağı 9’dan çıkarmaktır**; çünkü $(10^n - 1)$ sayısı n tane 9’dan oluşur (999 ... 9), ve basamak basamak çıkarma hiçbir borç gerektirmez.

Örnek 1.29: 546700’ün 9’un ve 10’un Tümleyeni

6 basamaklı 546700 sayısının 9’un tümleyeni, her basamağı 9’dan çıkararak:

$$9-5, 9-4, 9-6, 9-7, 9-0, 9-0 \Rightarrow 453299$$

Doğrulama: $(10^6 - 1) - 546700 = 999999 - 546700 = 453299$. ✓

10’un tümleyeni, 9’un tümleyenine 1 eklenerek bulunur:

$$453299 + 1 = 453300$$

Doğrulama: $10^6 - 546700 = 453300$. ✓

Örnek 1.30: 012398'in 9'un Tümleyeni

Baştaki sıfır da bir basamaktır, aynı kurala tabidir:

$$9-0, 9-1, 9-2, 9-3, 9-9, 9-8 \Rightarrow 987601$$

1.4.3 Kısayol Yöntemi ve İspatı

Bilgi Notu 1.7: Pratik Kısayol: r 'nin Tümleyenini Doğrudan Almak

r 'nin tümleyenini iki adımda (önce $(r-1)$ 'in tümleyenini alıp sonra 1 eklemek yerine) **tek adımda** bulmanın bir kısayolu vardır: sağdan başlayarak **sondaki sıfırları ve ilk sıfır-olmayan basamağı olduğu gibi bırakın**, geri kalan (soldaki) basamakları $(r-1)$ 'den çıkarın.

Örnek 1.31: Kısayolla 246700'ün 10'un Tümleyeni

Sondaki iki 0 aynı kalır; ilk sıfır olmayan basamak (7) 10'dan çıkarılır ($10-7 = 3$); geri kalan basamaklar (2, 4, 6, soldan sağa) 9'dan çıkarılır (7, 5, 3):

$$246700 \Rightarrow 753300$$

Doğrulama: $10^6 - 246700 = 753300$. ✓

Bilgi Notu 1.8: Kısayol Neden İşliyor? Borç Yayılımı İspatı

r 'nin tümleyeni, r^n (yani n basamaklı $1 \underbrace{00 \dots 0}_n$ sayısı) ile N arasındaki farktır: $r^n - N$. Bu çıkarmayı elle, sağdan sola borç alarak yapalım. N 'nin sondan itibaren k tane sıfırı olsun. İlk k basamakta $0 - 0$ işlemi yapılır. Borca hiç gerek yoktur, sonuç 0 kalır. Basamak k 'de (sondan itibaren ilk sıfır-olmayan basamak, değeri d) işlem $0 - d$ 'dir; bu borç gerektirir, sonuçta $(r-d)$ elde edilir ve bir borç sola, basamak $k+1$ 'e, yayılır. Basamak $k+1$ 'de r^n 'nin kendi basamağı da 0 olduğundan, gelen borç yüzünden işlem aslında $(0 - 1) - d_{k+1} = (r-1) - d_{k+1}$ 'dir (yine bir borç daha sola yayılarak); bu, en soldaki basamağa kadar devam eder. Yani ilk sıfır-olmayan basamağın solundaki **her** basamak, gelen borç nedeniyle otomatik olarak $(r-1)$ 'den çıkarılmış olur. Kısayol, bu borç yayılımının doğrudan bir sonucudur.

1.4.4 İkilide Tümleyenler

Aynı tanımlar $r = 2$ için doğrudan uygulanır. $(r-1) = 1$ olduğundan, **1'in tümleyeni, her biti ters çevirmektir** (NOT); çünkü $1 - d_i$ işlemi tam olarak biti tersine çevirir ($1 - 0 = 1$, $1 - 1 = 0$). **2'nin tümleyeni** ise 1'in tümleyenine 1 eklenerek bulunur.

rnek 1.32: $(1011000)_2$ 'nin 1'in ve 2'nin Tmleyeni

Her bit ters evrilir:

$$(1011000)_2 \xrightarrow{\text{NOT}} (0100111)_2 \quad (1'\text{in tmleyeni})$$

Bu sonuca 1 eklenir:

$$0100111 + 1 = 0101000 \quad (2'\text{nin tmleyeni})$$

rnek 1.33: Kısayolla $(1011000)_2$ 'nin 2'nin Tmleyeni

Sağdan başlayarak sondaki sıfırları (000) ve ilk 1'i olduėu gibi bırakırız (...1000), geri kalan sol taraf (101) ters evrilir (010):

$$(1011000)_2 \Rightarrow (0101000)_2$$

Yukarıdaki uzun yntemle bulduėumuz sonula aynı. ✓Bu kısayol, elle hesap yaparken (r. bir sonraki blmdeki ıkarma rneklerinde) ciddi zaman kazandırır.

1.4.5 Sekizlik ve Onaltılıkta $(r-1)$ 'in Tmleyeni

Aynı mantık her tabana genelleřir: sekizlikte $(r-1) = 7$ olduėundan, $(r-1)$ 'in tmleyeni **her basamaėı 7'den ıkararak** bulunur; onaltılıkta $(r-1) = 15 = F$ olduėundan, **her basamak F'den ıkarılır** (harf basamaklarını nce Tablo 1.1 ile sayıya evirmeyi, sonucu da gerekirse tekrar harfe evirmeyi unutmayın).

rnek 1.34: $(5643)_8$ 'n 7'nin Tmleyeni

Her basamak 7'den ıkarılır:

$$7-5, 7-6, 7-4, 7-3 \Rightarrow 2134$$

Yani $(5643)_8$ 'n 7'nin tmleyeni $(2134)_8$ 'dir. Doėrulama: $(7777)_8 - (5643)_8$ ondalıkta $4095 - 2979 = 1116$, ve 1116'nın sekizlik karřılıėı $(2134)_8$ 'tr. ✓

rnek 1.35: $(3A9)_{16}$ 'un F'nin Tmleyeni

Her basamak (harfler nce sayıya evrilerek) $F(= 15)$ 'ten ıkarılır: $15-3 = 12 = C$, $15-10 = A \rightarrow 5$, $15-9 = 6$:

$$(3A9)_{16} \Rightarrow (C56)_{16}$$

Doėrulama: $(FFF)_{16} - (3A9)_{16}$ ondalıkta $4095 - 937 = 3158$, ve 3158'in onaltılık karřılıėı $(C56)_{16}$ 'dir. ✓

1.4.6 Kesirli (Noktalı) Sayılarda Tmleyen Alma

Buraya kadarki tm tanımlar, sayının bir radix noktası (ondalık/ikili nokta) iermediėini varsaydı. Eėer N 'nin bir noktası varsa, tmleyeni almadan nce nokta **geici olarak kaldırılır**,

sayı sanki tam sayıymış gibi tümleyeni alınır, ve nokta **aynı görelî konumuna geri konur**.

Örnek 1.36: (546.700)'ün 9'un Tümleyeni

Nokta geçici olarak kaldırılır: 546700. Bunun 9'un tümleyeni (daha önce hesaplamıştık): 453299. Nokta, sondan üçüncü basamağın önündeki aynı görelî konumuna geri konur:

$$(546.700) \Rightarrow (453.299)$$

Örnek 1.37: $(101.11)_2$ 'nin 2'nin Tümleyeni

Nokta kaldırılır: 10111 (5 basamak). 2'nin tümleyeni: 1'in tümleyeni 01000, $+1 = 01001$. Nokta, sondan iki basamağın önündeki aynı görelî konuma geri konur:

$$(101.11)_2 \Rightarrow (010.01)_2$$

Yöntem tam sayılarla tamamen aynıdır. Tek fark, işlem bitince noktanın eski yerine konmasıdır.

Tümleyenin Tümleyeni: r 'nin tümleyeni tanımı gereği $r^n - N$ olduğundan, bu sonucun bir kez daha r 'nin tümleyenini almak

$$r^n - (r^n - N) = N$$

verir; yani **tümleyenin tümleyeni, sayının kendisini geri verir**. Bu, ilerideki çıkarma prosedüründe "sonuç negatifse, bulduğunuz değerin bir kez daha tümleyenini alıp başına eksi koyun" adımının neden doğru sonucu verdiğini açıklar: negatif durumda elde ettiğimiz ara sonuç, aslında gerçek büyüklüğün tümleyenidir; tümleyenini tekrar almak bizi gerçek büyüklüğe geri götürür.

1.4.7 r 'nin Tümleyeniyle Çıkarma İşlemi

Tümleyenlerin asıl amacına gelelim: $M - N$ çıkarma işlemini toplamaya çevirmek. n basamaklı M ve N için kural şöyledir:

r 'nin Tümleyeniyle Çıkarma Kuralı:

1. $M + [r$ 'nin tümleyeni(N)] toplamını hesaplayın.
2. Sonuç n basamaktan **taşarsa** (bir "uç elde" oluşursa), taşan basamağı **atın**. Kalan n basamak doğrudan $M - N$ 'dir ve sonuç **pozitif** ($M \geq N$ demektir).
3. Sonuç n basamaktan **taşmazsa**, elde ettiğiniz n basamaklı sonuç aslında $N - M$ 'nin r 'nin tümleyenidir ($M < N$ demektir). Gerçek büyüklüğü bulmak için bu sonucun bir kez daha r 'nin tümleyenini alın ve başına eksi işareti koyun.

rnek 1.38: Kuralın Doğrudan Uygulanışı: $(48)_{10} - (25)_{10}$

$n = 2$ basamak, $M = 48$, $N = 25$.

Adım 1: $M + [10^{\text{'un tmleyeni}(N)}]$ toplamını hesaplayın.

$$10^{\text{'un tmleyeni}(25)} = 100 - 25 = 75$$

$$M + \text{tmleyen}(N) = 48 + 75 = 123$$

Adım 2: Sonuç 2 basamaktan taşıyor mu diye bakın. 123'n en soldaki 1'i bir "ç elde"dir, yani sonuç 2 basamaktan taşmıřtır.

Adım 3: Tařtıđı iin, tařan basamak atılır. Kalan 23 doğrudan $M - N$ 'dir:

$$48 - 25 = 23 \quad \boxtimes$$

Bilgi Notu 1.9: nce Basamak Sayılarını Eřitleyin

Bu kural, M ve N 'nin **aynı sayıda basamađa** (n) sahip olmasını varsayar. Tmleyen tanımını da $(r^n - N)$ zaten belirli bir n 'ye gre yapılır. Sayıların basamak sayıları farklıysa, kısa olanın **bařına sıfır eklenerek** uzun olanla eřitlenir. Ařađıdaki ilk rnekte $N = 3250$ 'nin 4 basamađı var ama $M = 72532$ 'nin 5; bu yzden N 'yi 03250 olarak yazıp $n = 5$ zerinden iřlem yapıyoruz.

rnek 1.39: Ondalıkta ıkarma: Tařma Var, Pozitif Sonuç

$72532 - 03250$ iřlemine ($n = 5$ basamak) tmleyenle yapalım. $N = 03250$ 'nin 10'un tmleyeni: $100000 - 03250 = 96750$.

$$M + \text{tmleyen}(N) = 72532 + 96750 = 169282$$

Bu, 5 basamaklı sistemde bir "ç elde" (tařan 1) retir. Tařan basamak atılır:

$$72532 - 3250 = 69282$$

Dođrulama: $72532 - 3250 = 69282$. ✓

rnek 1.40: Ondalıkta ıkarma: Tařma Yok, Negatif Sonuç

Aynı sayılarla ters ynde, $03250 - 72532$ iřlemine yapalım. $N = 72532$ 'nin 10'un tmleyeni: $100000 - 72532 = 27468$.

$$M + \text{tmleyen}(N) = 03250 + 27468 = 30718$$

Bu 5 basamađa sığıyor, ç elde **yok**. Demek ki gerek sonuç negatif: 30718'in 10'un tmleyenini alıp bařına eksi koyuyoruz:

$$100000 - 30718 = 69282 \Rightarrow \text{sonuç} = -69282$$

Dođrulama: $3250 - 72532 = -69282$. ✓

Örnek 1.41: İkilide Çıkarma: 2'nin Tümleyeniyle, Taşma Var

8 bit üzerinde $(01010100)_2 = 84$ 'ten $(01000011)_2 = 67$ 'yi çıkaralım. N 'nin 2'nin tümleyeni (kısayolla): 10111101 .

$$01010100 + 10111101 = \underset{\text{uç elde}}{\overset{1}{1}} 00010001$$

Uç elde atılır: sonuç $(00010001)_2 = 17$. Doğrulama: $84 - 67 = 17$. ✓

Örnek 1.42: İkilide Çıkarma: 2'nin Tümleyeniyle, Taşma Yok

Ters yönde, $67 - 84$ işlemini yapalım. $84 = (01010100)_2$ 'nin 2'nin tümleyeni: 10101100 .

$$01000011 + 10101100 = 11101111$$

8 bite sığıyor, uç elde yok. Sonuç negatif. $(11101111)_2$ 'nin 2'nin tümleyeni: $00010001 = 17$. Sonuç: -17 . Doğrulama: $67 - 84 = -17$. ✓

1.4.8 $(r-1)$ 'in Tümleyeniyle Çıkarma: Uçtan-Dolaşan Elde

$(r-1)$ 'in tümleyeni (9 'un veya 1 'in tümleyeni) ile çıkarma yapmak isterseniz, kural bir noktada farklılaşır: uç elde oluştuğunda bu sefer **atılmaz**, bunun yerine sonucun en düşük anlamlı basamağına **eklenir**. Buna *uçtan-dolaşan elde* (end-around carry) denir. Bunun nedeni, $(r-1)$ 'in tümleyenin r 'nin tümleyeninden tam 1 eksik olması: kaybolan bu farkı telafi etmek için taşan biti geri "dolaştırıyoruz".

Örnek 1.43: Ondalıkta 9'un Tümleyeniyle Çıkarma

Aynı $72532 - 3250$ işlemi, bu kez 9 'un tümleyeniyle. $N = 03250$ 'nin 9 'un tümleyeni: 96749 .

$$M + \text{tümleyen}(N) = 72532 + 96749 = 169281$$

Uç elde (1) atılmaz, en düşük basamağa eklenir:

$$69281 + 1 = 69282$$

Aynı sonuç (r 'nin tümleyeni yöntemiyle bulduğumuzla özdeş). ✓

Örnek 1.44: İkilide 1'in Tümleyeniyle Çıkarma

$84 - 67$ işlemini bu kez 1 'in tümleyeniyle yapalım. $67 = (01000011)_2$ 'nin 1 'in tümleyeni: 10111100 .

$$01010100 + 10111100 = \underset{\text{uç elde}}{\overset{1}{1}} 00010000$$

Uç elde atılmaz, en düşük basamağa eklenir: $00010000 + 1 = 00010001 = 17$. Doğrulama: $84 - 67 = 17$. ✓

Alıştırma 1.6 ★★ $67 - 84$ işlemini 8 bit üzerinde **1'in tümleyeni** yöntemiyle (uçtan-dolaşan

elde kuralına dikkat ederek) çözünüz.

Çözüm: $84 = (01010100)_2$ 'nin 1'in tümleyeni: 10101011 .

$$01000011 + 10101011 = 11101110$$

8 bite sığıyor, uç elde **yok**. Bu durumda uçtan-dolaşan elde kuralı devreye girmez, sonuç doğrudan negatiftir. $(11101110)_2$ 'nin 1'in tümleyeni: $00010001 = 17$. Sonuç: -17 . Doğrulama: $67 - 84 = -17$. ✓

1.5 İşaretili İkili Sayılar

Şimdiye kadar gördüğümüz her sayı *işaretsizdi*; yalnızca sıfır ve pozitif değerleri temsil ediyordu. Ama gerçek dünyada negatif sayılara da ihtiyacımız var. Sıradan aritmetikte bunu bir "–" sembolüyle çözeriz; donanımda ise elimizde yalnızca bitler var, sembol diye bir şey yok. Bu bölümde, bir önceki bölümde kurduğumuz tümleyen aracını kullanarak, negatif sayıları saf bitlerle nasıl temsil edeceğimizi göreceğiz.

1.5.1 İşaret Biti ve Yorumlama

İşaret Biti: İşaretili bir ikili sayıda, en soldaki bit *işaret bitidir*: 0 pozitif, 1 negatif anlamına gelir. Kalan bitler büyüklüğü (ya da tümleyen sistemindeyse, aşağıda göreceğimiz gibi farklı bir şeyi) temsil eder.

Bilgi Notu 1.10: Aynı Bitler, Farklı Anlam

Bir bit dizisinin işaretili mi işaretsiz mi olduğu, dizinin kendisinde yazılı değildir; **yorumlayan tarafından bilinmesi gerekir**. Örneğin 01001 : işaretsiz okunursa 9, işaretili okunursa $+9$ 'dur (soldaki bit 0 olduğu için ikisi de aynı büyüklüğü verir). Ama 11001 : işaretsiz okunursa 25, işaretili okunursa -9 'dur. En soldaki 1 işaret biti olarak yorumlandığında, kalan 1001 (9) büyüklüğünün negatifini gösterir.

1.5.2 İşaret-Büyüklük Gösterimi

En sezgisel yöntem, sıradan aritmetikle birebir aynıdır: işaret biti ayrı, büyüklük ayrı saklanır.

Örnek 1.45: -5 'in 8 Bitte İşaret-Büyüklük Gösterimi

$+5 = 00000101$. İşaret-büyüklükte -5 , sadece işaret bitini 1 yaparak elde edilir:

$$-5 = 10000101$$

1.5.3 İşaretili Tümleyen Sistemi: 1'in ve 2'nin Tümleyeni

İşaret-büyüklük sezgisel olsa da, donanımda kullanılan asıl yöntem farklıdır: negatif sayı, pozitif sayının **tümleyeni** alınarak elde edilir; işaret biti dahil, **sayının tamamının** tümleyeni.

Örnek 1.46: -9 'un 8 Bitte Üç Gösterimi

$+9 = 00001001$ (8 bite tamamlamak için işaret bitinden sonra gerektiği kadar 0 eklenir).

Gösterim	-9
İşaret-büyüklik (sadece işaret biti değişir)	10001001
İşaretli 1'in tümleyeni (tüm bitler ters çevrilir)	11110110
İşaretli 2'nin tümleyeni (1'in tümleyenine +1)	11110111

Tablonun ikinci ve üçüncü satırı arasında dikkat edilmesi gereken bir nokta var: 1'in tümleyenine (11110110) eklenen +1, işaret bitini değiştirmemiştir; her iki satırda da en soldaki bit 1'dir. Bunun nedeni, +1 eklemenin sağdan başlayan bir elde zinciri üretmesi ve bu zincirin (burada $0 \rightarrow 1$ dönüşümüyle) daha ilk basamakta sönmesi; işaret bitine hiç ulaşmaz. Sayının negatif olduğunu gösteren işaret zaten bir önceki adımda, tüm bitlerin (işaret biti dahil) ters çevrilmesiyle oluşmuştu; +1 eklemek yalnızca büyüklüğü küçük bir miktar düzeltir, işareti bozamaz.

Bilgi Notu 1.11: İpucu: Ağırlıklı Toplamla Doğrudan Değer Okuma

2'nin tümleyeni gösterimindeki bir sayının değerini bulmak için her zaman "ters çevir, 1 ekle, başına eksi koy" yapmak gerekmez. Doğrudan bir kısayol vardır: n -bitlik sayı $b_{n-1} \dots b_1 b_0$ için, **en soldaki (işaret) bit negatif ağırlıklı** sayılırsa, değer doğrudan ağırlıklı toplamla okunur:

$$V = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

Örnek 1.47: Kısayolla $(11011)_2$ 'nin Değerinin Bulunması

İşaret biti ($b_4 = 1$) negatif ağırlıklı sayılır:

$$-1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -16 + 8 + 0 + 2 + 1 = -5$$

Yani $(11011)_2 = -5$ (2'nin tümleyeni gösteriminde).

Bu kısayolun neden çalıştığını görelim. Bölüm 1.4'de r 'nin tümleyenini $r^n - N$ olarak tanımlamıştık. n -bit 2'nin tümleyeninde negatif bir V değeri, bit kalıbı $X = 2^n + V$ olan işaretlessiz bir sayı olarak saklanır (yani $V = X - 2^n$). Bit kalıbının işaretlessiz değeri $X = \sum_{i=0}^{n-1} b_i 2^i$ dir; işaret biti $b_{n-1} = 1$ olduğundan $X = 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$ dir. Buradan:

$$\begin{aligned} V &= X - 2^n = 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i - 2^n \\ &= -2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i \end{aligned}$$

Bu, tam olarak yukarıdaki kısayol formülüdür.

Örnek 1.48: Kısayolun Doğrulanması: -9 'un 2 'nin Tümleyeni Gösterimi

Az önce -9 'un 8-bit 2 'nin tümleyeni gösterimini 11110111 olarak bulmuştuk. Aynı kısayolu bu bit kalıbına uygulayalım:

$$\begin{aligned} & -1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ & = -128 + 64 + 32 + 16 + 0 + 4 + 2 + 1 = -9 \end{aligned}$$

Doğrulandı.

Alıştırma 1.7 ★ Kısayolu kullanarak $(10101)_2$ 'nin (5-bit 2 'nin tümleyeni) değerini bulunuz.

Çözüm: Bitler soldan sağa $b_4b_3b_2b_1b_0 = 10101$, yani $b_4=1$, $b_3=0$, $b_2=1$, $b_1=0$, $b_0=1$. En soldaki (işaret) bit b_4 negatif ağırlıklı sayılarak kısayol formülü kurulur:

$$V = -b_4 \cdot 2^4 + b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0$$

Bit değerleri yerine konur:

$$V = -1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Her terim ayrı ayrı hesaplanır:

$$V = -16 + 0 + 4 + 0 + 1$$

Toplanır:

$$V = -11$$

1.5.4 Neden 2 'nin Tümleyeni Tercih Edilir

Tablo 1.2'ten iki gözlem çıkar:

Bilgi Notu 1.12: Sebep 1: Tek Sıfır

İşaret-büyüklik ve 1 'in tümleyeninde **hem** $+0$ **hem** -0 var. 16 kombinasyonun biri israf ediliyor, ayrıca "sıfır mı değil mi" karşılaştırması iki farklı bit dizisini kontrol etmeyi gerektiriyor. 2 'nin tümleyeninde sıfırın **tek** bir gösterimi var; bu yüzden bir negatif değer daha (-8) sığdırılabilir.

Bilgi Notu 1.13: Sebep 2 (Asıl Önemli Olan): Aritmetik Kolaylığı

İşaret-büyüklikte iki sayıyı toplamak için önce işaretlerini **karşılaştırmanız** gerekir: işaretler aynıysa büyüklükleri toplayıp ortak işareti verirsiniz; işaretler farklıysa küçük büyüklüğü büyükten **çıkartıp** büyük olanın işaretini verirsiniz. Bu, ayrı bir karşılaştırma ve çıkarma devresi gerektirir. 2 'nin tümleyeninde ise kural tek cümledir: **işaret biti dahil her şeyi normal topla, işaret basamağından taşan biti at**. Karşılaştırma yok, özel durum yok. Bir sonraki iki alt bölümde bunu somutlaştıracacağız. Bu yüzden

Ondalık	2'nin Tümleyeni	1'in Tümleyeni	İşaret-Büyüklik
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Tablo 1.2: 4-Bit İşaretli Sayılar: Üç Gösterimin Karşılaştırması

neredeyse tüm bilgisayarların aritmetik birimleri 2'nin tümleyeni sistemini kullanır.

1.5.5 İşaretli Toplama

İşaretli Toplama Kuralı (2'nin Tümleyeni): İki işaretli sayı, **işaret bitleri dahil** sıradan ikili toplama ile toplanır. İşaret basamağından taşan bir uç elde oluşursa, **atılır**.

Örnek 1.49: Dört Durum, 8 Bit Üzerinde

Aşağıdaki tabloda negatif sayılar (-6 ve -13) zaten kendi **2'nin tümleyeni** gösterimleriyle verilmiştir; yani 11111010, -6 'nın bit dizisidir (bir önceki alt bölümde gördüğümüz yöntemle hesaplanmıştır), rastgele bir dizi değildir. Yeni başlayan biri için bu netleştirme önemlidir: tabloya bakarken "bu 1'lerle dolu dizi nereden çıktı" diye düşünmeyin. Her zaman önce pozitif sayının ikili karşılığını yazıp sonra (gerekliyse) 2'nin tümleyenini alarak elde edilirler.

İşlem	Toplam	Sonuç
$(+6) + (+13)$: 00000110 + 00001101	00010011	+19
$(-6) + (+13)$: 11111010 + 00001101	1 00000111 (uç elde atılır)	+7
$(+6) + (-13)$: 00000110 + 11110011	11111001	-7
$(-6) + (-13)$: 11111010 + 11110011	1 11101101 (uç elde atılır)	-19

Her dört durumda da kural aynı: işaret dahil topla, taşarsa at. Sonucun negatif mi pozitif mi olduğu, sonucun kendi işaret bitinden (0/1) doğrudan okunur.

Bilgi Notu 1.14: Taşma (Overflow)

n bitlik iki sayının toplamı, n bite sığmayan (yani $n+1$ bit gerektiren) bir sonuç üretirse **taşma** oluşur. Kağıt üzerinde sorun değildir (sınırsız genişletebilirsiniz), ama donanımda bit genişliği sabittir, taşan sonuç kayba uğrar. Örneğin 4 bitlik (aralık -8 ile $+7$) bir sistemde $(+5) + (+4) = 0101 + 0100 = 1001$ işlemini yapalım: sonuç 1001, işaret biti 1 olduğu için *negatif* okunur; 2'nin tümleyeninde $1001 = -7$. Ama $5 + 4 = 9$ pozitif olmalıydı! Sonuç 4 bite sığmadığı ($9 > 7$, aralığın üst sınırını aştığı) için taşma oluştu ve anlamsız bir işaret üretti. Taşmanın devrede nasıl otomatik tespit edildiğini, toplayıcı devresini kurduğumuzda göreceğiz.

Alıştırma 1.8 ★ 4 bit üzerinde $(+3) + (-5)$ işlemini 2'nin tümleyeni kuralıyla yapınız.

Çözüm:

Adım 1 (her iki sayıyı da 4 bitte, işaretli biçimde yazın): $+3 = 0011$. -5 için önce $+5 = 0101$ yazılır, sonra 2'nin tümleyeni alınır: 1'in tümleyeni 1010, buna $+1$ eklenir, $-5 = 1011$.

Adım 2 (işaret bitleri dahil sıradan ikili toplama yapın):

$$0011 + 1011 = 1110$$

Adım 3 (uç elde var mı kontrol edin): Sonuç (1110) 4 bite sığıyor, uç elde **yok**.

Adım 4 (sonucu yorumlayın): İşaret biti 1 olduğu için sonuç negatif; bu durumda bit dizisi doğrudan büyüklüğü göstermez, kendisi bir 2'nin tümleyeni gösterimidir. Gerçek büyüklüğü bulmak için 1110'nin 2'nin tümleyenini alırız: 1'in tümleyeni 0001, buna $+1$ eklenir, $0010 = 2$. Dolayısıyla sonuç -2 'dir.

Doğrulama: $3 - 5 = -2$. ✓

Alıştırma 1.9 ★★ 4 bit üzerinde $(-3) + (+5)$ işlemini yapınız. Bu kez sonuçta gerçek bir **uç elde** oluşacak, atılması gerekecek.

Çözüm: -3 'ün 2'nin tümleyeni: $+3 = 0011$, 1'in tümleyeni 1100, $+1 = 1101$. $+5 = 0101$.

$$1101 + 0101 = 10010$$

Bu 5 bit. 4 bitlik sistemde bir **uç elde** (1) oluştu, atılır. Kalan $0010 = 2$. İşaret biti 0, yani pozitif: sonuç $+2$. Doğrulama: $-3 + 5 = 2$. ✓

1.5.6 İşaretili Çıkarma

İşaretili Çıkarma Kuralı (2'nin Tümleyeni): Çıkanın (çıkarılacak sayının) **2'nin tümleyenini** (işaret biti dahil) alın ve eksilene **toplayın**. İşaret basamağından taşan uç elde atılır.

Bu kural, çıkarmayı toplamaya indirger: $(\pm A) - (+B) = (\pm A) + (-B)$ ve $(\pm A) - (-B) = (\pm A) + (+B)$. Bir pozitif sayıyı negatife çevirmek (ya da tam tersi) zaten 2'nin tümleyenini almaktan ibaret olduğu için, çıkarma her zaman "tümleyen al, topla" işlemine dönüşür.

Örnek 1.50: $(-6) - (-13)$, 8 Bit Üzerinde

$-6 = 11111010$, $-13 = 11110011$. Çıkanın (-13) 2'nin tümleyeni alınır (bu, $+13$ 'ü verir): 00001101 . Eksilene (-6) eklenir:

$$11111010 + 00001101 = 1\ 00000111$$

Uç elde atılır: sonuç $00000111 = +7$. Doğrulama: $(-6) - (-13) = 7$. ✓

Örnek 1.51: $(+9) - (+13)$, 8 Bit Üzerinde: Uç Eldesiz Durum

$+9 = 00001001$, $+13 = 00001101$. Çıkanın $(+13)$ 2'nin tümleyeni alınır (bu, -13 'ü verir): 11110011 . Eksilene $(+9)$ eklenir:

$$00001001 + 11110011 = 11111100$$

Bu kez 8 bite sığıyor, uç elde **yok**; ama bu, bir önceki bölümdeki işaretsiz tümleyen çıkarmasından farklı olarak burada bir sorun **değildir**: işaretli çıkarmada kural her zaman aynıdır (varsa uç eldeyi at), uç eldenin olup olmaması sonucun işaretini etkilemez, sonuç zaten kendi işaret bitiyle doğru okunur. 11111100 'ün işaret biti 1, yani negatif; 2'nin tümleyeni $00000100 = 4$, sonuç -4 . Doğrulama: $9 - 13 = -4$. ✓

Alıştırma 1.10 ★ $(+15) - (+6)$ işlemini 8 bit üzerinde 2'nin tümleyeni kuralıyla yapınız.

Çözüm: $+15 = 00001111$. Çıkanın $(+6)$ 2'nin tümleyeni: $+6 = 00000110$, 1'in tümleyeni 11111001 , $+1 = 11111010$ (-6). Eksilene eklenir:

$$00001111 + 11111010 = 1\ 00001001$$

Uç elde atılır: sonuç $00001001 = +9$. Doğrulama: $15 - 6 = 9$. ✓

Alıştırma 1.11 ★★ $(+4) - (+9)$ işlemini 8 bit üzerinde 2'nin tümleyeni kuralıyla yapınız.

Çözüm: $+4 = 00000100$. Çıkanın $(+9)$ 2'nin tümleyeni: $+9 = 00001001$, 1'in tümleyeni 11110110 , $+1 = 11110111$ (-9). Eksilene eklenir:

$$00000100 + 11110111 = 11110111$$

8 bite sığıyor, uç elde yok — önceki örnekte gördüğümüz gibi bu bir sorun değil, kural aynı şekilde işliyor. İşaret biti 1, yani negatif: 11110111 'in 2'nin tümleyeni $00000101 = 5$, sonuç -5 . Doğrulama: $4 - 9 = -5$. ✓

Bilgi Notu 1.15: Büyük Resim: Tek Bir Devre Yeter

İşaretli sayılar, işaretsiz sayılarla **birebir aynı toplama donanımını** kullanır. Çıkarma bile aynı toplayıcı üzerinden, sadece bir tümleyen alma adımıyla yürür. Bölüm 1.4'in başında sorduğumuz soruya ("neden çıkarmayı toplamaya çeviriyoruz?") artık tam cevabımız var: hem işaretsiz hem işaretli aritmetik, aynı tek toplayıcı devre üzerinden, iki ayrı devre kurmaya hiç gerek kalmadan yürüyor. Bu, neredeyse tüm bilgisayarların aritmetik birimlerinde 2'nin tümleyeni sisteminin evrensel olarak kullanılmasının temel

nedenidir.

1.6 IEEE 754 ile Kayan Noktalı Sayı Gösterimi

Bölüm 1.2.9’da, bazı kesirli sayıların ikilide hiçbir zaman sonlanmadığını görmüş, bu durumun kayan noktalı gösterimle olan ilişkisini ise ileriki bir bölüme bırakmıştık. Şimdi sırası geldi: o zaman bahsettiğimiz konuya, kayan noktalı sayı gösterimine, bu bölümde değineceğiz.

Kayan noktalı gösterimin temelinde üç kavram vardır: bir **işaret**, normalize edilmiş bir **anlamli kısım** (mantissa) ve ölçeği belirleyen bir **üs**. Bu üç kavramdan en alışık olmadığımız kısım üstür, çünkü IEEE 754 standardında üs, ham (işaretli) hâliyle değil, *bias* (kaydırma) adı verilen sabit bir değer eklenerek saklanır: gerçek üse *bias* eklenip elde edilen yeni değere *biaslı üs* denir, ve bit dizisinde fiilen saklanan değer budur, gerçek üs değil. Örneğin tek duyarlıklı formatta *bias* 127’dir; gerçek üs -1 ise, saklanan *biaslı üs* $-1 + 127 = 126$ olur. Bu tercihin mühendislik gerekçesini Bölüm 1.6.3’te göreceğiz; şimdilik terimin ne anlama geldiğini bilmemiz yeterlidir.

Bilgi Notu 1.16: Üç Terimin Kökeni: Mantissa, Bias, Üs

Bu üç terimin öyküsü, birbirinden bağımsız üç dilden gelip burada bir araya geliyor. **Mantissa**, Etrüsk kökenli bir Latince sözcüktür; aslında ”önemsiz bir ek, kefevi tamamlamak için eklenen ıvır zıvır” anlamına gelir. 17. yüzyılda logaritma cetvelleri hazırlanırken, bir logaritmanın tam sayı kısmına (”karakteristik”, ondalık noktanın konumunu belirler) karşılık, ondalık kısmına bu ad verilmişti; çünkü tam sayı kısmının yanında önemsiz bir ”ek” gibi görünüyordu. Oysa asıl anlamli basamaklar tam da o kısımdaydı: en önemli parça, en önemsiz görünen adı almış oldu. Bu köken uyuşmazlığı yüzünden IEEE 754 standardının kendisi bile ”mantissa” yerine ”significand” (anlamli kısım) terimini önerir; çünkü mantissa tarihsel olarak bir logaritmanın ondalık kısmı içindi, kayan nokta gösterimi ise logaritmik değildir.

Bias, Eski Fransızca ”biais” (eğik, çapraz) sözcüğünden İngilizceye geçmiştir; bilye oyunlarında topun içine kasıtlı asimetrik ağırlık konarak düz değil eğri gitmesini sağlamak anlamında kullanılıyordu. IEEE 754’teki kullanımı bu köke tam uyar: gerçek üs değeri bir sabitle ”eğilir” (kaydırılır), böylece işaretsiz tam sayı gibi karşılaştırılabilir hâle gelir.

Üs ise üçünün en şaşırtıcısıdır. Sözcük Arapça ”’uss” kökünden gelir ve doğrudan ”temel, taban, esas” anlamına gelir (Aramice/Süryanice ”Awş” ve Akatça ”uşşu” ile aynı kökten; her ikisi de ”temel” demektir); Türkçedeki ilk yazılı kaydı 1680 tarihlidir. Askeri ”üs” (karargâh, temel nokta) ile matematikteki ”üs” tam olarak aynı kelimedir. İlginç olan şudur: Türkçe matematik terminolojisinde ”taban” zaten ayrı bir terim olarak 2^3 ’teki 2’yi karşılarken, ”üs” kelimesinin kendisi etimolojik olarak neredeyse tam bu anlamı taşır, yani kuvveti değil ”temel/tabana” anlamını. Ayrıca ”üs” ile ”üst” (yukarı, üzerinde) sıkça karıştırılır, ama bu ikisinin kökeni bile ortak değildir: ”üst” muhtemelen Eski Türkçe ”üze” (üzerinde) kökünden gelir, ”üs” ise yukarıda anlatıldığı gibi tamamen ayrı, Arapça kökenli bir sözcüktür. Benzerlik yalnızca yazılıştan kaynaklanır.

1.6.1 Neden Sabit Nokta Yetmez?

Bölüm 1.2.9’da öğrendiğimiz kesirli gösterimde, ikili noktanın konumu **sabittir**. Örneğin 8 bitli tam sayı kısmına, 8 bitli kesir kısmına ayırdığımızı düşünelim: bu düzende hem çok büyük bir sayıyı (milyonlarca gibi) hem çok küçük, hassas bir kesri (milyonda bir gibi) aynı anda temsil edemeyiz. Tam kısma ayrılan 8 bit en fazla 255’e kadar sayı tutabilir, kesir kısmına ayrılan 8 bit de belirli bir hassasiyetin ötesine hiç geçemez. Yani sabit noktalı gösterimde **aralık ile hassasiyet arasında sabit, değiştirilemez bir ödünleşim** vardır.

Kayan noktalı gösterim bu sorunu, bilimsel gösterimden ödünç alınan bir fikirle çözer: çok büyük bir sayıyı 1.234×10^5 olarak, çok küçük bir sayıyı da 6.02×10^{-23} olarak yazabildiğimiz gibi, sayıyı doğrudan saklamak yerine **üç parçaya ayırıp** saklarız: işaret, normalize edilmiş anlamlı kısım ve üs. Aynı sayıda bit ile, üs değeri gerektiğinde noktayı çok sola kaydırarak çok küçük sayıları, gerektiğinde çok sağa kaydırarak çok büyük sayıları temsil etmemizi sağlar. Sabit noktalı gösterimden farklı olarak, noktanın sayı içindeki gerçek konumu artık tek bir yere sabitlenmemiştir: hangi basamağın önünde durduğu, her sayı için üs değerine göre değişir, yani noktanın konumu sayıdan sayıya hareket eder. Gösterimin adı da buradan gelir: nokta tek bir yerde sabit durmaz, "kayar" (yani İngilizcesiyle "float"lar).

Kayan Noktalı Sayının Genel Formu:

$$(-1)^S \times 1.M \times 2^{E-\text{bias}}$$

Burada S işaret biti (0 pozitif, 1 negatif), M mantissa (anlamlı kısmın noktadan sonraki basamakları), E saklanan biaslı üs, ve bias bir önceki paragrafta tanımladığımız sabit kaydırma değeridir.

1.6.2 Normalizasyon: Bedava Bir Bit

Normalizasyon, bir sayıyı $1.xxxx \times 2^k$ biçimine getirmek için ikili noktayı kaydırma işlemidir. Kaydırma yönü tek bir kurala bağlıdır: amaç, noktayı her zaman sayının **ilk** (en soldaki) 1’inin hemen sağına getirmektir; nokta hangi yönde bu konuma daha yakınsa o yöne kaydırılır.

- Sayının tam kısmında birden fazla basamak varsa (yani sayı 2’ye eşit ya da ondan büyükse), ilk 1 noktanın **solundadır**. Nokta, bu 1’in hemen sağına gelene kadar **sola** kaydırılır. Kaydırılan basamak sayısı, üsse **pozitif** olarak yansır (çünkü sayıyı eski hâline getirmek için sonradan 2’nin o kadar kuvvetiyle çarpmamız gerekir).
- Sayı 1’den küçükse (yani $0.xxxx$ biçimindeyse), ilk 1 noktanın **sağında**, bir veya daha fazla sıfırdan sonra gelir. Nokta, bu 1’in hemen sağına gelene kadar **sağa** kaydırılır. Kaydırılan basamak sayısı, üsse **negatif** olarak yansır.

Her iki durumda da üs, noktanın kaç basamak ve hangi yönde kaydırıldığının bir kaydırma; aşağıdaki örnekte bu iki durumu da (biri sola, biri sağa kaydırma gerektiren) ayrı ayrı göreceğiz.

Bilgi Notu 1.17: Örtük Baştaki 1

İkilide (sıfır hariç) her sayı, tek bir şekilde $1.xxxx \times 2^k$ biçiminde normalize edilebilir; çünkü ikili sistemde ilk anlamlı basamak için tek bir seçenek vardır: ondalıkta ilk basamak 1’den 9’a kadar herhangi bir değer olabilirken, ikilide 0’dan farklı tek rakam 1’dir.

Bu evrensel gerçek sayesinde, IEEE 754 formatında baştaki bu 1 hiçbir zaman saklanmaz. M (mantissa) alanına yalnızca 1'den sonraki basamaklar yazılır. IEEE 754 standardının kendisi tek duyarlıklı formatta bu M alanına tam olarak 23 bit ayırır (Tablo 1.3); standardın doğrudan belirttiği sayı budur, 24 değil. 24, bizim buradan türettiğimiz bir sonuçtur: saklanan 23 bite, saklanmayan ama normalizasyon sayesinde her zaman var olduğunu bildiğimiz baştaki 1'i (yani 1 bit daha) eklersek, anlamlı kısmın *gerçek* hassasiyeti $23 + 1 = 24$ bit olur. Yani donanımda yalnızca 23 bit harcayarak 24 bitlik bir hassasiyete sahip oluruz: baştaki 1 "örtük" (implicit) olduğu için bedavadan kazanılmış bir bit budur. (Aynı mantıkla, çift duyarlıkta M alanı 52 bit olduğundan gerçek hassasiyet $52 + 1 = 53$ bittir.)

Örnek 1.52: İki Normalizasyon Örneği: Büyük ve Küçük Bir Sayı

Büyük bir sayı: $(1101.011)_2$ 'yi normalize edelim. Nokta 3 basamak sola kaydırılır (ilk 1'in hemen sağına):

$$(1101.011)_2 = 1.101011 \times 2^3$$

Kontrol: $2^3 = 8$ ile çarpmak, ikili noktayı 3 basamak sağa kaydırmakla aynı şeydir (tıpkı onlukta bir sayıyı 10^3 ile çarpmanın, ondalık noktayı 3 basamak sağa kaydırmasına benzer şekilde). 1.101011×2^3 işleminde nokta 3 basamak sağa kayar:

$$1.101011 \xrightarrow{3 \text{ basamak sağa}} 1101.011$$

Bu, normalize etmeden önceki orijinal sayıyla birebir eşleşir. Aynı sonucu ondalık tabanda çarparak da doğrulayabiliriz:

$$1.101011_2 = 1 + 0.5 + 0.125 + 0.03125 + 0.015625 = 1.671875$$

$$1.671875 \times 8 = 13.375$$

$$1101.011_2 = 8 + 4 + 1 + 0.25 + 0.125 = 13.375$$

İki taraf da eşit çıktığından doğrulanmış olur. ✓

Küçük bir sayı: $(0.000101)_2$ 'yi normalize edelim. Nokta 4 basamak sağa kaydırılır (ilk 1'in hemen sağına gelene kadar):

$$(0.000101)_2 = 1.01 \times 2^{-4}$$

Kontrol: $1.01 \times 2^{-4} = 1.25 \times 0.0625 = 0.078125$, ve $(0.000101)_2 = 2^{-4} + 2^{-6} = 0.0625 + 0.015625 = 0.078125$, eşleşiyor.

İki örnek birlikte gösteriyor ki: sayı ne kadar büyükse üs o kadar pozitif, sayı ne kadar küçükse (sıfıra o kadar yakınsa) üs o kadar negatif olur. Kayan nokta, üssü ayarlayarak her iki uca da uzanabilir.

1.6.3 IEEE 754 Formatları ve Bias

Bilgi Notu 1.18: Neden İşaretli Değil, Biaslı Üs?

Üs için 2'nin tümleyeni yerine biaslı gösterim kullanılmasının mühendislik gerekçesi şudur: bu sayede iki kayan noktalı sayı, **sıradan işaretsiz tam sayı karşılaştırıcısıyla** karşılaştırılabilir. Biaslı gösterimde büyüklük sıralaması korunur. 2'nin tümleyeni kullanılsaydı, negatif üsler (en soldaki bit 1) pozitif üslerden büyükmüş gibi görünürdü ve karşılaştırma devresi ek mantık gerektirirdi. Bias değeri, n bitlik üs alanı için $2^{n-1} - 1$ formülüyle seçilir; tek duyarlıkta bu $2^7 - 1 = 127$ değerini verir.

Örnek 1.53: Biaslı Üsün Karşılaştırmadaki Avantajı: $e = -2$ ile $e = +3$

İki kayan noktalı sayının gerçek üsleri $e_1 = -2$ ve $e_2 = +3$ olsun ($e_2 > e_1$, yani ikinci sayının büyüklük mertebesi daha yüksektir). Tek duyarlıkta 8 bitlik üs alanı ve bias = 127 kullanarak saklanan biaslı üsleri bulalım:

$$E_1 = e_1 + 127 = -2 + 127 = 125 = (01111101)_2$$

$$E_2 = e_2 + 127 = 3 + 127 = 130 = (10000010)_2$$

Bu iki bit dizisi **sıradan işaretsiz tam sayı** olarak karşılaştırılır: $130 > 125$, yani $E_2 > E_1$; bu da gerçek üsler arasındaki $e_2 > e_1$ ilişkisiyle birebir uyuşur. ✓

Şimdi aynı üsleri, bias **kullanmadan**, doğrudan 8 bitlik 2'nin tümleyeniyle saklasaydık ne olacağını görelim:

$$e_1 = -2 \Rightarrow (11111110)_2, \quad e_2 = +3 \Rightarrow (00000011)_2$$

Bu iki diziyi işaretsiz tam sayı olarak karşılaştırsak: $(11111110)_2 = 254$ ve $(00000011)_2 = 3$; $254 > 3$ görünür, yani $e_1 > e_2$ sonucuna varılır. Ama gerçekte $e_2 > e_1$ 'dir ($3 > -2$). İşaretsiz karşılaştırıcı, 2'nin tümleyeninde yanlış sonuç verir; bilgi notunda anlatılan sorun tam olarak budur. Bias, üsleri işaretsiz tam sayı uzayına kaydırarak bu sorunu ortadan kaldırır.

Format	İşaret	Üs	Mantissa
Tek duyarlık (32 bit)	1 bit	8 bit (bias = 127)	23 bit
Çift duyarlık (64 bit)	1 bit	11 bit (bias = 1023)	52 bit

Tablo 1.3: IEEE 754 Tek ve Çift Duyarlıklı Formatlar

1.6.4 Ondalıktan IEEE 754'e Dönüşüm

Örnek 1.54: $(0.6875)_{10}$ 'un Tek Duyarlıklılı IEEE 754 Gösterimi

Bu sayının ikili karşılığını zaten biliyoruz (Bölüm 1.2.9): $(0.6875)_{10} = (0.1011)_2$.

Normalize et: nokta bir basamak sağa kaydırılır, üssü telafi etmek için 2^{-1} çarpanı eklenir:

$$0.1011 = 1.011 \times 2^{-1}$$

İşaret: sayı pozitif olduğu için işaret biti $S = 0$ olarak belirlenir.

Üs: gerçek üs -1 'dir; biaslı üs $-1 + 127 = 126$ olarak hesaplanır, bu değer 8 bitte $(01111110)_2$ şeklinde yazılır.

Mantissa: normalize edilmiş formdaki baştaki 1'den sonraki basamaklar (011) alınır, 23 bite tamamlanana kadar sıfırla doldurulur: 01100000000000000000000.

Üç alan yan yana yazıldığında 32 bitlik sonuç elde edilir:

$$\underbrace{0}_{\substack{S \\ \overline{S}}} \underbrace{01111110}_E \underbrace{011000000000000000000000}_M$$

Alıştırma 1.12 ★★ $(2.5)_{10}$ 'un tek duyarlıklılı IEEE 754 gösterimini bulunuz.

Çözüm:

Adım 1 (ondalığı ikiliye çevirin): Tam kısım $2 = (10)_2$, kesirli kısım $0.5 = (0.1)_2$; birleştirilince $2.5 = (10.1)_2$.

Adım 2 (normalize edin): Nokta 1 basamak sola kaydırılır: $10.1 = 1.01 \times 2^1$.

Adım 3 (işaret bitini belirleyin): Sayı pozitif olduğundan $S = 0$.

Adım 4 (biaslı üsü hesaplayın): Gerçek üs 1 'dir; biaslı üs $1 + 127 = 128 = (10000000)_2$ 'dir.

Adım 5 (mantissa alanını doldurun): Normalize edilmiş anlamlı kısmın noktadan sonraki basamakları ("01"), 23 bite tamamlanana kadar sağdan sıfırla doldurulur:

$$M = 010000000000000000000000$$

Adım 6 (üç alanı birleştirin): $S = 0$, $E = 10000000$, M yukarıdaki gibidir.

$$0 \ 10000000 \ 010000000000000000000000$$

Alıştırma 1.13 ★★★ $(-1.5)_{10}$ 'un tek duyarlıklılı IEEE 754 gösterimini bulunuz.

Çözüm:

Adım 1 (büyüklüğü ikiliye çevirin): $|-1.5| = 1.5 = (1.1)_2$.

Adım 2 (normalize edin): Sayı zaten 1.1×2^0 biçimindedir; ek bir kaydırma gerekmez.

Adım 3 (işaret bitini belirleyin): Sayı negatif olduğundan $S = 1$.

Adım 4 (biaslı üsü hesaplayın): Gerçek üs 0 'dır; biaslı üs $0 + 127 = 127 = (01111111)_2$ 'dir.

Adım 5 (mantissa alanını doldurun): Normalize edilmiş anlamlı kısmın noktadan sonraki tek basamağı ("1"), 23 bite tamamlanana kadar sağdan sıfırla doldurulur:

$$M = 100000000000000000000000$$

Adım 6 (üç alanı birleştirin): $S = 1$, $E = 01111111$, M yukarıdaki gibidir.

$$1 \ 01111111 \ 100000000000000000000000$$

1.6.5 IEEE 754'ten Ondalığa Dönüşüm

Örnek 1.55: Bit Dizisinin Çözülmesi

1 10000001 010000000000000000000000 dizisini çözelim.

İşaret: $S = 1$ olduğundan sayı negatiftir.

Üs: biaslı üs $(10000001)_2 = 129$ 'dur; buradan gerçek üs $129 - 127 = 2$ olarak bulunur.

Mantissa: $M = 01 \dots 0$ 'dır; örtük baştaki 1 ile birlikte anlamlı kısım 1.01 olarak okunur.

Bu üç alan birleştirildiğinde değer şu şekilde hesaplanır:

$$(-1)^1 \times 1.01_2 \times 2^2 = -(101)_2 = -5$$

Alıştırma 1.14 ★★ 0 10000010 110000000000000000000000 dizisini çözümleyip ondalık değerini bulunuz.

Çözüm:

İşaret: $S = 0$ olduğundan sayı pozitifdir.

Üs: Biaslı üs $(10000010)_2 = 130$ 'dur; buradan gerçek üs $130 - 127 = 3$ olarak bulunur.

Mantissa: $M = 11 \dots 0$ 'dır; örtük baştaki 1 ile birlikte anlamlı kısım 1.11 olarak okunur.

Bu üç alan birleştirildiğinde değer şu şekilde hesaplanır:

$$(-1)^0 \times 1.11_2 \times 2^3 = (1110)_2 = 14$$

1.6.6 Özel Değerler ve Kapanış

Özel Bit Kalıpları: Üs alanının tamamen 0 veya tamamen 1 olması özel anlamlar taşır; her durum için tek duyarlıklı (32 bit) bir örnek aşağıda verilmiştir.

- **Üs tümü 0, mantissa tümü 0:** ± 0 değerini temsil eder, işaret biti hangisi olduğunu belirler (ör. 0 00000000 0 ... 0 ifadesi $+0$ 'ı, 1 00000000 0 ... 0 ifadesi -0 'ı verir). İlginçtir: 2'nin tümleyeninde tek bir sıfır kodu elde etmek için özenle tasarım yapmıştık (Bölüm 1.5.4), ama IEEE 754'te $+0$ ile -0 yine iki farklı bit kalıbı olarak karşımıza çıkar.
- **Üs tümü 1, mantissa tümü 0:** $\pm\infty$ değerini temsil eder (ör. 0 11111111 0 ... 0 ifadesi $+\infty$ 'u verir). Örneğin çok büyük iki sayının çarpımı, temsil edilebilir aralığı aştığında bu değer üretilir.
- **Üs tümü 1, mantissa sıfır değil:** NaN (Not a Number) değerini temsil eder (ör. 0 11111111 10 ... 0), $0/0$ ya da $\infty - \infty$ gibi tanımsız işlemlerin sonucudur.
- **Üs tümü 0, mantissa sıfır değil:** *denormalize* (subnormal) sayıları temsil eder (ör. 0 00000000 0 ... 01). Bu durumda örtük baştaki 1 varsayımı bırakılır ve değer $(-1)^S \times 0.M \times 2^{1-bias}$ formülüyle hesaplanır. Amaç, sıfıra çok yakın sayıların aniden değil kademeli olarak küçülmesini sağlamaktır.

NaN kavramı soyut kalmasın; IEEE 754'ün tanımladığı en tuhaf davranışlardan biri, üç yaygın programlama dilinde de birebir aynı şekilde karşımıza çıkar: **NaN, kendisine bile eşit değildir** ($x \neq x$). Bu, bir hata değil, standardın kendisinin bilinçli bir tasarım kararıdır. Bu yüzden her dil, eşitlik operatörü yerine ayrı bir *isnan/isNaN* fonksiyonu sağlar.

C:

```
1 #include <stdio.h>
2 #include <math.h>
3
```

```

4  int main(void) {
5      double x = 0.0 / 0.0;           /* NaN uretir */
6      printf("x = %f\n", x);          /* "x = nan" */
7      printf("x == x: %d\n", x == x); /* 0 (yanlıs!) */
8      printf("isnan(x): %d\n", isnan(x)); /* 1 (dogru yontem) */
9      return 0;
10 }

```

Çıktı:

```

x = nan
x == x: 0
isnan(x): 1

```

Kod 1.1: C’de NaN üretimi ve $x \neq x$ **C++:**

```

1  #include <iostream>
2  #include <cmath>
3  #include <limits>
4
5  int main() {
6      double x = std::numeric_limits<double>::quiet_NaN();
7      std::cout << "x = " << x << "\n";
8      std::cout << "x == x: " << (x == x) << "\n"; // false
9      std::cout << "isnan(x): " << std::isnan(x) << "\n"; // true
10     return 0;
11 }

```

Çıktı:

```

x = nan
x == x: 0
isnan(x): 1

```

Kod 1.2: C++’ta NaN üretimi ve $x \neq x$ **Java:**

```

1  public class NanOrnegi {
2      public static void main(String[] args) {
3          double x = 0.0 / 0.0;           // NaN uretir
4          System.out.println("x = " + x); // "x = NaN"
5          System.out.println("x == x: " + (x == x)); // false
6          System.out.println("Double.isNaN(x): " + Double.isNaN(x)); // true
7      }
8  }

```

Çıktı:

```

x = NaN
x == x: false
Double.isNaN(x): true

```

Kod 1.3: Java’da NaN üretimi ve $x \neq x$

Üçünde de aynı sonuç: $x == x$ karşılaştırması `false/0` döner, oysa her tür sağduyu bize bir sayının kendisine eşit olması gerektiğini söyler. Bunun nedeni, IEEE 754 standardının NaN karşılaştırmalarını *tanımsız* kabul etmesi ve tüm karşılaştırma operatörlerinin (`==`, `<`, `>` vb.) NaN içeren her işlemde `false` döndürmesini zorunlu kılmasıdır. `≠` operatörü bu kuralın tek istisnasıdır ve NaN için her zaman `true` döner. Bu yüzden bir değişkenin NaN olup olmadığını kontrol etmek için asla $x == x$ gibi bir eşitlik testi değil, dilin sağladığı özel fonksiyon (`isnan/std::isnan/Double.isNaN`) kullanılmalıdır.

Bilgi Notu 1.19: Artık Biliyoruz: $0.1 + 0.2 \neq 0.3$

Daha önce (Bölüm 1.2.9) $(0.1)_{10}$ 'un ikilide hiç sonlanmadığını görmüştük. IEEE 754'te mantissa **sonlu** (23 ya da 52 bit) olduğundan, sonlanmayan bu sayı en yakın temsil edilebilir değere **yuvarlanır**; gerçek 0.1'den son derece küçük ama sıfır olmayan bir farkla. 0.1 ve 0.2 ayrı ayrı yuvarlandıktan sonra toplanınca, bu küçük yuvarlama hataları birikir ve sonuç tam olarak 0.3'ün temsiline eşit çıkmaz. Bu, bir yazılım hatası değil, sonlu bit genişliğinin kaçınılmaz bir sonucudur.

1.7 İkili Kodlama Sistemleri

Şimdiye kadar Bölüm 1 boyunca hep *sayıları* ikiliye çevirmekle uğraştık. Ama bir bilgisayarın işlediği bilginin çoğu aslında sayısal bir büyüklük değildir: harfler, semboller, ya da "hangi çalışan", "hangi renk" gibi kategorik kimlikler de ikili düzende temsil edilmelidir. Bu bölümde, sayısal değeri değil bir *kimliği* temsil eden bit dizilerini, yani **kodları**, inceleyeceğiz.

İkili Kod: Bir kümede N farklı eleman varsa, bu elemanları ayırt eden bir n -bit *ikili kod*, her elemana 2^n olası bit kombinasyonundan benzersiz birini atar. Ayırt etmek için gereken minimum bit sayısı $n = \lceil \log_2 N \rceil$ 'dir, çünkü n bit tam olarak 2^n farklı kombinasyon üretir. Ancak **maksimum** bit sayısına dair bir sınır yoktur: N elemanı kodlamak için gerekenden fazla bit de kullanılabilir, fazladan kombinasyonlar basitçe kullanılmadan bırakılır.

Örneğin, 10 ondalık rakamı (0-9) ayırt etmek için teorik minimum $n = 4$ bittir, çünkü $2^3 = 8 < 10 \leq 2^4 = 16$. Bu durumda 16 olası kombinasyondan yalnızca 10 tanesi kullanılır; geri kalan 6 kombinasyon (1010'dan 1111'e kadar) tanımsız/kullanılmaz kalır. Birazdan göreceğimiz BCD kodu tam olarak bu stratejiyi izler.

1.7.1 BCD (Binary-Coded Decimal) Kodu

En yaygın kullanılan ondalık kod, her ondalık rakamı **ayrı ayrı** 4 bitlik ikili karşılığına çevirip bu grupları yan yana dizen **BCD** (Binary-Coded Decimal, ikili-kodlanmış ondalık) kodudur. BCD'ye bazen **8421 kodu** da denir, çünkü her bitin ağırlığı sırasıyla 8, 4, 2, 1'dir; yani her 4-bitlik grup, doğrudan o rakamın kendi ikili karşılığıdır.

BCD Kodu: k ondalık haneli bir sayının BCD karşılığı $4k$ bit gerektirir; her hane, 0000'dan 1001'e kadarki (yani 0'dan 9'a kadarki) 4-bit ikili karşılığıyla ayrı ayrı kodlanır. 1010'dan 1111'e kadarki 6 kombinasyon BCD'de **geçersizdir**, hiçbir anlamı yoktur.

Önemli: BCD, bir sayının ikili karşılığıyla **aynı şey değildir**. Bunu bir örnekle netleştirelim.

Örnek 1.56: $(185)_{10}$ 'in BCD ve İkili Karşılıklarının Karşılaştırılması

Gerçek ikili karşılık, sayının bütünü tek seferde ikiliye çevirerek bulunur:

$$(185)_{10} = (10111001)_2 \quad (8 \text{ bit yeterli})$$

BCD karşılık ise her haneyi **ayrı ayrı** kodlayıp yan yana dizerek bulunur:

$$1 \rightarrow 0001, \quad 8 \rightarrow 1000, \quad 5 \rightarrow 0101 \quad \Rightarrow \quad (185)_{10} = (0001 \ 1000 \ 0101)_{BCD}$$

BCD karşılık 12 bit gerektirirken gerçek ikili karşılık yalnızca 8 bit yeterlidir; aynı sayı için iki tamamen farklı bit deseni.

BCD, gerçek ikiliden daha fazla bit harcadığı halde neden kullanılır? Çünkü BCD bir **hız ya da verimlilik** kodu değil, bir **insan-makine arayüz kolaylığı** kodudur. Bilgisayarın girdi/çıkırtısını üreten insan ondalık sistemle düşünür; bir ekrana "185" yazdırmak ya da klavyeden "1", "8", "5" tuşlarını okumak, sayının tamamını ikiliye çevirip geri çevirmektense, her hanenin kendi 4-bit kutusunda ayrı ayrı durduğu bir temsille çok daha doğrudandır.

Alıştırma 1.15 ★ $(84)_{10}$ sayısının BCD karşılığını bulunuz.

Çözüm: $8 \rightarrow 1000, 4 \rightarrow 0100$, yan yana dizilince $(84)_{10} = (1000 \ 0100)_{BCD}$.

1.7.2 BCD Toplama

BCD'nin en can sıkıcı yönü burada ortaya çıkar: düz ikili toplama kuralları BCD'de doğrudan işlemez. İki BCD hanesi (ve olası bir önceki elde) toplandığında, sonuç en fazla $9 + 9 + 1 = 19$ olabilir; bu iki 4-bit grubu sıradan ikili sayıların gibi toplarsak, donanım bize doğru *ikili* toplamı verir (0'dan 19'a kadar). Ama BCD'de yalnızca 0000-1001 (0-9) geçerlidir. Toplam 1010 (10) veya üzerindeyse, ya da 4. bitten bir elde taştıysa, elde edilen sonuç BCD açısından geçersizdir.

BCD Toplama Kuralı (+6 Düzeltmesi): İki BCD hanesi (ve varsa önceki elde) sıradan ikili sayıların gibi toplanır. Elde edilen 4-bitlik ikili toplam:

- ≤ 1001 (9) ise ve 4. bitten bir elde taşmadıysa, sonuç zaten geçerli bir BCD hanesidir, düzeltme gerekmez.
- ≥ 1010 (10) ise **veya** 4. bitten bir elde taştıysa, kalan 4 bite 0110 (6) eklenir; bu hem doğru BCD hanesini üretir hem de (gerekmiyorsa) bir sonraki haneye giden elde bitini doğurur.

Düzeltilmenin neden tam 6 olduğu, 4 bitin sağladığı 16 kombinasyondan yalnızca 10'unun BCD'de geçerli olmasından kaynaklanır: $16 - 10 = 6$. Toplam geçersiz bölgeye (10-15) girdiğinde, 6 eklemek onu bu altı kullanılmayan kodun üzerinden atlatarak doğru hane ve elde bitini üretir.

Örnek 1.57: Düzeltme Gerekmeyen Toplam: $4 + 5$

$$\begin{array}{r} 0100 \\ + 0101 \\ \hline 1001 \end{array}$$

İkili toplam $1001 (9) \leq 1001$ olduğundan zaten geçerli bir BCD hanesidir, düzeltme gerekmez.

Örnek 1.58: Düzeltme Gereken Toplam: $4 + 8$

$$\begin{array}{r} 0100 \\ + 1000 \\ \hline 1100 \end{array}$$

İkili toplam $1100 (12) \geq 1010$ olduğundan geçersizdir; $0110 (6)$ eklenir:

$$\begin{array}{r} 1100 \\ + 0110 \\ \hline 1\ 0010 \end{array}$$

Sonuç: hane $0010 (2)$, elde 1 — yani ”12” (elde= 1, hane= 2). Doğru!

Örnek 1.59: Taşan Elde ile Düzeltme: $8 + 9$

$$\begin{array}{r} 1000 \\ + 1001 \\ \hline 1\ 0001 \end{array}$$

Burada zaten 4. bitten bir elde taşmış durumdadır. Kalan 4 bite yine 0110 eklenir:

$$\begin{array}{r} 0001 \\ + 0110 \\ \hline 0111 \end{array}$$

Elde zaten mevcuttu, kalan hane $0111 (7)$ — sonuç ”17” (elde= 1, hane= 7). Doğru!

Bu tek haneli kural, çok haneli sayılara şöyle genişler: hanelere sağdan sola, birer birer uygulanır ve her hanenin ürettiği elde bir sonraki (daha soldaki) hanenin toplamına dahil edilir, tıpkı okulda öğrendiğimiz elle toplamada olduğu gibi.

Örnek 1.60: Çok Haneli BCD Toplama: $(184)_{BCD} + (576)_{BCD}$

Sağdan sola, hane hane:

Hane	İkili Toplam (elde dahil)	Düzeltilme	Sonuç / Elde
Birler	$0100 + 0110 = 1010$	≥ 1010 , +0110	0000 / 1
Onlar	$1000 + 0111 + 0001 = 1\ 0000$	elde taşı, +0110	0110 / 1
Yüzler	$0001 + 0101 + 0001 = 0111$	≤ 1001 , gerek- mez	0111 / 0

Elde zincirinin izlenmesi önemlidir: birler hanesinin ürettiği elde (1), onlar hanesinin toplamına üçüncü terim olarak girmiştir ($1000 + 0111 + 0001$); onlar hanesinin ürettiği elde (1) de aynı şekilde yüzler hanesinin toplamına girmiştir ($0001 + 0101 + 0001$). Yüzler hanesinde üretilen elde 0 olduğundan zincir burada sona erer. Eğer 1 olsaydı, dördüncü bir (binler) haneye taşınacaktı. Sağdan sola haneler birleştirilince: $(0111\ 0110\ 0000)_{BCD} = (760)_{10}$. Doğrulama: $184 + 576 = 760$. ✓

Alıştırma 1.16 ★ $4 + 6$ toplamının BCD karşılığı (gerekliyorsa düzeltme dahil) bulunuz.

Çözüm: $0100 + 0110 = 1010$ (10), ≥ 1010 olduğundan 0110 eklenir: $1010 + 0110 = 1\ 0000$. Sonuç: hane 0000 (0), elde 1; yani BCD toplamı $(1\ 0000)_{BCD}$, ondalıkta 10.

1.7.3 Diğer Ondalık Kodlar

BCD, dört bitle on rakamı kodlamanın **tek** yolu değildir; sadece en yaygın olanıdır. Aynı 4 bitten 10 farklı kombinasyon seçmenin başka yolları da vardır, ve her seçim farklı bir mühendislik avantajı sağlar.

Ağırlıklı Kod: BCD’de her bitin sabit bir *ağırlığı* vardır (8, 4, 2, 1) ve bir hanenin değeri, 1 olan bitlerin ağırlıklarını toplayarak bulunur. Aynı fikir farklı ağırlık kümeleriyle de uygulanabilir; buna **ağırlıklı kod** denir. Ağırlıkların hepsinin pozitif olması da gerekmez.

Örnek 1.61: 2421 Kodu: Aynı Rakam İçin İki Geçerli Kodlama

2421 kodunda ağırlıklar 2, 4, 2, 1’dir; bir bit kalıbının değeri, 1 olan bitlerin ağırlıklarını toplayarak bulunur.

Birinci kalıp: 0100 bit kalıbını değerlendirelim:

$$2 \cdot 0 + 4 \cdot 1 + 2 \cdot 0 + 1 \cdot 0 = 4$$

İkinci kalıp: Şimdi 1010’u değerlendirelim:

$$2 \cdot 1 + 4 \cdot 0 + 2 \cdot 1 + 1 \cdot 0 = 4$$

Aynı değer, iki farklı bit kalıbı! Rakam 4, hem 0100 hem de 1010 ile kodlanabilir. 2421 kodunda bazı rakamlar için birden fazla geçerli kodlama vardır; bu seçim özgürlüğü rastgele değildir. Birazdan göreceğimiz *öz-tümleyen* özelliğini sağlamak için kasıtlı olarak kullanılır.

Ağırlıklı olmayan bir kod da mümkündür: **Excess-3** (fazla-3) kodunda hiçbir ağırlık yorumu yoktur, kural yalnızca şudur: *her rakamın düz ikili karşılığına 3 ekleyin*. Örneğin rakam 5:

$0101 + 0011 = 1000$.

Aşağıdaki tablo, dört farklı ondalık kodu karşılaştırmalı olarak gösterir.

Rakam	BCD (8421)	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111

Tablo 1.4: Dört Farklı Ondalık Kodun Karşılaştırılması

Bu özelliğin neye yaradığını anlamak için önce nereden geldiğini hatırlayalım: Bölüm 1.4’de çıkarmayı toplamaya çevirmek için tümleyen almıştık. Aynı fikri şimdi *ondalık* sayılar için uygulayacağız; bu da her seferinde bir rakamın 9’un tümleyenini ($9-d$) hesaplamayı gerektirir. Bunu donanımda hesaplamamanın iki yolu var:

- **Aritmetik yoldan:** Girdisi d , çıktısı $9-d$ olan gerçek bir çıkarma devresi (subtractor) kurulur. Bu yol, kapı sayısı bakımından pahalı ve gecikme bakımından yavaştır.
- **Kod seçimiyle:** Rakamlar öyle bir kodla saklanırsa ki, kodun bitlerini teker teker **tersine çevirmek** ($1 \rightarrow 0, 0 \rightarrow 1$) otomatik olarak $9-d$ ’nin kodunu verir; bu durumda çıkarma devresine hiç gerek kalmaz, yalnızca 4 paralel NOT kapısı yeterli olur.

Burada terminolojiyi netleştirmek gerekir: bir **kodlama şeması** (Excess-3, BCD gibi), her rakama bir **bit kalıbı** atayan kuraldır. Aşağıdaki tanım bu iki kavramı ayrı tutarak ifade edilmiştir.

Öz-Tümleyen (Self-Complementing) Kod: Bir kodlama şemasında her rakama bir bit kalıbı atanır. Bu şema, herhangi bir rakamın bit kalıbındaki tüm bitler tersine çevrildiğinde ($1 \rightarrow 0, 0 \rightarrow 1$) ortaya çıkan yeni bit kalıbı, tam olarak $9-d$ rakamına atanmış bit kalıbıyla aynı oluyorsa *öz-tümleyen* olarak adlandırılır.

Örnek 1.62: Tanımın Somut Bir Kontrolü: 2421 Kodunda Rakam 4

Tanımı, elimizde zaten olan Tablo 1.4’deki 2421 kodu üzerinden tek bir rakamla sınavalım. Rakam 4’ün 2421 kodundaki bit kalıbı 0100’dür. Bitlerini tek tek tersine çevirelim:

$$0100 \Rightarrow 1011$$

Şimdi $9-4=5$ ’in 2421 kodundaki bit kalıbına bakalım: yine Tablo 1.4’a göre 1011’dir.

İki sonuç birebir aynı ($1011 = 1011$); yani rakam 4'ün bit kalıbını çevirmek, bize doğrudan rakam 5'in bit kalıbını verdi. 2421 kodu, en azından bu rakam için, tanımdaki öz-tümleyen koşulunu sağlıyor.

Tek bir rakam için doğrulamak fikri somutlaştırır, ama genel olarak geçerli olduğunu kanıtlamaz. Bunu Excess-3 üzerinden, adım adım görelim. Ama önce, bu kodun tanımında ilk bakışta kafa karıştırabilecek bir noktayı vurgulayalım: Excess-3 kuralı, "*rakam d 'nin bit kalıbı, $d + 3$ sayısının düz ikili karşılığıdır*" der. Yani bir bit kalıbının **sayısal değeri** ile o kalıbın **temsil ettiği rakam**, aralarında sabit bir $+3$ fark olacak şekilde, birbirinden farklıdır. Kodun adındaki "excess" ("fazlalık") kelimesi tam olarak bu farkı anlatır.

Örnek 1.63: Excess-3'ün Öz-Tümleyen Özelliği

Amacımız şunu göstermek: rakam 3'ün Excess-3 bit kalıbının bitlerini tersine çevirdiğimizde elde ettiğimiz yeni kalıbın, rakam $9 - 3 = 6$ 'nın Excess-3 bit kalıbıyla birebir aynı çıktığını görmek. Bunu dört adımda yapalım.

Adım 1 (rakam 3'ün bit kalıbını bulalım): Rakam 3'ün düz ikili karşılığı 0011'dir. Excess-3 kuralı gereği buna 3 eklenir (yani $0011 + 0011$):

$$0011 + 0011 = 0110$$

Burada dikkat edilmesi gereken can alıcı nokta şudur: 0110'nın **sayısal değeri** 6'dır ($4 + 2 = 6$). Ama bu bit kalıbı, rakam 6'yı değil, **rakam 3'ü** temsil etmek üzere atanmıştır, çünkü kuralımız " $d + 3$ " idi ve $3 + 3 = 6$. Yani rakam 3'ün bit kalıbı, sayısal değeri 6 olan 0110'dır; bu bir çelişki değil, kodun tanımının doğal bir sonucudur.

Adım 2 (aynı kuralı rakam 6'ya uygulayalım): Rakam 6'nın düz ikili karşılığı 0110'dır. Buna 3 eklenir: $0110 + 0011 = 1001$. 1001'in sayısal değeri 9'dur, ama bu bit kalıbı **rakam 6'yı** temsil eder (yine $+3$ farkla: $6 + 3 = 9$).

Adım 3 (Adım 1'in kalıbının bitlerini çevirelim, hiç toplama yapmadan): Rakam 3'ün bit kalıbı olan 0110'ı ele alalım ve her bitini tek tek tersine çevirelim:

$$\begin{array}{ccccccc} 0 & 1 & 1 & 0 & \Rightarrow & 1001 \\ \rightarrow 1 & \rightarrow 0 & \rightarrow 0 & \rightarrow 1 & & \end{array}$$

Adım 4 (sonuçları karşılaştıralım): Adım 2'de toplama yaparak bulduğumuz rakam 6'nın bit kalıbı (1001) ile Adım 3'te hiç toplama yapmadan, sadece rakam 3'ün bit kalıbının bitlerini çevirerek bulduğumuz sonuç (1001) birebir aynıdır. Yani rakam 3'ün bit kalıbının bitlerini çevirmek, bize doğrudan rakam $9 - 3 = 6$ 'nın bit kalıbını verdi, hiç toplama veya çıkarma yapmadan.

Bu bir rastlantı değildir; "3" sayısının seçimi de rastgele değildir. Kanıtlayalım. Bir 4-bitlik sayının bitlerini tek tek çevirmek, matematiksel olarak $15 - x$ işlemine eşittir (çünkü 4 bit 0'dan 15'e kadar sayar, ve her biti çevirmek sayıyı 15'e göre "aynalar": $0000 \rightarrow 1111$ yani $0 \rightarrow 15$, $0001 \rightarrow 1110$ yani $1 \rightarrow 14$, ve genel olarak $x \rightarrow 15 - x$). Rakam d 'nin bit kalıbının sayısal değeri $d + 3$ olduğundan, bu bit kalıbının bitlerini çevirmek bize $15 - (d + 3) = 12 - d$ değerinde bir bit kalıbı verir. Öte yandan, bizim aslında elde etmek istediğimiz şey, rakam $9 - d$ 'ye atanmış bit kalıbıdır; bu kalıbın sayısal değeri de $(9 - d) + 3 = 12 - d$ 'dir. **İkisi de $12 - d$; birebir eşit!**

Bu sabit ekleme miktarına (Excess-3'te 3, BCD'de 0) kodun **ofseti** denir. Ofset, bir bit kalıbının sayısal değeri ile o kalıbın temsil ettiği rakam arasındaki sabit farktır. İşte Excess-3'ün ofseti olan "3", tam olarak yukarıdaki eşitliği ($15 - 3 = 9 + 3$) sağlayacak şekilde seçilmiştir; rastgele bir tercih değildir.

Bu özellik neden önemlidir? Çünkü ondalık çıkarmada (bir sonraki alt bölümde göreceğimiz gibi) bir tümleyen alma işlemi normalde ayrı bir çıkarma devresi gerektirir; ama öz-tümleyen bir kodda bu işlem tek bir NOT kapısı katmanına indirgenir, ekstra donanımına hiç gerek kalmaz.

Alıştırma 1.17 ★ BCD (8421) kodunun öz-tümleyen olmadığını, yukarıdaki kanıtla aynı yöntemi kullanarak gösteriniz.

Çözüm:

Adım 1 (BCD'nin ofsetini belirleyin): BCD'nin ofseti 0'dır, yani bit kalıbının sayısal değeri rakamın kendisine eşittir ($d + 0 = d$). Excess-3'teki "+3" yerine burada "+0" vardır.

Adım 2 (bit çevirmenin verdiği değeri hesaplayın): Bit çevirme yine $15 - x$ işlemine eşittir; BCD kodunu çevirirsek $15 - d$ elde ederiz.

Adım 3 (elde etmek istediğimiz değerle karşılaştırın): Bizim istediğimiz, $9 - d$ 'nin BCD kodu, yani sadece $9 - d$ 'nin kendisidir (ofset 0 olduğu için). Öz-tümleyen olması için $15 - d = 9 - d$ eşitliğinin sağlanması gerekirdi; bu ise d 'den bağımsız olarak asla doğru olamaz ($15 \neq 9$).

Adım 4 (somut bir örnekle doğrulayın): 0'ın BCD kodu 0000'dır; bitleri çevirirsek 1111 (15) elde ederiz. Ama $9 - 0 = 9$ 'un BCD kodu 1001'dir; $1111 \neq 1001$.

Demek ki BCD'nin öz-tümleyen olmaması tek bir rakama özgü bir tesadüf değil, BCD'nin ofsetinin (0) Excess-3'ün gerektirdiği değere (3) eşit olmamasından kaynaklanan **yapısal bir imkansızlıktır**.

Son olarak, yukarıdaki tablonun son sütunu bir **negatif ağırlıklı kod** örneğidir: 8, 4, -2, -1 kodunda ağırlıkların hepsi pozitif olmak zorunda değildir. Örneğin 0110 bu kodda $8 \cdot 0 + 4 \cdot 1 + (-2) \cdot 1 + (-1) \cdot 0 = 2$ olarak yorumlanır – ve bu kod da (2421 gibi) öz-tümleyendir.

1.7.4 BCD'de İşaretli Ondalık Aritmetik (10'un Tümleyeni)

Bölüm 1.5'de ikili sayılarda negatifleri saklamak için 2'nin tümleyenini kullanmış, çıkarmayı toplamaya çevirmiştik. Aynı fikri şimdi BCD ile saklanan ondalık sayılara uygulayacağız; bu sefer kullanılan araç **10'un tümleyenidir**.

İşaret Hanesi: Bir işaretli BCD sayısında işaret, ayrı bir bit değil, ayrı bir **hane** olarak saklanır: bu hane pozitif sayılar için 0000, negatif sayılar için 1001 (rakam 9'un BCD kodu) değerini alır.

Neden özellikle 9 seçilmiştir? Çünkü 10'un tümleyenini alma işlemi, negatif bir sayının en soluna hep 9 hanesi(leri) dizer. Bu, 2'nin tümleyeninde negatif sayıların en soluna 1 bitleri dizilmesinin (işaret genişletme) ondalıktaki karşılığıdır.

Bir BCD Sayısının 10'un Tümleyenini Bulma: İki adımlı bir işlemdir:

1. Her hanenin 9'un tümleyeni alınır (her rakam d için $9 - d$ hesaplanır); buna *9'un tümleyeni* denir.
2. Sonucun en sağdaki hanesine 1 eklenir.

Bilgi Notu 1.20: Öz-Tümleyen Kodlarla Bağlantı

Bir önceki alt bölümde BCD'nin öz-tümleyen olmadığını kanıtlamıştık. Bunun donanımsal bedeli burada ortaya çıkar: yukarıdaki 1. adımda her hanenin 9'un tümleyenini bulmak gerçek bir çıkarma gerektirir. Rakamlar öz-tümleyen bir kodla (Excess-3 gibi) saklansaydı, bu adım yalnızca bit çevirmeyle (NOT kapıları) yapılabilirdi.

İşaretili Ondalık Toplama Kuralı: İşaret hanesi dahil tüm haneler, BCD toplama kuralıyla (gerektiğinde +6 düzeltmesiyle) toplanır. En soldaki haneden (işaret hanesinden) taşan son elde atılır, tıpkı 2'nin tümleyeninde işaret bitinden taşan eldenin atılması gibi.

Örnek 1.64: $(+375) + (-240)$

Önce -240 'ı 10 'un tümleyeniniyle temsil edelim. 240 'ın 9 'un tümleyenini: $9-2=7$, $9-4=5$, $9-0=9$, yani 759 . En sağdaki haneye 1 eklenir: 760 . İşaret hanesi 9 olduğundan, $-240 \rightarrow 9760$.

Şimdi $0375 + 9760$ toplamını hane hane (sağdan sola) yapalım. Hatırlatma: bir hanenin toplamı 9 'u aşarsa (ya da elde üretirse), Bölüm 1.7.2'de gördüğümüz +6 BCD düzeltmesi uygulanır:

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$5 + 0 = 5$	gerekmez	$5 / 0$
Onlar	$7 + 6 = 13$	≥ 10 : +6 eklenir	$3 / 1$
Yüzler	$3 + 7 + 1 = 11$	≥ 10 : +6 eklenir	$1 / 1$
İşaret	$0 + 9 + 1 = 10$	≥ 10 : +6 eklenir	$0 / 1$ (atılır)

Son elde (işaret hanesinden taşan) atılır. Sonuç haneleri (işaret, yüzler, onlar, birler) $0, 1, 3, 5$ 'tir, yani $+135$. Doğrulama: $375 - 240 = 135$.

Alıştırma 1.18 ★★ İki parçada çözünüz: (a) $370 + (-250)$, (b) $250 + (-370)$. İkinci kısımda sonucun işaret hanesi negatif çıkacaktır; gerçek büyüklüğü bulmak için sonucun büyüklük hanelerinin 10 'un tümleyenini alınız.

Çözüm (a): -250 'nin 10 'un tümleyenini: 9 'un tümleyenini $9-2=7$, $9-5=4$, $9-0=9$, yani 749 ; en sağa 1 eklenir: 750 ; işaret hanesi 9 : $-250 \rightarrow 9750$.

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$0 + 0 = 0$	gerekmez	$0 / 0$
Onlar	$7 + 5 = 12$	≥ 10 : +6 eklenir	$2 / 1$
Yüzler	$3 + 7 + 1 = 11$	≥ 10 : +6 eklenir	$1 / 1$
İşaret	$0 + 9 + 1 = 10$	≥ 10 : +6 eklenir	$0 / 1$ (atılır)

Sonuç haneleri $0, 1, 2, 0$ 'dır, yani $+120$. Doğrulama: $370 - 250 = 120$.

Çözüm (b): -370 'in 10 'un tümleyeni: 9 'un tümleyeni $9 - 3 = 6$, $9 - 7 = 2$, $9 - 0 = 9$, yani 629 ; en sağa 1 eklenir: 630 ; işaret hanesi 9 : $-370 \rightarrow 9630$.

Hane	Toplam	Düzeltilme (+6 BCD kuralı)	Sonuç / Elde
Birler	$0 + 0 = 0$	gerekmez	$0 / 0$
Onlar	$5 + 3 = 8$	gerekmez	$8 / 0$
Yüzler	$2 + 6 = 8$	gerekmez	$8 / 0$
İşaret	$0 + 9 = 9$	gerekmez	$9 / 0$

Sonuç haneleri $9, 8, 8, 0$ 'dır. İşaret hanesi 9 olduğundan sonuç negatiftir; gerçek büyüklüğü bulmak için büyüklük hanelerinin (880) 10 'un tümleyenini alalım: 9 'un tümleyeni $9 - 8 = 1$, $9 - 8 = 1$, $9 - 0 = 9$, yani 119 ; en sağa 1 eklenir: 120 . Sonuç -120 'dir. Doğrulama: $250 - 370 = -120$. Ondalık çıkarma ($M - N$) da ikilideki gibi yapılır: N 'nin (çıkanın) 10 'un tümleyeni alınıp M 'ye eklenir.

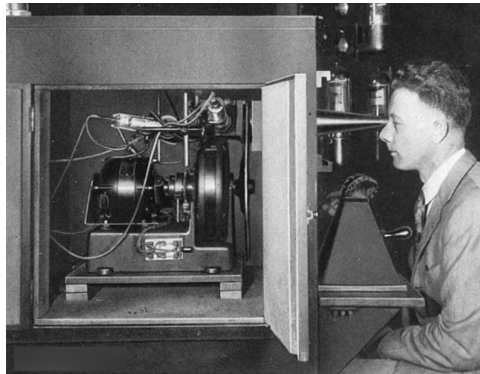
1.7.5 Gray Kodu

Düz ikili sayma sırasında, ardışık iki sayı arasında **birden fazla bit aynı anda** değişebilir. Örneğin $7 \rightarrow 8$ geçişi $0111 \rightarrow 1000$ 'dir: dört bitin dördü de aynı anda değişir.

Bu, kağıt üzerinde bir sorun değildir. Gerçek donanımda ise ciddi bir soruna yol açabilir: farklı bitlerin mantık kapılarından geçip yeni değerlerine ulaşması tam olarak aynı anda gerçekleşmez. Üretim toleransları, kapı gecikmeleri ve tel uzunlukları gibi nedenlerle, bitler arasında nanosaniyeler mertebesinde gecikme farkları oluşur. $0111 \rightarrow 1000$ geçişinde, eğer örneğin en sağdaki bit diğerlerinden biraz daha yavaş değişirse, devre kısa bir süreliğine 1001 gibi hiç var olmaması gereken, yanlış bir ara değer üretebilir.

Gray Kodu: Gray kodu, ardışık iki değer arasında **yalnızca tek bir bitin** değiştiği bir ikili kodlama şemasıdır.

Kodun adı nereden geliyor? Gray kodu, adını Bell Telefon Laboratuvarları'nda çalışan Amerikalı fizikçi ve mühendis Frank Gray'den (13 Eylül 1887-23 Mayıs 1969) alır. Purdue Üniversitesi'nden 1911'de fizik diplomasıyla mezun olan Gray, kariyerinin büyük bölümünü televizyon teknolojisine adanmış: 1927'de erken televizyon sistemleri için "uçan nokta tarayıcı" (flying spot scanner) fikrini önerdi, 1930'da ise iki yönlü, mekanik taramalı bir televizyon sisteminin geliştirilmesine katkıda bulundu.



Şekil 1.1: Frank Gray, 1930'da geliştirdiği mekanik televizyon aygıtını incelerken.

1947’de başvurduğu ve 1953’te kendisine verilen, ”Pulse Code Communication” (Darbe Kodlu Haberleşme) başlıklı 2,632,058 numaralı patentte Gray, analog bir sinyali sayısal darbelere çeviren bir darbe kodu modülasyonu (PCM) aygıtı tanımladı. Bu aygıtın içindeki plaka, ardışık değerler arasında yalnızca tek bir bitin değiştiği bir kod düzeniyle kaplanmıştı; Gray, patent başvurusunda bu kodun o tarihte henüz tanınmış bir adı olmadığını belirtmişti. Bugün bu kod, onun adıyla anılıyor.

March 17, 1953

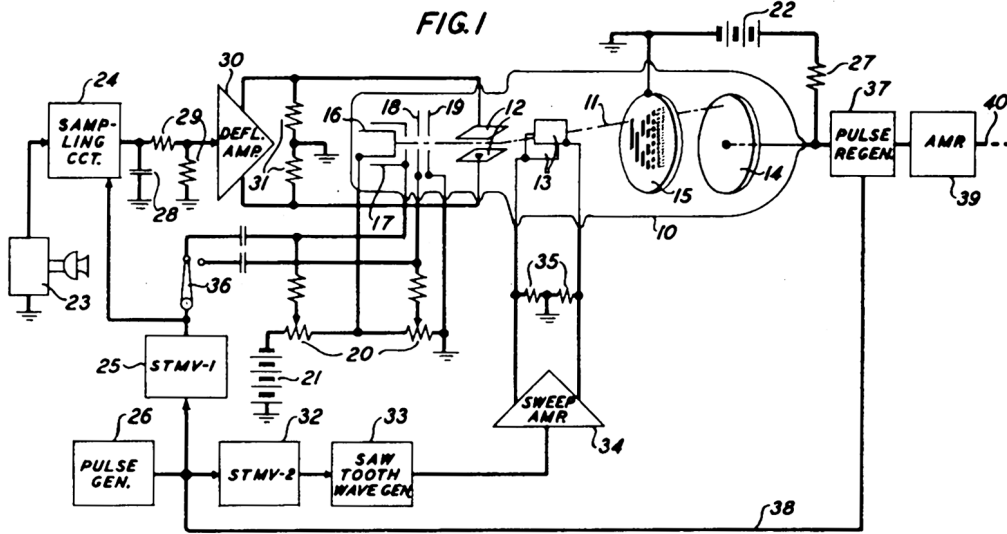
F. GRAY

2,632,058

PULSE CODE COMMUNICATION

Filed Nov. 13, 1947

4 Sheets-Sheet 1



Şekil 1.2: Gray’in 2,632,058 numaralı patentinin ön sayfasından bir bölüm; yansıtılmış ikili kodu taşıyan plakayı (15) içeren PCM tüpü (10) görülüyor.

Kodun kendisi aslında Gray’dan çok önce de kullanılmıştı: Fransız mühendis Émile Baudot, 1870’lerde telgraf makinesinin harf çarkını sıralarken benzer bir kod düzeni kullanmış, Alman-Avusturyalı Otto Schäffler de 1874’te benzer bir düzeni göstermişti. Ama kodun bugünkü ”Gray kodu” adı, onu titizlikle tanımlayıp somut bir mühendislik problemine (PCM, yani darbe kodu modülasyonu ile yapılan analog-sayısal dönüşüm) uygulayan ve patentleyen kişinin Gray olmasından geliyor.

Aşağıdaki tablo, 0’dan 15’e kadar düz ikili ile Gray kodunu karşılaştırmalı gösterir. Her satırdan bir sonrakine geçerken Gray sütununda yalnızca tek bir bitin değiştiğine dikkat edin.

Ondalık	İkili	Gray Kodu
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Tablo 1.5: 0'dan 15'e kadar ikili ve Gray kodu karşılaştırması.**Örnek 1.65:** 7 → 8 Geçişinde İkili ile Gray Kodunun Karşılaştırılması

Düz ikilide 7 → 8 geçişi 0111 → 1000'dir: dört bitin dördü de değişir. Gray kodunda aynı geçiş 0100 → 1100'dür: yalnızca en soldaki bit değişir, diğer üç bit aynı kalır. Geçiş sırasında hangi an yakalanırsa yakalansın, okunan değer ya eski değer ya da yeni değerdir. İki değerın karışımı olan geçersiz bir ara değer hiçbir zaman ortaya çıkmaz.

Gray kodunun tipik bir kullanım alanı, dönen bir milin açısal konumunu ölçen sensörlerdir (encoder). Mil döndükçe, üzerindeki kodlanmış disk bir sensör tarafından okunur. Düz ikili kullanılsaydı, birden fazla bitin aynı anda değiştiği geçiş anlarında yanlış okuma riski olurdu; Gray kodu bu riski ortadan kaldırır.

Bilgi Notu 1.21: İkiliden Gray Koduna Dönüşüm

Yukarıdaki tabloyu ezbere bilmeye gerek yoktur; bir ikili sayıdan Gray koduna doğrudan bir formülle geçilebilir. n bitlik ikili sayı $b_{n-1} \dots b_1 b_0$ için, Gray kodunun bitleri $g_{n-1} \dots g_1 g_0$ şöyle hesaplanır:

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i \quad (0 \leq i \leq n-2)$$

Burada $\oplus$ sembolü XOR (özel-veya) işlemidir; Bölüm 2'de tanıtılacaktır. Yani en soldaki bit aynen korunur, diğer her bit ise kendisinden bir solundaki bitle XOR'lanır.

Örnek 1.66: Formülle $7 = (0111)_2$ 'nin Gray Koduna Çevrilmesi

$b_3b_2b_1b_0 = 0111$ için:

$$g_3 = b_3 = 0, \quad g_2 = b_3 \oplus b_2 = 0 \oplus 1 = 1$$

$$g_1 = b_2 \oplus b_1 = 1 \oplus 1 = 0, \quad g_0 = b_1 \oplus b_0 = 1 \oplus 1 = 0$$

Sonuç: $(0100)_2$. Tabloyla birebir uyuyor.

Alıştırma 1.19 ★ Yukarıdaki formülü kullanarak $(1010)_2$ 'nin (10'un ikili karşılığı) Gray kodunu bulunuz.

Çözüm: $b_3b_2b_1b_0 = 1010$. $g_3 = b_3 = 1$. $g_2 = b_3 \oplus b_2 = 1 \oplus 0 = 1$. $g_1 = b_2 \oplus b_1 = 0 \oplus 1 = 1$. $g_0 = b_1 \oplus b_0 = 1 \oplus 0 = 1$. Sonuç: $(1111)_2$. Tabloda 10 satırına bakıldığında Gray kodu 1111'dir. Doğrulandı.

1.7.6 ASCII Karakter Kodu

Şu ana kadar gördüğümüz kodlar (BCD, Gray) hep *rakamları* temsil ediyordu. Ama bir bilgisayarın işlemesi gereken bilginin çoğu rakam değildir; harfler, noktalama işaretleri ve klavyedeki her tuş da ikili düzende temsil edilmelidir.

Kaç bite ihtiyacımız var? Bölüm 1.7'deki İkili Kod tanımını hatırlayalım: n bit, 2^n farklı eleman ayırt eder. Temsil etmemiz gereken kümeyi sayalım: 26 büyük harf, 26 küçük harf, 10 rakam, noktalama/özel karakterler (% , \$, @ gibi) ve birkaç görünmez kontrol karakteri (satır başı, sekme gibi). Bu toplam 128'e kadar çıkar. $2^6 = 64$ yetmez, $2^7 = 128$ yeterlidir; bu yüzden **ASCII 7 bit** kullanır.

ASCII (American Standard Code for Information Interchange): ASCII, 128 karakteri (harfler, rakamlar, noktalama işaretleri ve kontrol karakterleri) 7-bit bir kodla temsil eden standart karakter kodudur. Bit dizisi, bu kitap boyunca kullandığımız 0-indeksli düzene uygun olarak $b_6b_5b_4b_3b_2b_1b_0$ şeklinde adlandırılır; b_6 en soldaki (en anlamlı) bit, b_0 en sağdaki (en az anlamlı) bittir.

Bilgi Notu 1.22: Tarihi Bir Not: ASCII'nin Orijinal Bit Numaralandırması

ASCII standardının 1963 tarihli ilk hâli ve onu izleyen birçok ders kitabı, bitleri b_1 'den b_7 'ye kadar (b_0 hiç kullanılmadan) adlandırır. Bu, telgraf/teleprinter mühendisliğinden kalma eski bir numaralandırmadır. Modern dijital tasarımda (ve bu kitabın geri kalanında, örn. Gray kodunda $b_{n-1} \dots b_1b_0$) standart olan 0-indeksli düzen kullanılır; burada tutarlılık için bu düzeni tercih ediyoruz.

Standart ASCII tablosu, 7 biti ikiye bölerek okunur: en soldaki 3 bit ($b_6b_5b_4$) tablonun **sütunu**, kalan 4 bit ($b_3b_2b_1b_0$) tablonun **satırını** belirler. Bu bölünme rastgele değildir: sütun 000-001 tamamen kontrol karakterlerine (görünmez, yazdırılmaz), sütun 010-011 rakamlara ve temel noktalamaya, sütun 100-101 büyük harflere, sütun 110-111 küçük harflere ayrılmıştır. Yani sütun numarası, karakterin kategorisini anında belli eder.

$b_3b_2b_1b_0 \setminus b_6b_5b_4$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	'	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Tablo 1.6: Standart ASCII karakter tablosu (7 bit, $b_6 \dots b_0$).

Örnek 1.67: Harf 'A'nın ASCII Kodunun Bulunması

Tabloda 'A' harfi, sütun 100 ve satır 0001'de yer alır. Yani $b_6b_5b_4 = 100$, $b_3b_2b_1b_0 = 0001$, birleştirince: $(1000001)_2$.

Tabloya dikkatlice bakılırsa ilginç bir örüntü görülür: 'A' = 1000001, 'a' = 1100001. Aralarındaki tek fark b_5 bitidir: 0 ise büyük harf, 1 ise küçük harf. Bu, klavye ve yazılım tasarımını kolaylaştıran bilinçli bir tercihtir; büyük/küçük harf dönüşümü donanımda tek bir bitin çevrilmesine indirgenir.

Bilgi Notu 1.23: Neden 8 Bit (Byte) ile Saklanır?

ASCII yalnızca 7 bit gerektirdiği halde, bilgisayarlar veriyi 8-bitlik gruplar (*byte*) halinde işler. Fazladan kalan 8. bit boşa gitmez; genellikle **parity** (hata sezme) için kullanılır. Bir sonraki alt bölümün konusu tam olarak budur.

Alıştırma 1.20 ★ Harf 'Z'nin ASCII kodunu, yukarıdaki tablodan sütun/satır konumunu bularak yazınız.

Çözüm: 'Z' harfi, tabloda sütun 101 ve satır 1010'da yer alır. Yani $b_6b_5b_4 = 101$, $b_3b_2b_1b_0 = 1010$, birleştirince: $(1011010)_2$.

1.7.7 Hata Sezme: Parity Biti

Bir önceki alt bölümün sonunda bıraktığımız yerden devam edelim: ASCII 7 bit ($b_6 \dots b_0$) kullanılır, ama bilgisayarlar veriyi 8-bitlik byte'lar hâlinde işler. Fazladan kalan 8. bit genellikle **parity** için kullanılır. Bu bit, tam olarak b_7 'dir: 7-bit ASCII ($b_6 \dots b_0$) artı 1 parity biti (b_7), hiç boşluk kalmadan tam bir byte oluşturur.

Bir veri, bir kablodan veya bir depolama ortamından geçerken, elektriksel gürültü nedeniyle bir bit yanlışlıkla $0 \rightarrow 1$ veya $1 \rightarrow 0$ dönebilir. Parity biti, bu tür tek bitlik hataları sezme için

kullanılan basit bir yöntemdir.

Parity Biti: Parity biti, bir mesaja eklenen ve mesajdaki toplam 1 sayısını istenen bir pariteye (çift ya da tek) tamamlayan fazladan bir bittir.

- **Çift parity (even parity):** parity biti, toplam 1 sayısı çift olacak şekilde seçilir.
 - **Tek parity (odd parity):** parity biti, toplam 1 sayısı tek olacak şekilde seçilir.
- Sistemde ikisinden biri seçilir ve tutarlı biçimde kullanılır; çift parity daha yaygındır.

Örnek 1.68: Harf 'A' İçin Parity Biti Hesaplanması

'A' = 1000001 (7 bit). İçindeki 1 sayısını sayalım: 2 tanedir (zaten çift).

- Çift parity isteniyorsa, parity biti 0 kalır (toplam zaten çift): 01000001.
- Tek parity isteniyorsa, toplamı tek yapmak için parity biti 1 olur: 11000001.

Alıcı tarafta kontrol basittir: alıcı, gelen 8 bitteki 1 sayısını sayar. Beklenen pariteye (çift/tek) uymuyorsa, en az bir bitin yolda bozulduğu anlaşılır; bu durumda yeniden gönderim istenir (NAK, *Negative Acknowledge*, yani "olumsuz onay/red bildirimi"). Uyuyorsa aktarım onaylanır (ACK, *Acknowledge*, yani "onay bildirimi").

Bilgi Notu 1.24: Parity'nin Kritik Sınırlaması: Çift Sayıda Hatayı Yakalayamaz

Eğer **tek sayıda** bit (1, 3, 5, 7 tanesi) bozulursa, toplam 1 sayısının paritesi değişir ve hata sezilir. Ama **çift sayıda** bit (2, 4, ...) aynı anda bozulursa, toplam 1 sayısındaki değişimler birbirini götürür; parite değişmez ve hata sessizce kaçır.

Somut bir örnekle görelim: 'A'nın çift parity ile gönderilen hâli 01000001'dir (içinde 2 tane 1 vardır). Aktarım sırasında bu iki 1 biti de 0'a dönerse, alıcı 00000000 alır. Bu byte'taki 1 sayısı 0'dır, yine çift; parity kontrolü "hata yok" der, oysa veri tamamen farklıdır.

Bunun temel nedeni, parity bitinin veri hakkında çok kaba bir özet tutmasıdır: yalnızca toplam 1 sayısının tek mi çift mi olduğunu bilir, hangi bitlerin değiştiğine dair hiçbir bilgi taşımaz. İki bit birden değiştiğinde, birinin etkisi diğerininkini götürür ve toplam parite üzerindeki net etki sıfırlanır.

Peki bu zayıflığa rağmen parity neden hâlâ yaygın kullanılır? Çünkü gerçek dünyadaki gürültü kaynaklı hatalarda, aynı anda tam olarak iki (ya da dört, altı, ...) bitin birden bozulması istatistiksel olarak çok daha nadirdir. En sık karşılaşılan hata türü tek bir bitin bozulmasıdır, ve parity tam olarak bunu, tek bir ekstra bitle, ucuza yakalar. Daha güçlü hata sezme/düzeltilme gerektiren uygulamalarda (örn. bellek denetimi, ağ protokolleri) checksum, CRC (döngüsel artıklık denetimi) veya Hamming kodu gibi daha gelişmiş yöntemler kullanılır; bunlar bu kitabın kapsamı dışındadır.

Alıştırma 1.21 ★ A = 0101100 (7 bit) için çift parity bitini bulunuz.

Çözüm: 0101100 içindeki 1 sayısı: 3 tanedir (tek). Çift parity istediğimiz için, toplamı çift yapacak parity biti gerekir: $3 + 1 = 4$ (çift), yani parity biti 1'dir.

Bu, Bölüm 1.7'da başladığımız §1.7'nin son alt bölümüdür: BCD'den Gray koduna, ASCII'den parity'ye kadar, sayıların ve sembollerin ikili düzende nasıl kodlandığını ve bu kodlamalardaki hataların nasıl sezilebileceğini gördük.

Bool Cebiri ve Mantık Kapıları

Öğrenme Hedefleri

Bu bölümü tamamladığınızda aşağıdakileri yapabileceksiniz:

- İkili değişken ve mantık fonksiyonu kavramlarını tanımlamak.
- Bool cebirinin Huntington aksiyomlarını ve temel teoremlerini uygulamak.
- De Morgan teoremiyle Bool ifadelerini sadeleştirmek.
- Yedi temel mantık kapısının (AND, OR, NOT, NAND, NOR, XOR, XNOR) doğruluk tablolarını ve Bool ifadelerini yazmak.
- NAND/NOR kapılarının fonksiyonel tamlığının ASIC tasarımındaki önemini açıklamak.

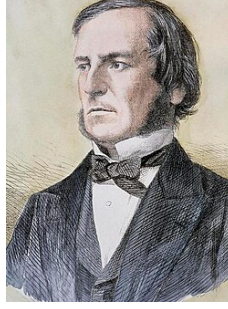
2.1 Giriş

Önceki bölümde gördüğümüz ikili sayılar, bir bilgisayarın belleğinde *durağan* veriyi temsil eder: bir sayı, bir karakter, bir renk kodu. Ama bir işlemci sadece veri saklamaz — toplama yapar, karşılaştırır, dallanma kararı verir. Yani veriyi *işleyen* bir mekanizmaya ihtiyacımız var; bu mekanizma da yine 0 ve 1’lerle, ama bu kez statik değil, girişe bağlı olarak çıkış üreten devrelerle kurulur. İşte bu devrelerin arkasındaki matematik, konumuz olan *Bool cebiridir*.

2.1.1 Bool Cebirinin Kısa Tarihçesi

Bool cebirinin hikayesi, matematiğin soyut bir kolunun onlarca yıl sonra mühendisliğin temel taşı hâline gelişinin klasik bir örneğidir.

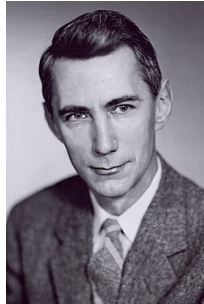
1815-1864 yılları arasında yaşayan, büyük ölçüde kendi kendini yetiştirmiş İngiliz matematikçi George Boole, 1854’te yayımladığı *An Investigation of the Laws of Thought* adlı eserinde, mantıksal önermeleri 0 ve 1 değerleriyle simgeleyen, toplama ve çarpmaya benzeyen işlemlerle işleyen cebirsel bir sistem kurdu. Bugün kullandığımız “Bool cebiri” (Boolean algebra) adı doğrudan onun soyadından gelir. Bu adlandırma kendisinden sonraki matematikçiler tarafından, onun onuruna yapılmıştır. Ne var ki Boole’un çalıştığı dönemde bu cebiri fiziksel bir devre üzerinde gerçekleyecek herhangi bir teknoloji yoktu; bu çalışma, seksen yıl boyunca saf matematiğin/mantık felsefesinin bir parçası olarak kaldı.



Şekil 2.1: Boole'un portresi, *The Illustrated London News*, 21 Ocak 1865.

Bu boşluğu, 1937'de MIT'de yüksek lisans öğrencisi olan Claude Shannon doldurdu. *A Symbolic Analysis of Relay and Switching Circuits* başlıklı tezinde (1938'de yayımlandı), Boole'un 0/1 cebirinin, elektromekanik röle ve anahtarlama devrelerini tasarlamak ve analiz etmek için birebir kullanılabileceğini gösterdi. Bu çalışma sık sık "20. yüzyılın en önemli yüksek lisans tezi" olarak anılır ve sayısal devre tasarımının (switching theory) kurucu belgesi kabul edilir. Boole'un soyut matematiğini ilk kez somut bir mühendislik aracına dönüştürdü.

Shannon'ın bilim tarihindeki ikinci büyük katkısı, bundan on yıl kadar sonra, 1948'de yayımlandığı *A Mathematical Theory of Communication* makalesiyle geldi; bu çalışmayla **bilgi teorisi-nin (information theory) kurucusu** kabul edilir. Bilginin nasıl ölçülebileceğini ve gürültülü bir kanaldan ne kadar güvenilir şekilde iletilebileceğini matematiksel olarak formüleştiren bu alan, bugünkü tüm sayısal iletişim (internet, cep telefonu, veri sıkıştırma) sistemlerinin teorik temelini oluşturur. Böylece Shannon, hem sayısal devre tasarımının hem de bilgi teorisinin kurucusu olarak, sayısal çağın mimarlarından biri kabul edilir.



Şekil 2.2: Claude Shannon, y. 1950'ler (Tekniska Museet).

Bu bölümde, Boole'un kurduğu ikili mantığın aksiyomlarını ve temel teoremlerini, Shannon'ın gösterdiği fiziksel karşılığı yani mantık kapılarını inceleyeceğiz.

2.1.2 İkili Değişkenler ve Mantık Fonksiyonları

İkili Değişken: Yalnızca iki değer alabilen bir değişkene *ikili değişken* denir. Bu değerler genellikle $\{0, 1\}$, $\{\text{DOĞRU (T), YANLIŞ (F)}\}$ ya da $\{\text{YÜKSEK (H), DÜŞÜK (L)}\}$ gerilim seviyeleri olarak yorumlanır. Sayısal devre tasarımında 0 ve 1 gösterimi kullanılır.

Bilgi Notu 2.1: Bir Devre 0 ve 1'i Fiziksel Olarak Nasıl Ayırt Eder?

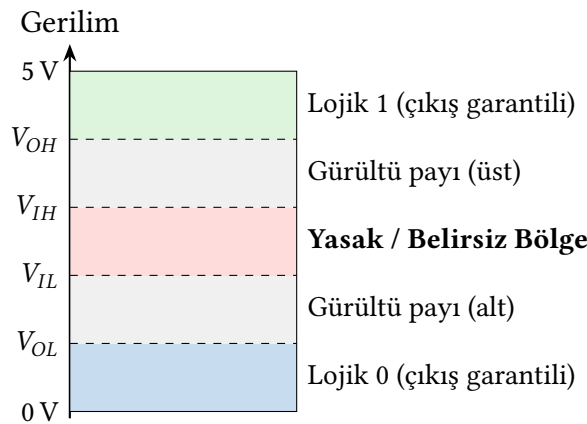
Bir sayısal devre, girişine gelen gerilimi bir *sayı* olarak okumaz. Gerilimin iki bölgeden hangisine düştüğüne bakar. Devredeki her kapı, girişindeki gerilimi bir **eşik değeriyle** (threshold) karşılaştıran bir tür karşılaştırmacı gibi davranır: gerilim belirli bir üst eşik üzerindeyse bu giriş 1 olarak, belirli bir alt eşik altındaysa 0 olarak okunur. Bu ikisinin arasında kalan, ne yeterince yüksek ne yeterince düşük, bir gerilim ise **tanımsız/yasak bölgededir**; devrenin normal çalışması sırasında burada hiç bulunulmaması beklenir, buraya yalnızca bir durumdan diğerine geçerken, çok kısa bir süreliğine uğranır. Bu eşiklerin standart mühendislik gösterimi ve tam anlamları aşağıdaki tablolarda açıklanmıştır.

Simge	Açılımı	Anlamı
V_{IL}	Voltage, I nput, L ow	Bir GİRİŞİN hâlâ Lojik 0 sayılacağı en yüksek gerilim
V_{IH}	Voltage, I nput, H igh	Bir GİRİŞİN Lojik 1 sayılacağı en düşük gerilim
V_{OL}	Voltage, O utput, L ow	Bir ÇIKIŞIN Lojik 0 için üreteceği garanti gerilim
V_{OH}	Voltage, O utput, H igh	Bir ÇIKIŞIN Lojik 1 için üreteceği garanti gerilim

Tablo 2.1: Gerilim eşiği simgelerinin açılımı.

Alt simgenin ilk harfi Giriş (I) ya da Çıkış (O) tarafını, ikinci harfi Yüksek (H) ya da Düşük (L) seviyeyi belirtir. Bu dört simge İngilizce “Input/Output” ve “High/Low” kelimelerinin baş harflerinden gelir.

Lojik Seviye	Gerilim Aralığı	Anlamı
Lojik 1	$V_{IH} - 5\text{ V}$	Devre bunu kesin 1 olarak okur
Belirsiz / Yasak	$V_{IL} - V_{IH}$	Geçerli bir çalışma durumu değildir
Lojik 0	$0\text{ V} - V_{IL}$	Devre bunu kesin 0 olarak okur

Tablo 2.2: Örnek gerilim eşikleri (5 V'luk bir sistem örneği).**Şekil 2.3:** Beş bölgeli sinyal seviyesi diyagramı (şematik, ölçekli değildir).

Gerçekte gürültü payı bantları çok daha incedir; burada okunaklı olması için abartılı çizilmiştir. Yaklaşık 5 V TTL (Transistör-Transistör Lojik, klasik bir dijital devre ailesi) değerleri: $V_{OL} =$

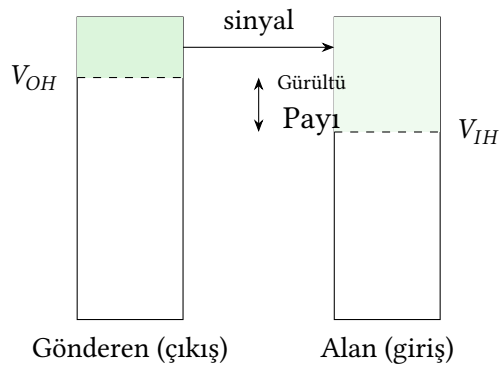
0.4, $V_{IL} = 0.8$, $V_{IH} = 2.0$, $V_{OH} = 2.4$. Bir kapının çıkışı yalnızca en dıştaki iki bölgede ($0-V_{OL}$ ya da $V_{OH}-5\text{ V}$) durur; bir sonraki kapının girişi ise daha geniş bir aralığı ($0-V_{IL}$ ya da $V_{IH}-5\text{ V}$) kabul eder. Aradaki iki ince şerit gürültü payıdır.

Detaylı Bilgi 2.1: Yasak Bölge Gerçekten Ne Anlama Gelir?

Yasak bölgenin asıl tehlikesi cihaza zarar vermesi değil, sinyalin anlamının belirsizleşmesidir: gerilim bu aralıkta kalırsa devre onu güvenilir şekilde 0 ya da 1 olarak sınıflandıramaz, sonuç üreticiye, sıcaklığa, hatta zamana göre değişebilir ve bu doğrudan bir mantık hatasına yol açar. Buna ek olarak gerçek bir fiziksel risk de vardır: özellikle CMOS teknolojisinde, giriş gerilimi bu ara bölgede kalırsa kapının içindeki hem p -tipi hem n -tipi transistörler aynı anda kısmen iletken hâle gelir ve besleme geriliminden toprağa doğrudan bir “şönt akımı” akar. Hızlı ve kısa geçişlerde (normal anahtarlama) bu kaçınılmazdır ve zararsızdır. Ama bir sinyal bu bölgede beklenenden uzun süre kalırsa (ör. çok yavaş yükselen/düşen bir kenar), bu şönt akımı gereksiz güç tüketimine ve ısınmaya yol açabilir. Özetle: tanımlayıcı/asıl sebep mantıksal belirsizlik, ısınma riski ise ikincil bir sonuçtur.

Detaylı Bilgi 2.2: Gürültü Payı (Noise Margin)

Bölüm 1’de, iki seviyeli gösterimin “seviyeler arasında büyük bir güvenlik payı bırakılabildiği” için güvenilir olduğunu söylemiştik; şimdi bunu netleştirebiliriz. Bir teli, birbirine bağlı iki kapı olarak düşünelim: **gönderen** kapı (çıkışı) ve **alan** kapı (girişi). Gönderen kapı hiçbir zaman keyfi bir gerilim üretmez: Lojik 1 göndermek istediğinde çıkışını en az V_{OH} ’a, Lojik 0 göndermek istediğinde en fazla V_{OL} ’a çeker. Bunlar üreticinin *garanti ettiği* değerleridir: 1 veya 0’ın kendisi değil, o değeri temsil eden gerilim sınırlarıdır. Alan kapının girişi ise daha **gevşek** davranır: V_{OH} kadar yüksek beklemez, V_{IH} kadarını da Lojik 1 olarak kabul eder ($V_{IH} \leq V_{OH}$); benzer şekilde V_{OL} kadar düşük beklemez, V_{IL} kadarını da Lojik 0 olarak kabul eder ($V_{IL} \geq V_{OL}$). Aradaki fark, $V_{OH} - V_{IH}$ ve $V_{IL} - V_{OL}$, işte *gürültü payı*: hat üzerinde oluşan küçük bir parazit ya da gerilim düşümü, bu payı aşmadığı sürece sinyal hâlâ doğru okunur.



Şekil 2.4: Gönderen ve alan kapı arasındaki gürültü payı.

Gönderen kapının garanti ettiği V_{OH} ile alan kapının kabul ettiği V_{IH} arasındaki boşluk gürültü payıdır: sinyal bu aralık kadar zayıflasa/parazit alsan bile alan kapı hâlâ Lojik 1 olarak okur.

Mantık Fonksiyonu: n adet ikili değişken $x_1, x_2, \dots, x_n$ alan ve tek bir ikili çıktı üreten $f : \{0, 1\}^n \rightarrow \{0, 1\}$ eşlemesine *mantık fonksiyonu* (Boole fonksiyonu) denir. Bir mantık fonksiyonu üç eşdeğer biçimde ifade edilebilir: **doğruluk tablosu**, **Bool ifadesi** ve **lojik devre şeması**. İlerleyen bölümlerde aynı fonksiyonu bu üç biçimde de üretip bir-biriyle karşılaştıracacağız.

Bilgi Notu 2.2: Neden Üç Eşdeğer Gösterim?

Doğruluk tablosu bir fonksiyonun *davranışını* kesin biçimde tanımlar ama büyüdükçe (2^n satır) okunması zorlaşır. Bool ifadesi kısa ve cebirsel işlemlere (sadeleştirme, De Morgan) uygundur. Devre şeması ise fiziksel gerçeklemeye en yakın gösterimdir; ilerleyen bölümlerde bu üç gösterimi aynı fonksiyon üzerinde üreteceğiz.

2.2 Cebirsel Yapı Kavramı

Bir sonraki bölümde Bool cebirinin kendi kurallarını (Huntington postülatları) göreceğiz. Ama bu kurallara "neden bunlar" diye sormadan geçmemek için, önce bir adım geriye gidip daha genel bir soru soralım: bir *cebirsel yapı* tam olarak nedir, ve bir matematiksel sistemi tanımlarken neden ispatsız kabul edilen *postülatlara* ihtiyaç duyarız?

2.2.1 Postülat Nedir?

Postülat (Aksiyom): Tümdengelimli (deductive) bir matematiksel sistemde, ispat gerektirmeden doğru kabul edilen ve sistemin tüm diğer önermelerinin (teoremlerinin) kendisinden mantıksal çıkarımla türetildiği temel önermeye *postülat* ya da *aksiyom* denir.

Bilgi Notu 2.3: Kelimenin Kökeni: Kabul mü, İspat mı?

"Postülat" kelimesi Fransızca *postulat* üzerinden Türkçeye geçmiştir; onun da kökeninde Latince *postulare* fiili vardır: "**istemek, talep etmek**." Kelimenin özünde "ispatlanmış" değil, "doğru kabul edilmesi **istenen**" bir önerme fikri yatar. Bir postülat, bir sistemin kurucusunun "bunu tartışmasız veri olarak kabul edelim" diye *talep ettiği* bir başlangıç noktasıdır. "Aksiyom" kelimesi de yakın bir akrabadır: Yunanca *axioma* ("değerli/layık görülen şey"), kökü *axioun* ("değerli görmek"). Yani bir aksiyom, "kendiliğinden o kadar açık ki ispat gerektirmeyecek kadar değerli görülen" bir önermedir. Eski Yunan'da (Euclid) bu ikisi ayrı tutulurdu: tüm bilimler için geçerli "ortak kavramlar" (ör. "eşitlere eşit eklenirse sonuç eşit olur") ile yalnızca geometriye özgü "postülatlar" (ör. "iki nokta arasından bir doğru çizilebilir"). Günümüzde bu ayrım büyük ölçüde kalkmış, iki terim neredeyse eşanlamlı kullanılıyor. Ama postülatın kökeninde hâlâ o "kabul edilmesi talep edilen" vurgu durur.

Bilgi Notu 2.4: Neden İspatsız Kabul Ederiz?

Bunun adı *sonsuz gerileme* (infinite regress) sorunudur: her önermeyi bir öncekinden ispatlamaya kalkarsak, ispat zinciri hiç bitmez. Bir yerde durup “bunu veri kabul ediyorum” demek zorundayız. Euclid geometrisinde “nokta” ve “doğru” kavramları bile tanımsız temel terimlerdir; onları başka bir şeyden tanımlamaya çalışırsanız yine sonsuz gerilemeye düşersiniz. Aynı mantık, doğal sayıları tanımlayan Peano aksiyomlarında ve ileride göreceğimiz Huntington’ın Bool cebiri postülatlarında da geçerlidir. Günlük hayattan bir benzetme: bir anayasanın temel maddeleri de başka bir kanundan “ispatlanmaz”, toplumsal bir uzlaşmayla *kabul edilir*. Diğer tüm kanunlar bu maddelerden türetilir. Ya da bir oyunun kuralları: “futbolda topa elle dokunulmaz” kanıtlanamaz, oyunun tanımının kendisidir. Huntington da 1904’te Bool cebirini tanımlarken tam olarak bunu yaptı: aşağıda göreceğimiz 6 postülatı *seçti*, ispatlamadı. Bool cebirinin tüm teoremleri bu 6 kuraldan türetililecektir.

2.2.2 Küme ve İkili İşlem

Küme: Ortak bir özelliği olan nesneler topluluğuna *küme* denir. x nesnesi S kümesinin bir üyesiye $x \in S$, değilse $x \notin S$ yazılır. Sonlu sayıda elemanı olan bir küme süslü parantezle gösterilir: $A = \{1, 2, 3, 4\}$.

İkili İşlem: Bir S kümesi üzerinde tanımlı *ikili işlem* (binary operator), S ’nin her eleman çiftine yine S ’den tek bir eleman atayan kuraldır. Örneğin $a * b = c$ ilişkisinde, eğer $a, b \in S$ olduğunda her zaman $c \in S$ oluyorsa, $*$ işlemi S üzerinde bir ikili işlemdir.

Örnek 2.1: Kapalı Olmayan Bir İşlem

Doğal sayılar kümesi $N = \{1, 2, 3, \dots\}$, toplama işlemi altında kapalıdır: her $a, b \in N$ için $a + b \in N$ ’dir. Ama aynı küme, çıkarma işlemi altında kapalı **değildir**: $2, 3 \in N$ olduğu hâlde $2 - 3 = -1 \notin N$ ’dir. Bu yüzden çıkarma, N üzerinde bir ikili işlem değildir.

2.2.3 Cebirsel Yapı İçin Altı Genel Postülat

Bir küme, üzerinde tanımlı ikili işlem(ler) ve bu işlemlerin uyduğu postülatlarla birlikte bir *cebirsel yapı* oluşturur. Çeşitli cebirsel yapıları tanımlamakta en sık kullanılan altı postülat şunlardır:

Genel Postülatlar:

- **Kapalılık (Closure):** S üzerindeki her eleman çifti için işlem, yine S ’den bir eleman üretir.
- **Birleşme (Associative):** her $x, y, z \in S$ için $(x * y) * z = x * (y * z)$.
- **Değişme (Commutative):** her $x, y \in S$ için $x * y = y * x$.
- **Birim Eleman (Identity):** öyle bir $e \in S$ vardır ki her $x \in S$ için $e * x = x * e = x$.
- **Ters Eleman (Inverse):** birim elemanı e olan bir yapıda, her $x \in S$ için öyle bir $y \in S$ vardır ki $x * y = e$.

- **Dağılma (Distributive):** S üzerinde $*$ ve $\#$ iki ayrı işlemse, $*$ 'in $\#$ üzerine dağılması şu demektir: $x * (y\#z) = (x * y)\#(x * z)$.

Örnek 2.2: Somut Bir Cebirsel Yapı: Cisim (Field)

Gerçek sayılar kümesi, $+$ ve $\times$ işlemleriyle birlikte bir *cisim* (field) oluşturur; bu, sıradan aritmetiğin ve cebirin temelidir. Altı genel postülat burada şöyle karşılık bulur:

- Toplamsal birim eleman: 0 (çünkü $x + 0 = x$).
- Çarpımsal birim eleman: 1 (çünkü $x \times 1 = x$).
- Toplamsal ters eleman **çıkarmayı** tanımlar: a 'nın tersi $-a$ 'dır, çünkü $a + (-a) = 0$.
- Çarpımsal ters eleman **bölmeyi** tanımlar: $a \neq 0$ için a 'nın tersi $1/a$ 'dır, çünkü $a \times (1/a) = 1$.
- Geçerli tek dağılma kuralı, $\times$ 'in $+$ üzerine dağılmasıdır: $a \times (b + c) = (a \times b) + (a \times c)$. Ters, yani $+$ 'nin $\times$ üzerine dağılması, sıradan cebirde **geçerli değildir**.

Bilgi Notu 2.5: Bool Cebiri Nereye Oturuyor?

Bool cebiri de tam olarak bu çerçeveye uyan bir cebirsel yapıdır: bir küme ($B = \{0, 1\}$), iki ikili işlem ($+$ ve $\cdot$) ve bu işlemlerin uyduğu postülatlar. Ama bir sonraki bölümde göreceğimiz gibi, Huntington'ın seçtiği postülatlar yukarıdaki altı genel postülatın **hepsini içermez: ters eleman postülatı yoktur**. Dolayısıyla Bool cebirinde çıkarma ve bölme işlemleri de yoktur. Buna karşılık cisimde bulunmayan bir postülat, *tümleme* (complement), Bool cebirine özgü olarak eklenir. Bu, Bool cebirinin neden sıradan cebirden temelden farklı davrandığının ilk ipucudur.

2.3 Bool Cebirinin Aksiyomları

Bool cebiri, Öklid geometrisi gibi *aksiyomatik* bir sistemdir: az sayıda temel varsayımdan (aksiyom) yola çıkılır ve cebirin tüm teoremleri bu varsayımlardan mantıksal çıkarımla türetilir. Boole'un 1854'teki özgün formülasyonunu ve Shannon'ın bunu anahtarlama devrelerine uyarlamasını Bölüm 2.1.1'de görmüştük. Burada kullanacağımız kesin/formal tanım ise matematikçi Edward V. Huntington'ın 1904'te önerdiği altı postülatı dayanır.

Huntington (1874–1952), Harvard'da matematik profesörü olarak çalışmış bir Amerikalı matematikçiydi; cebirsel yapıların aksiyomatik temellerine katkılarıyla tanınır. 1904'te Boole'un cebirini altı postülatı indirgeyerek bugün kullandığımız formu ortaya koydu.



Şekil 2.5: Edward V. Huntington (1874–1952).

Bu postülatlar altı tanedir: beşi toplama (+) ve çarpma (·) için ayrı ayrı ifade edilen çiftler hâlinde, biri (altıncısı) ise düali olmayan tekil bir kuraldır. Bu çiftlenme, ilerleyen kısımda göreceğimiz *düalite ilkesinin* kaynağıdır.

Huntington Postülatları: $B = \{0, 1\}$ kümesi, + (OR) ve · (AND) ikili işlemleri ile ' (NOT) tekli işlemi altında, her $x, y, z \in B$ için aşağıdaki altı özdeşliği sağlıyorsa *Bool cebiri* adını alır:

No	Postülat	Postülat (düali)	Adı
1	$x + y \in B$	$x \cdot y \in B$	Kapalılık
2	$x + 0 = x$	$x \cdot 1 = x$	Birim eleman
3	$x + y = y + x$	$x \cdot y = y \cdot x$	Değişme
4	$x + (yz) = (x + y)(x + z)$	$x(y + z) = xy + xz$	Dağılma
5	$x + x' = 1$	$x \cdot x' = 0$	Tümleme
6	B' 'de $x \neq y$ olacak şekilde en az iki eleman vardır		Farklılık

Tablo 2.3: Huntington Postülatları

2.3.1 Sıradan Cebirle Karşılaştırma

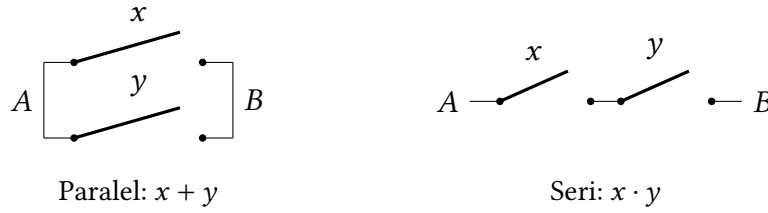
Bool cebirini sıradan cebirle (reel sayılar, + ve ×) karşılaştırmak hem farklarını netleştirir hem de Bölüm 2.2.3'da tanıdığımız genel çerçeveye nasıl oturduğunu somutlaştırır:

1. Huntington postülatları **birleşme (associative)** kuralını içermez. Ama bu kural Bool cebirinde de geçerlidir ve diğer postülatlardan türetilir. Bunu Bölüm 2.4'de teorem olarak göreceğiz.
2. + 'nin · üzerine dağılması ($x + (yz) = (x + y)(x + z)$, Postülat 4) Bool cebirinde geçerlidir, ama sıradan cebirde **geçerli değildir** (ör. $x=2, y=3, z=4$ için sol taraf 14, sağ taraf 30 verir). Bool cebirinde geçerli olmasının nedeni, değişkenlerin yalnızca $\{0, 1\}$ değerlerini alması: bu sonlu kümede tüm $2^3 = 8$ kombinasyon tek tek denenerek özdeşlik doğrulanabilir. Bu ispat yöntemine *mükemmel tümevarım* (perfect induction) denir; bir sonraki alt bölümde bu yöntemi bizzat uygulayacağız.
3. Bölüm 2.2.3'da gördüğümüz gibi, Huntington postülatlarında **ters eleman yoktur**; dolayısıyla Bool cebirinde çıkarma ve bölme işlemleri de yoktur.
4. Yine Bölüm 2.2.3'da değindiğimiz gibi, Postülat 5'in tanımladığı **tümleme** (complement) işlemi sıradan cebirde hiç yoktur.

5. Sıradan cebir, sonsuz elemanlı reel sayılar kümesi üzerine kuruludur. Bool cebiri ise genel/tanımsız bir B kümesi üzerine kuruludur. Bizim kullanacağımız **iki-değerli** özel hâlde ise $B = \{0, 1\}$ 'dir; bunu bir sonraki alt bölümde doğrulayacağız.

Bilgi Notu 2.6: Anahtarlama Devresi Yorumu

Shannon'ın 1938'de gösterdiği gibi, bu postülatlar fiziksel bir anahtarlama devresi olarak okunabilir: $x + y$ iki anahtarın **paralel** bağlanması (biri kapalıysa akım geçer), $x \cdot y$ **seri** bağlanması (ikisi de kapalı olmalı), x' ise x 'in tersidir (biri kapalıyken diğeri açıktır). Şekil 2.6 bu iki bağlantıyı gösterir.

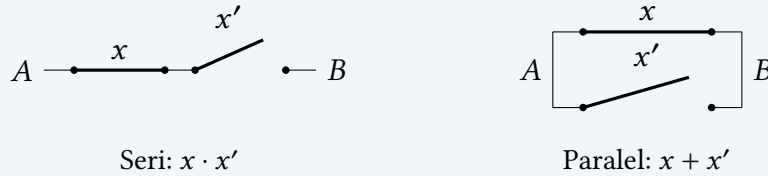


Şekil 2.6: Paralel ($x + y$) ve seri ($x \cdot y$) anahtar bağlantısı.

Soldaki paralel bağlantıda A 'dan B 'ye akım geçmesi için x ya da y 'den en az birinin kapalı olması yeterlidir. Sağdaki seri bağlantıda ise akım geçmesi için hem x hem y 'nin kapalı olması gerekir.

Örnek 2.3: Postülat 2'nin Anahtarlama Devresiyle Doğrulanması

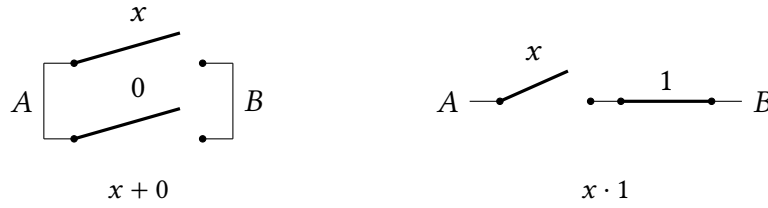
x ve x' tanım gereği her zaman zıt durumdadır: biri kapalıyken diğeri mutlaka açıktır.



Şekil 2.7: Postülat 2'nin ($x + x' = 1$, $x \cdot x' = 0$) anahtarlama devresi doğrulanması.

- **Seri bağlantı ($x \cdot x'$):** Akım geçmesi için *ikisinin de* kapalı olması gerekir. Zıt oldukları için bu hiçbir zaman gerçekleşmez $\Rightarrow x \cdot x' = 0$.
- **Paralel bağlantı ($x + x'$):** Akım geçmesi için *en az birinin* kapalı olması yeterlidir. Zıt oldukları için her zaman biri kapalıdır $\Rightarrow x + x' = 1$.

Alıştırma 2.1 ★ Postülat 1'i ($x + 0 = x$ ve $x \cdot 1 = x$) yukarıdaki örnekteki gibi anahtarlama devresi yorumuyla açıklayınız: 0 değerindeki bir anahtar her zaman açık, 1 değerindeki bir anahtar her zaman kapalıdır. Bu anahtarın devreye paralel ya da seri eklenmesi neden devrenin davranışını değiştirmez?



Şekil 2.8: Postülat 1'in $(x + 0 = x, x \cdot 1 = x)$ anahtarlama devresi doğrulaması.

Çözüm: 0 değerindeki bir anahtar her zaman *açık*tır, yani hiçbir zaman akım geçirmez. Böyle bir anahtar x 'e **paralel** eklendiğinde $(x + 0)$, devreye ek bir akım yolu sağlamaz. Akımın geçip geçmemesi hâlâ tamamen x 'e bağlıdır, dolayısıyla $x + 0 = x$. 1 değerindeki bir anahtar ise her zaman *kapalı*dır, yani her zaman akım geçirir. Böyle bir anahtar x 'e **seri** eklendiğinde $(x \cdot 1)$, akımı hiçbir zaman engellemez. Yine akımın geçip geçmemesi tamamen x 'e bağlı kalır, dolayısıyla $x \cdot 1 = x$.

2.3.2 İki-Değerli Bool Cebrinin Doğrulanması

Şimdiye kadar Huntington postülatlarını genel bir B kümesi için tanımladık. Ama bizim asıl ilgilendiğimiz özel durum, $B = \{0, 1\}$ olan *iki-değerli* Bool cebridir. Bu kümede $+$, $\cdot$ ve $'$ işlemlerini aşağıdaki tablolarla tanımlıyoruz:

x	y	$x \cdot y$	x	y	$x + y$	x	x'
0	0	0	0	0	0	0	1
0	1	0	0	1	1	1	0
1	0	0	1	0	1		
1	1	1	1	1	1		

Tablo 2.4: İki-değerli Bool cebrinin işlem tabloları.

Bu tablolar, sırasıyla AND, OR ve NOT işlemleriyle birebir aynıdır. Şimdi altı postülatın bu tablolar üzerinde gerçekten sağlandığını, *mükemmel tümevarımla* (perfect induction, yani sonlu sayıdaki tüm kombinasyonu tek tek deneyerek) tek tek göstereyim:

1. **Kapalılık:** Tablolardaki her sonuç 0 ya da 1'dir, yani her zaman B 'nin içindedir.
2. **Birim eleman:** Tablodan $0 + 0 = 0, 0 + 1 = 1 + 0 = 1$; yani $x + 0 = x$. Benzer şekilde $1 \cdot 1 = 1, 1 \cdot 0 = 0 \cdot 1 = 0$; yani $x \cdot 1 = x$.
3. **Değişme:** Tabloların köşegene göre simetrik olması, $x + y = y + x$ ve $x \cdot y = y \cdot x$ 'i doğrudan gösterir.
4. **Dağılma:** $x \cdot (y + z) = xy + xz$ özdeşliği, olası $2^3 = 8$ kombinasyonun tümü için Tablo 2.5'da doğrulanmıştır (son iki sütun her satırda eşittir). $+$ 'nin $\cdot$ üzerine dağılması da benzer bir tabloyla doğrulanabilir.
5. **Tümleme:** x' tablosundan, $x + x' = 1$ ($0 + 1 = 1, 1 + 0 = 1$) ve $x \cdot x' = 0$ ($0 \cdot 1 = 0, 1 \cdot 0 = 0$) her iki x değeri için de sağlanır.
6. **Farklılık:** B kümesinde $1 \neq 0$ olacak şekilde iki eleman vardır.

Böylece iki-değerli Bool cebrinin gerçekten bir Bool cebri olduğunu, yani altı Huntington postülatını da sağladığını göstermiş olduk. Mühendisler bu yapıya *anahtarlama cebri* (switching algebra) da der. Bu, Bölüm 2.1.2'de tanıdığımız AND/OR/NOT işlemleriyle birebir aynı şeydir. Bundan sonra "iki-değerli" sıfatını düşürüp sade *Bool cebri* diyeceğiz.

x	y	z	$y + z$	$x \cdot (y + z)$	xy	xz	$xy + xz$
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Tablo 2.5: Dağılma postülatının $(x \cdot (y + z) = xy + xz)$ mükemmel tümevarımla doğrulanması.

2.4 Döualite İlkesi ve Temel Teoremler

Döualite İlkesi: Bir Bool özdeşliđinin *döuali*, ifadedeki her $+$ ile $\cdot$ 'nin, her 0 ile 1'in yer deđiştirilmesiyle elde edilir. Postülatların her biri kendi döualiyle çift geldiđi için (bkz. Tablo 2.3), bu çiftlenme **tüm türetilmiř teoremlere de yansır**: bir teoremi postülatlardan kanıtladıđınızda, döualini kanıtlamak için hiřbir ek iřlem gerekmez. Kanıttaki her satırda aynı yer deđiřtirmeyi uygulamak yeterlidir.

Bunu somutlařtırmak için, listedeki ilk teoremi dođrudan postülatlardan türetilim.

Örnek 2.4: Teorem 1'in Postülatlardan Türetilmesi: $x + x = x$

$$\begin{aligned}
 x + x &= (x + x) \cdot 1 && [\text{Postülat 2: } y \cdot 1 = y, \text{ burada } y = x + x] \\
 &= (x + x)(x + x') && [\text{Postülat 5: } 1 = x + x'] \\
 &= x + x \cdot x' && [\text{Postülat 4: } (x + y)(x + z) = x + yz] \\
 &= x + 0 && [\text{Postülat 5: } x \cdot x' = 0] \\
 &= x && [\text{Postülat 2: } x + 0 = x]
 \end{aligned}$$

Bilgi Notu 2.7: Döualitenin Getirisi: İkinci Teorem Bedava

Yukarıdaki kanıtın her satırında $+$ $\leftrightarrow \cdot$ ve $0 \leftrightarrow 1$ deđiřimini uygularsanız, $x \cdot x = x$ özdeşliđinin kanıtını elde edersiniz. Tek bir ek adım atmadan. Bu yüzden Bool cebiri kaynaklarında teoremler genellikle çift çift, tek bir kanıtla sunulur.

Somut olarak görelim: yukarıdaki beř satırın her birinde $+$ $\leftrightarrow \cdot$ ve $0 \leftrightarrow 1$ deđiřimini uyguladıđımızda, Teorem 1(b)'nin ispatı ortaya ıkar:

$x \cdot x = x \cdot x + 0$	[Postülat 2: $x + 0 = x$, burada $x \rightarrow x \cdot x$]
$= x \cdot x + x \cdot x'$	[Postülat 5: $x \cdot x' = 0$]
$= x \cdot (x + x')$	[Postülat 4: $xy + xz = x(y + z)$, dağılma]
$= x \cdot 1$	[Postülat 5: $x + x' = 1$]
$= x$	[Postülat 2: $x \cdot 1 = x$]

Aynı türetme yöntemiyle elde edilen diğer temel teoremler, ileride sadeleştirme işlemlerinde sürekli kullanacağımız için aşağıda topluca listelenmiştir.

No	Teorem	Teorem (düali)	Adı
1	$x + x = x$	$x \cdot x = x$	Yinelenirlik (idempotence)
2	$x + 1 = 1$	$x \cdot 0 = 0$	Yutan eleman
3	$(x')' = x$	—	Çifte tümleme
4	$x + xy = x$	$x(x + y) = x$	Yutma (absorption)
5	$x + (y + z) = (x + y) + z$	$x(yz) = (xy)z$	Birleşme (associative)

Tablo 2.6: Temel Teoremler

Örnek 2.5: Teorem 2'nin İspatı: $x + 1 = 1$

$x + 1 = 1 \cdot (x + 1)$	[Postülat 2: $x \cdot 1 = x$, tersten]
$= (x + x')(x + 1)$	[Postülat 5: $x + x' = 1$, tersten]
$= x + x' \cdot 1$	[Postülat 4: $(x + y)(x + z) = x + yz$, dağılma]
$= x + x'$	[Postülat 2: $x \cdot 1 = x$]
$= 1$	[Postülat 5: $x + x' = 1$]

Düal teorem $x \cdot 0 = 0$, düalite ilkesiyle (her satırda $+ \leftrightarrow \cdot$, $0 \leftrightarrow 1$ değişimiyle) doğrudan elde edilir.

Bilgi Notu 2.8: Teorem 3: Çifte Tümleme Neden Farklı İspatlanır?

$(x')' = x$ teoremi, diğerlerinden farklı bir ispat yöntemi gerektirir: postülat zincirinden değil, her elemanın tümleyeninin **bir tane olmasından** (unique) gelir. Postülat 5'e göre x' , $x + x' = 1$ ve $x \cdot x' = 0$ 'ı sağlayan (tek) elemandır. Şimdi x'' 'in kendi tümleyenine, yani $(x')'$ 'e bakalım: bu da $x' + (x')' = 1$ ve $x' \cdot (x')' = 0$ 'ı sağlamalıdır. Ama x zaten bu iki denklemi sağlıyor ($x' + x = 1$ ve $x' \cdot x = 0$, Postülat 3 ve 5'ten). Bir elemanın tümleyeni yalnızca bir tane (unique) olduğu için, $(x')' = x$ olmak zorundadır.

Alıştırma 2.2 ★★ Teorem 4'ü ($x + xy = x$) yukarıdaki Teorem 1 kanıtını örnek olarak postülatlardan türetiniz. *İpucu:* x' 'i önce $x \cdot 1$ olarak yazıp Postülat 4'ü (dağılma) tersten uygulamayı deneyin.

Çözüm:

$$\begin{aligned}
x + xy &= x \cdot 1 + x \cdot y && [\text{Postülat 2: } x = x \cdot 1] \\
&= x \cdot (1 + y) && [\text{Postülat 4: } xy_1 + xy_2 = x(y_1 + y_2), \text{ dağılma}] \\
&= x \cdot 1 && [\text{Teorem 2: } 1 + y = 1] \\
&= x && [\text{Postülat 2: } x \cdot 1 = x]
\end{aligned}$$

Bilgi Notu 2.9: İspatların İkinci Yolu: Doğruluk Tablosu

Bool teoremleri, postülat zincirinden yürüyen cebirsel ispatlar yerine **doğruluk tablosu**yla da ispatlanabilir: eşitliğin iki tarafı, değişkenlerin tüm olası kombinasyonlarında aynı sonucu veriyorsa, özdeşlik doğrulanmış olur (bu da bir tür mükemmel tümevarımdır, Bölüm 2.3.2’de kullandığımız yöntemin aynısı). Örneğin yutma teoremi ($x + xy = x$) şöyle doğrulanır:

x	y	xy	$x + xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Son sütun ($x + xy$) baştaki x sütunuyla birebir aynıdır. Teorem doğrulanmıştır. Birleşme ve De Morgan gibi teoremlerin cebirsel ispatları uzun olduğundan, pratikte doğruluk tablosu yöntemi çok daha kullanışlıdır; De Morgan’ı ele alırken bu yöntemi tekrar kullanacağız.

2.4.1 Operatör Önceliği

Bir Bool ifadesini doğru değerlendirmek için işlem önceliği şu sırayla uygulanır: (1) parantez içi, (2) tümlleme ($'$), (3) çarpma ($\cdot$, AND), (4) toplama ($+$, OR). Parantez içindeki her şey önce hesaplanır; sonra tümlleme, sonra çarpma, en son toplama işlemi uygulanır. Bu sıralama, çarpmanın toplamadan önce yapıldığı sıradan aritmetikte uyumludur (tümlleme hariç, bu sıradan aritmetikte karşılığı olmayan bir adımdır).

Örnek 2.6: İşlem Önceliğinin Uygulanması

$(x + y)'$ ifadesinde önce parantez içi $x + y$ hesaplanır, sonra sonucun tümleyeni alınır. $x'y'$ ifadesinde ise önce x' ve y' ayrı ayrı hesaplanır (tümlleme), sonra ikisi çarpılır (AND). Bu iki ifade farklı bir sırayla değerlendirilir; birbirlerine eşit olup olmadıklarını De Morgan teoremini işlerken göreceğiz.

Alıştırma 2.3 ★★ Postülat ve teoremleri kullanarak şu ifadeyi sadeleştiriniz: $F = x'y'z + xyz + x'yz + xy'z$.

Çözüm:

$$\begin{aligned}
F &= x'y'z + xyz + x'yz + xy'z \\
&= (x'y'z + x'yz) + (xyz + xy'z) && \text{[yeniden gruplama]} \\
&= x'z(y' + y) + xz(y + y') && \text{[Postülat 4: dağılma]} \\
&= x'z \cdot 1 + xz \cdot 1 && \text{[Postülat 5: } y + y' = 1 \text{]} \\
&= x'z + xz && \text{[Postülat 2: } x \cdot 1 = x \text{]} \\
&= z(x' + x) && \text{[Postülat 4: dağılma]} \\
&= z \cdot 1 && \text{[Postülat 5: } x' + x = 1 \text{]} \\
&= z && \text{[Postülat 2: } x \cdot 1 = x \text{]}
\end{aligned}$$

Alıştırma 2.4 ★ $F = x'y'z$ ifadesinin doğruluk tablosunu oluşturunuz.**Çözüm:**

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

F yalnızca $x = 0$, $y = 0$, $z = 1$ olduğunda 1'dir, çünkü ancak bu durumda $x' = 1$, $y' = 1$ ve $z = 1$ olup üçünün çarpımı 1 olur.

2.5 De Morgan Teoremi

Augustus De Morgan (1806–1871), Hindistan'da doğmuş, University College London'ın ilk matematik profesörü olarak görev yapmış, İngiliz matematik ve mantık bilimciydi. Mantık cebirine kesin/biçimsel bir temel kazandırma çabalarıyla tanınır; bu bölümde adını taşıyan iki özdeşlik de onun bu çalışmalarından gelir.



Şekil 2.9: Augustus De Morgan (1806–1871).

De Morgan Teoremi: Her $x, y \in B$ için aşağıdaki iki özdeşlik geçerlidir:

$$(x + y)' = x' y' \quad (xy)' = x' + y'$$

Bu teoremin cebirsel ispatı biraz uzundur, ama görmeye değer: kullandığı yöntem, Bölüm 2.4'de Teorem 3'te $((x')' = x)$ gördüğümüz **tümleyen bir tane olması** (unique) ilkesinin bir başka güzel uygulamasıdır.

Örnek 2.7: De Morgan'ın İlk Yarısının İspatı: $(x + y)' = x' y'$

Bir elemanın tümleyeni, onunla toplandığında 1, çarpıldığında 0 veren **bir tane (unique)** elemandır (Postülat 5 ve Bölüm 2.4'deki Teorem 3'ün ispatı). O hâlde $(x + y)'$ 'nin ne olduğunu bulmak için, $x' y'$ 'nin bu iki şartı sağladığını göstermemiz yeterli:

Şart 1: $(x + y) \cdot x' y' = 0$

$$\begin{aligned} (x + y) \cdot x' y' &= x \cdot x' y' + y \cdot x' y' && [\text{Postülat 4: dağılma}] \\ &= (xx')y' + x'(yy') && [\text{Postülat 3: yeniden grupta}] \\ &= 0 \cdot y' + x' \cdot 0 && [\text{Postülat 5: } xx' = 0, yy' = 0] \\ &= 0 + 0 && [\text{Teorem 2: } x \cdot 0 = 0] \\ &= 0 \end{aligned}$$

Şart 2: $(x + y) + x' y' = 1$

$$\begin{aligned} (x + y) + x' y' &= [(x + y) + x'][(x + y) + y'] && [\text{Postülat 4: } a + bc = (a + b)(a + c)] \\ &= [(x + x') + y][x + (y + y')] && [\text{Postülat 3: yeniden grupta}] \\ &= [1 + y][x + 1] && [\text{Postülat 5: } x + x' = 1, y + y' = 1] \\ &= 1 \cdot 1 && [\text{Teorem 2: } 1 + y = 1] \\ &= 1 \end{aligned}$$

$x' y'$, $(x + y)$ 'nin tümleyeni olma şartlarının ikisini de sağladı; tümleyen bir tane (unique) olduğu için $(x + y)' = x' y'$ olmak zorundadır.

Bilgi Notu 2.10: İkinci Yarı Düallite ile Bedava

$(xy)' = x' + y'$, yukarıdaki ispatın her satırında $+ \leftrightarrow \cdot$ ve $0 \leftrightarrow 1$ değişimi uygulanarak doğrudan elde edilir; Bölüm 2.4'de gördüğümüz düallite ilkesinin bir başka örneğidir.

2.5.1 Doğruluk Tablosuyla Doğrulama

Bölüm 2.3.2'da tanıdığımız yöntemle, De Morgan teoreminin ilk yarısını $((x + y)' = x' y')$ bir doğruluk tablosuyla da doğrulayabiliriz:

x	y	$x + y$	$(x + y)'$	x'	y'	$x'y'$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Tablo 2.7: $(x + y)' = x'y'$ özdeşliğinin doğruluk tablosuyla doğrulanması.

Son iki sütun $((x + y)'$ ve $x'y'$) her satırda birebir aynıdır; özdeşlik doğrulanmıştır.

Alıştırma 2.5 ★ İkinci yarıyı $((xy)' = x' + y')$ doğrulayan doğruluk tablosunu oluşturunuz.

Çözüm:

x	y	xy	$(xy)'$	x'	y'	$x' + y'$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

Son iki sütun $((xy)'$ ve $x' + y'$) yine her satırda birebir aynıdır.

2.5.2 n -Değişkenli Genelleme

İki değişken için kanıtladığımız De Morgan teoremi, aslında herhangi bir n değişken için de geçerlidir:

De Morgan Teoreminin Genel Hâli: Her $n \geq 2$ ve $x_1, x_2, \dots, x_n \in B$ için:

$$(x_1 + x_2 + \dots + x_n)' = x_1'x_2' \dots x_n' \quad (x_1x_2 \dots x_n)' = x_1' + x_2' + \dots + x_n'$$

Sözel olarak: bir toplamın tümleyeni, terimlerin tümleyenlerinin çarpımına eşittir; bir çarpımın tümleyeni, terimlerin tümleyenlerinin toplamına eşittir, kaç terim olursa olsun.

Bilgi Notu 2.11: Kanıt Yöntemi: Matematiksel Tümevarım (Induction)

Bunu ispatlamanın doğal yolu *matematiksel tümevarım* (mathematical induction). Bu yöntem iki adımdan oluşur: (1) **temel adım**, iddianın en küçük durumda ($n = 2$) doğru olduğunu göstermek (bunu zaten yaptık); (2) **tümevarım adımı**, iddianın $n = k$ için doğru olduğunu *varsayıp*, bu varsayımı kullanarak $n = k + 1$ için de doğru olduğunu göstermek. Bu iki adım birlikte, iddianın $n = 2, 3, 4, \dots$ diye **sonsuz kadar tüm** n değerleri için doğru olduğunu garanti eder. Tıpkı bir domino dizisinde ilk taşı devirmenin ve “her taş devrilirse bir sonraki de devrilir” kuralının, tüm dizinin devrileceğini garanti etmesi gibi.

Örnek 2.8: Tümevarım Adımının Uygulanması

Toplamla ilgili yarıyı ele alalım. **Tümevarım hipotezi:** $n = k$ için $(x_1 + \dots + x_k)' = x_1' \dots x_k'$ doğru olsun. Şimdi $n = k + 1$ için:

$$\begin{aligned}
 (x_1 + \dots + x_k + x_{k+1})' &= [(x_1 + \dots + x_k) + x_{k+1}]' && \text{[gruplama]} \\
 &= (x_1 + \dots + x_k)' \cdot x_{k+1}' && \text{[2-değişkenli De Morgan]} \\
 &= (x_1' \dots x_k') \cdot x_{k+1}' && \text{[tümevarım hipotezi]} \\
 &= x_1' \dots x_k' x_{k+1}'
 \end{aligned}$$

Bu tam istediğimiz sonuç: $k+1$ terim için de özdeşlik sağlanıyor. $n = 2$ temel adımı zaten kanıtlanmıştı ve tümevarım adımı da gösterildiğine göre, teorem tüm $n \geq 2$ için doğrudur. Çarpımla ilgili yarı $((x_1 \dots x_n)' = x_1' + \dots + x_n')$ da aynı yöntemle, düalete kullanılarak ispatlanır.

Örnek 2.9: Dört Değişkenli Bir Uygulama

$F = (a + b + c + d)'$ ifadesinin tümleyensiz hâlini bulalım. Genel De Morgan formülünü doğrudan uygularız:

$$F = (a + b + c + d)' = a'b'c'd'$$

Benzer şekilde $G = (pqrs)'$ için $G = p' + q' + r' + s'$ olur.

Alıştırma 2.6 ★★ $H = (w' + x + y' + z)'$ ifadesini genel De Morgan formülüyle sadeleştiriniz. *İpucu:* önce toplamın tümleyenini alın, sonra her terimin kendi tümleyenini tekrar sadeleştirin (Bölüm 2.4'deki Teorem 3'ü, $(x')' = x$, hatırlayın).

Çözüm:

$$\begin{aligned}
 H &= (w' + x + y' + z)' = (w')' \cdot x' \cdot (y')' \cdot z' && \text{[genel De Morgan]} \\
 &= w \cdot x' \cdot y \cdot z' && \text{[Teorem 3: } (x')' = x \text{]}
 \end{aligned}$$

2.5.3 Bir Fonksiyonun Tümleyenini Bulmanın İki Yolu

Karmaşık bir Bool ifadesinin (F) tümleyenini bulmak için iki yöntemimiz var.

Yol 1 (doğrudan): De Morgan'ı ifadenin en dışındaki işlemde başlayıp içe doğru, adım adım uygulamak, yukarıdaki örneklerde yaptığımız gibi.

Yol 2 (kısayol): Aşağıdaki iki adımı uygulamak:

1. F 'in **dualini** alın: her $+$ 'yi $\cdot$ 'ye, her $\cdot$ 'yi $+$ 'ye, her 0 'ı 1 'e, her 1 'i 0 'a çevirin (değişkenlerin kendisine dokunmayın).
2. Ortaya çıkan dual ifadedeki **her değişkeni kendi tümleyeniyile** değiştirin ($x \rightarrow x'$, $x' \rightarrow x$).

Sonuç, doğrudan F'' 'tir. Bu kısayol işe yarar çünkü n -değişkenli De Morgan teoremi, tekrar tekrar uygulandığında tam olarak bu iki işlemi (dual alma + literal tümleme) yapar; biz sadece bunu tek seferde, satır satır ispat yapmadan uyguluyoruz.

Örnek 2.10: İki Yöntemin Karşılaştırılması: $F = xy + x'z$ **Yol 1 (doğrudan De Morgan):**

$$\begin{aligned}
F' &= (xy + x'z)' = (xy)' \cdot (x'z)' && [\text{De Morgan, dış toplama}] \\
&= (x' + y') \cdot (x + z') && [\text{De Morgan + Teorem 3: } (x')' = x]
\end{aligned}$$

Yol 2 (kısayol):

1. $F = xy + x'z$ 'in duali: $+ \leftrightarrow \cdot$ değişimiyle $F^d = (x + y)(x' + z)$.
2. Her değişkeni tümle: $x \rightarrow x', y \rightarrow y', x' \rightarrow x, z \rightarrow z'$. Sonuç: $F' = (x' + y')(x + z')$. İki yöntem de aynı sonucu verdi: $F' = (x' + y')(x + z')$.

Alıştırma 2.7 ★★ ★ $F = x'y + xz'$ ifadesinin tümleyenini kısayol yöntemiyle (dual al, her değişkeni tümle) bulunuz. Sonucu doğrudan De Morgan uygulayarak da doğrulayınız.

Çözüm (Kısayol):

1. Dual: $F^d = (x' + y)(x + z')$.
2. Her değişkeni tümle ($x \rightarrow x', y \rightarrow y', x' \rightarrow x, z' \rightarrow z$): $F' = (x + y')(x' + z)$.

Doğrulama (doğrudan De Morgan):

$$\begin{aligned}
F' &= (x'y + xz')' = (x'y)' \cdot (xz')' && [\text{De Morgan, dış toplama}] \\
&= (x + y') \cdot (x' + z) && [\text{De Morgan + Teorem 3}]
\end{aligned}$$

İki yöntem yine aynı sonucu verdi.

2.5.4 Diğer Mantık İşlemleri

Şimdiye kadar x, y değişkenleriyle AND (xy) ve OR ($x + y$) fonksiyonlarını kullandık. Ama bunlar, x ve y 'den kurulabilecek fonksiyonların yalnızca ikisi. n ikili değişkenin 2^n farklı giriş kombinasyonu vardır (doğruluk tablosunun 2^n satırı); her satırda F bağımsız olarak 0 ya da 1 olabileceğinden, toplam 2^{2^n} farklı fonksiyon mümkündür. İki değişken için ($n = 2$) bu sayı $2^4 = 16$ 'dır. Bu 16 fonksiyonun doğruluk tabloları Tablo 2.8'da listelenmiştir:

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Tablo 2.8: İki ikili değişkenin 16 fonksiyonu için doğruluk tabloları

Her sütun, Bool ifadesi ve (varsa) özel bir adla Tablo 2.9'de özetlenmiştir:

Fonksiyon	Operatör sembolü	Adı	Açıklama
$F_0 = 0$	—	Sıfır	İkili sabit 0
$F_1 = xy$	$x \cdot y$	AND	x ve y
$F_2 = xy'$	x/y	Engelleme (inhibition)	x var, y yok
$F_3 = x$	x	Aktarım (transfer)	x
$F_4 = x'y$	y/x	Engelleme (inhibition)	y var, x yok
$F_5 = y$	y	Aktarım (transfer)	y
$F_6 = xy' + x'y$	$x \oplus y$	XOR (Özel-VEYA)	x veya y , ikisi birden değil
$F_7 = x + y$	$x + y$	OR	x veya y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	XNOR (Denklik)	x , y 'ye eşit
$F_{10} = y'$	y'	Tümleme (complement)	y değil
$F_{11} = x + y'$	$x \subset y$	Çıkarım (implication)	y ise x
$F_{12} = x'$	x'	Tümleme (complement)	x değil
$F_{13} = x' + y$	$x \supset y$	Çıkarım (implication)	x ise y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$	—	Birim	İkili sabit 1

Tablo 2.9: İki değişkenin 16 fonksiyonu için Bool ifadeleri

Bu 16 fonksiyon üç grupta toplanabilir: iki sabit fonksiyon (Sıfır, Birim), dört tekli (*unary*) işlem (iki Tümleme, iki Aktarım) ve sekiz farklı ikili işlemi tanımlayan on fonksiyon (AND, OR, NAND, NOR, XOR, XNOR, Engelleme, Çıkarım). Bu özel sembollerin (örneğin $\uparrow$, $\downarrow$, $\subset$) hiçbiri Bölüm 2.6'de göreceğimiz kapı sembolleri kadar yaygın değildir; yalnızca $\oplus$ (XOR) sayısal tasarımcılar arasında gerçekten kullanılır. Engelleme ve çıkarım işlemleri mantıkçılar tarafından kullanılsa da sayısal tasarımda neredeyse hiç görülmez.

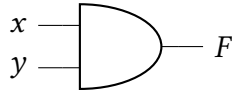
XOR ile XNOR birbirinin tümleyenidir: Tablo 2.8'da F_6 (XOR) ile F_9 (XNOR) sütunlarını karşılaştırırsanız, her satırda tam tersi değerler taşıdıklarını görürsünüz. Bu yüzden XNOR'a *eşdeğerlik* de denir: x ile y eşit olduğunda 1 üretir.

2.6 Sayısal Mantık Kapıları

Şimdiye kadar Bool cebirini yalnızca cebirsel bir sistem olarak ele aldık: değişkenler, postülatlar, teoremler. Ama bu bölümün başında da vurguladığımız gibi, Bool cebirinin asıl gücü, fiziksel bir devreye (*mantık kapısına*) birebir karşılık gelmesinden gelir. Bir mantık fonksiyonu üç eşdeğer biçimde ifade edilebileceğini (Bölüm 2.1.2) hatırlayın: doğruluk tablosu, Bool ifadesi ve lojik devre şeması. Şimdiye kadar ilk ikisiyle çalıştık; bu bölümde üçüncü gösterimi, yani gerçek kapı sembollerini tanıyacağız.

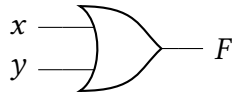
2.6.1 Sekiz Temel Kapı

Bir Bool fonksiyonunu AND, OR ve NOT işlemleriyle gerçeklemek en doğal yoldur, çünkü Bool cebirinin kendisi bu üç işlem üzerine kurulu. Ama pratikte başka kapılar da kullanılır: üretim kolaylığı, çok girişe genişleyebilme ve diğer kapılarla birlikte bir fonksiyonu tek başına gerçekleyebilme gibi nedenlerle. Aşağıdaki sekiz kapı, sayısal tasarımda standart kabul edilen kapılardır: her biri için grafik sembolü, doğruluk tablosu ve Bool ifadesi birlikte verilmiştir.

AND

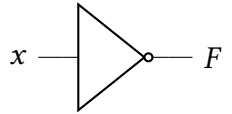
$$F = x \cdot y$$

x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

Şekil 2.10: AND kapısı: sembol ve doğruluk tablosu.**OR**

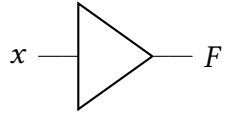
$$F = x + y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

Şekil 2.11: OR kapısı: sembol ve doğruluk tablosu.**NOT (Tersleyici)**

$$F = x'$$

x	F
0	1
1	0

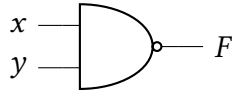
Şekil 2.12: NOT (tersleyici) kapısı: sembol ve doğruluk tablosu.**Tampon (Buffer)**

$$F = x$$

x	F
0	0
1	1

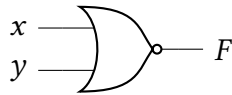
Şekil 2.13: Tampon (buffer) kapısı: sembol ve doğruluk tablosu.**Bilgi Notu 2.12: Tampon Bir Mantık İşlemi mi?**

Tampon (buffer) kapısı, girişini olduğu gibi çıkışa aktarır; mantıksal bir işlem yapmaz. Peki neden var? Sinyal zayıfladığında (uzun bir tel üzerinde ya da çok fazla kapıyı sürerken) gerilim seviyesini tazelemek için kullanılır; elektriksel olarak birbirine ardı ardına bağlı iki tersleyiciye eşdeğerdir ($((x')') = x$, Bölüm 2.4 Teorem 3).

NAND

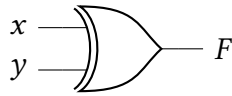
$$F = (x \cdot y)'$$

x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

Şekil 2.14: NAND kapısı: sembol ve doğruluk tablosu.**NOR**

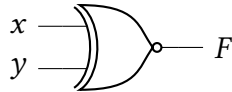
$$F = (x + y)'$$

x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

Şekil 2.15: NOR kapısı: sembol ve doğruluk tablosu.**XOR (Özel-VEYA)**

$$F = xy' + x'y = x \oplus y$$

x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

Şekil 2.16: XOR (özel-VEYA) kapısı: sembol ve doğruluk tablosu.**XNOR (Denklik)**

$$F = xy + x'y' = (x \oplus y)'$$

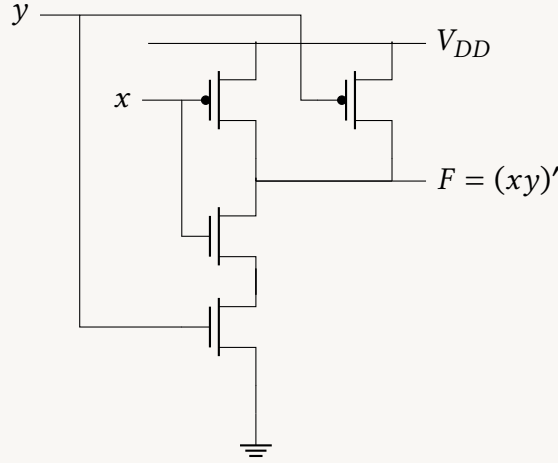
x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

Şekil 2.17: XNOR (denklik) kapısı: sembol ve doğruluk tablosu.**Bilgi Notu 2.13: NAND ve NOR Neden Bu Kadar Yaygın?**

AND ve OR yerine pratikte NAND ve NOR kapıları çok daha sık kullanılır. Nedeni cebirsel değil, fiziksel: NAND ve NOR, transistör devreleriyle AND/OR'dan daha az elemanla ve daha hızlı gerçekleşir. İleride (kombinasyonel mantık bölümünde) göreceğimiz gibi, tek başına NAND ya da tek başına NOR kullanarak *herhangi* bir Bool fonksiyonu gerçekleştirilebilir. Bu özelliğe *fonksiyonel tamlik* denir.

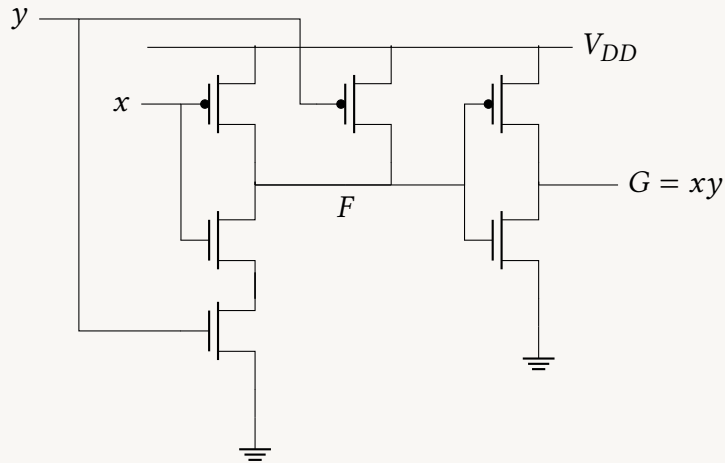
Detaylı Bilgi 2.3: CMOS Transistör Düzeyinde NAND ve NOR Neden Daha Az Eleman Gerektirir?

Bu "daha az eleman" iddiasını CMOS (bkz. Bölüm 2.8.2) transistör düzeyinde somutlaştıralım. CMOS'ta her kapı, girişleri VDD'ye bağlayan bir PMOS ağı ile girişleri toprağa (GND) bağlayan bir NMOS ağından oluşur. İki girişli bir NAND, iki PMOS'un *paralel* (VDD'ye bağlı) ve iki NMOS'un *seri* bağlanmasıyla doğrudan gerçekleşir; toplam dört transistör yeterlidir:



Toplam: 4 transistör (2 PMOS + 2 NMOS).

Aynı fonksiyonu doğrudan bir AND kapısıyla elde etmek isteseydik, NAND'ın ürettiği $(xy)'$ sonucunu bir tersleyiciden (2 transistör: 1 PMOS + 1 NMOS) geçirip yeniden tersi alınması gerekirdi, çünkü Bool cebirinde AND'i doğrudan üreten tek bir PMOS/NMOS ağı yoktur. Aşağıdaki devrede F , bir önceki NAND devresinin çıkışıyla aynı düğümdür; devrenin geri kalanı bu F 'i tersleyip $G = xy$ 'yi üretir:

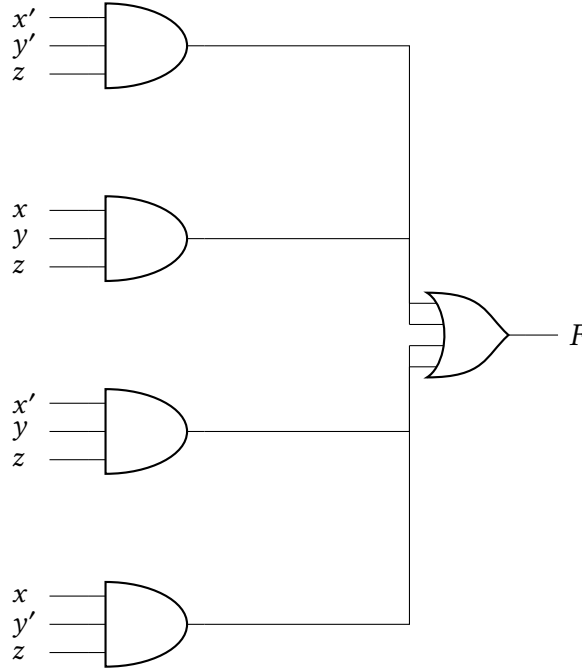


Toplam: 6 transistör (3 PMOS + 3 NMOS) — NAND'a göre bir PMOS-NMOS çifti fazla. Aynı fark OR/NOR arasında da geçerlidir. Bir işlemci milyonlarca kapı içerdiğinden, her kapı başına iki fazla transistör devasa bir alan ve güç maliyetine dönüşür; NAND/NOR'un tercih edilme nedeni tam olarak budur.

2.6.2 Sadeleştirmenin Önemi: Devre Üzerinden Gösterim

Artık kapı sembollerini tanıdığımıza göre, Bölüm 2.4.1’de sadeleştirdiğimiz $F = x'y'z + xyz + x'yz + xy'z$ örneğine geri dönüp, sadeleştirmenin devre üzerinde ne anlama geldiğini somut olarak görelim.

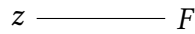
Sadeleştirilmemiş hâl dört çarpım terimi içerir; her terim üç girişli bir AND kapısı gerektirir, dört AND çıkışı da bir OR kapısında birleşir. Ayrıca x' ve y' için iki tersleyici (NOT) gerekir (şekle dahil edilmedi, ama toplam kapı sayısına dahildir):



Şekil 2.18: Sadeleştirilmemiş devre: $F = x'y'z + xyz + x'yz + xy'z$ (4 AND + 1 OR; 2 NOT şekle dahil edilmedi).

Toplam: 2 NOT + 4 AND (3 girişli) + 1 OR (4 girişli) = 7 kapı.

Sadeleştirilmiş hâl ise $F = z$ idi: yani F , z girişinin **doğrudan kendisi**. Hiçbir kapıya gerek yok:



Şekil 2.19: Sadeleştirilmiş devre: $F = z$, tek bir tel.

Toplam: 0 kapı. Aynı fonksiyonu 7 kapı yerine tek bir telle gerçekliyoruz.

Bilgi Notu 2.14: Sadeleştirme Neden Bu Kadar Önemli?

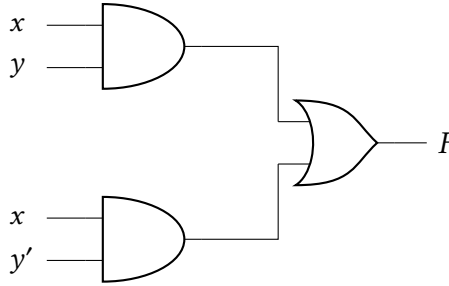
Bu örnek uç bir durum ($7 \rightarrow 0$) ama gerçek tasarımlarda sadeleştirme her zaman gerçek bir kazanç sağlar: daha az kapı demek daha az transistör, daha az silikon alanı, daha az güç tüketimi ve (sinyalin daha az kapıdan geçmesi sayesinde) daha kısa yayılma gecikmesi demektir. Bir işlemci milyonlarca kez tekrar eden bir alt devrede tek bir kapıyı bile kaldırabilmek, üretimde büyük bir maliyet/performans farkı yaratır. Bu yüzden sadeleştirme yöntemleri (Karnaugh haritaları gibi), kitabın ilerleyen bir bölümünde ayrıca ve derinlemesine ele alınacak.

Alıştırma 2.8 ★★ $F = xy + xy'$ fonksiyonunu sadeleştirip, sadeleştirilmemiş ve sadeleştirilmiş hâllerin kapı sayılarını karşılaştırınız.

Çözüm:

$$\begin{aligned}
 F &= xy + xy' = x(y + y') && \text{[Postülat 4: dağılma]} \\
 &= x \cdot 1 && \text{[Postülat 5: } y + y' = 1\text{]} \\
 &= x && \text{[Postülat 2: birim eleman]}
 \end{aligned}$$

Sadeleştirilmemiş hâl: y' için 1 NOT, xy ve xy' için 2 AND (2 girişli), ikisini birleştirmek için 1 OR (2 girişli) gerekir (NOT şekle dahil edilmedi, ama toplam kapı sayısına dahildir):



Toplam: 1 NOT + 2 AND + 1 OR = **4 kapı.**

Sadeleştirilmiş hâl: $F = x$, yani x girişinin doğrudan kendisi:



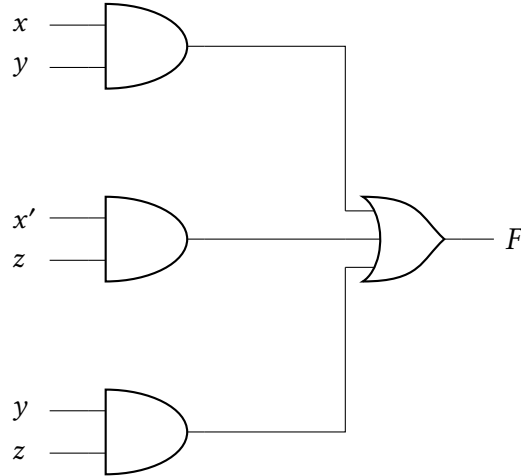
Toplam: **0 kapı.**

Alıştırma 2.9 ★★★ $F = xy + x'z + yz$ fonksiyonunu sadeleştiriniz. *İpucu:* yz terimini $x + x' = 1$ ile çarpıp açın.

Çözüm:

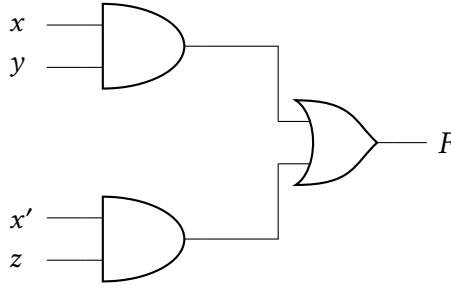
$$\begin{aligned}
 F &= xy + x'z + yz = xy + x'z + yz(x + x') && \text{[Postülat 5: } x + x' = 1, \text{ birim ile çarp]} \\
 &= xy + x'z + xyz + x'yz && \text{[Postülat 4: dağılma]} \\
 &= xy(1 + z) + x'z(1 + y) && \text{[gruplama + Postülat 4]} \\
 &= xy \cdot 1 + x'z \cdot 1 && \text{[Teorem 2: } 1 + z = 1, 1 + y = 1\text{]} \\
 &= xy + x'z && \text{[Postülat 2: birim eleman]}
 \end{aligned}$$

yz terimi, xy ve $x'z$ 'nin yanında **gereksizdi**: x ile x' 'nin ikisi de 0 olamayacağından, xy veya $x'z$ 'den biri zaten yz 'nin kapsadığı her durumu içeriyordu. **Sadeleştirilmemiş hâl:** x' için 1 NOT, $xy/x'z/yz$ için 3 AND (2 girişli), 1 OR (3 girişli) gerekir (NOT şekle dahil edilmedi, ama toplam kapı sayısına dahildir):



Toplam: 1 NOT + 3 AND + 1 OR = 5 kapı.

Sadeleştirilmiş hâl: x' için 1 NOT, $xy/x'z$ için 2 AND (2 girişli), 1 OR (2 girişli) gerekir:



Toplam: 1 NOT + 2 AND + 1 OR = 4 kapı.

2.6.3 Çok Girişli Genişletme

Tersleyici ve tampon dışındaki kapılar iki girişten fazlasına genişletilebilir. Bir kapının çok girişe genişleyebilmesi için temsil ettiği işlemin hem **değişmeli** hem **birleşmeli** olması gerekir. OR işlemi için:

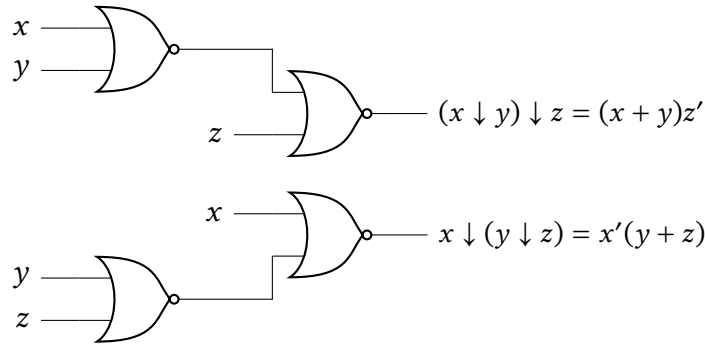
$$x + y = y + x \quad (\text{değişme}), \quad (x + y) + z = x + (y + z) = x + y + z \quad (\text{birleşme})$$

Bu, kapı girişlerinin yer değiştirebileceğini ve OR fonksiyonunun üç ya da daha çok değişkene genişleyebileceğini gösterir. Aynısı AND için de geçerlidir.

NAND ve NOR işlemleri değişmelidir, ama **birleşmeli değildir**: $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$. Bunu cebirsel olarak gösterelim:

$$\begin{aligned} (x \downarrow y) \downarrow z &= [(x + y)' + z]' = (x + y)z' = xz' + yz' \\ x \downarrow (y \downarrow z) &= [x + (y + z)']' = x'(y + z) = x'y + x'z \end{aligned}$$

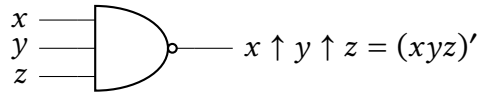
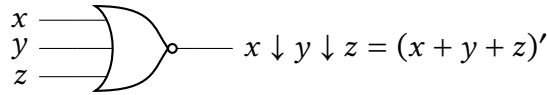
İki sonuç farklı olduğundan $(xz' + yz' \neq x'y + x'z)$, NOR birleşmeli değildir. Aynı durum NAND için de geçerlidir. Devre üzerinde bu fark açıkça görülür:



Şekil 2.20: NOR işleminin birleşmeli olmadığını gösterimi: $(x \downarrow y) \downarrow z \neq x \downarrow (y \downarrow z)$.

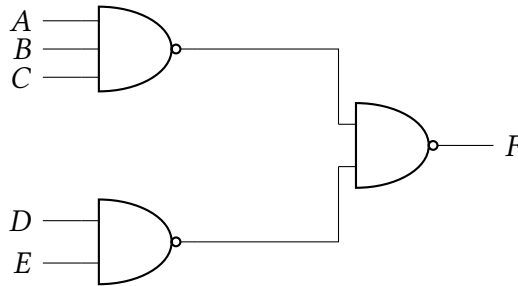
Bu sorunu aşmak için çok girişli NOR (ya da NAND) kapısı, ardı ardına bağlantı üzerinden değil, **doğrudan tümlenmiş bir OR (ya da AND) kapısı** olarak tanımlanır:

$$x \downarrow y \downarrow z = (x + y + z)', \quad x \uparrow y \uparrow z = (xyz)'$$



Şekil 2.21: Üç girişli NOR ve NAND kapılarının grafik sembolleri.

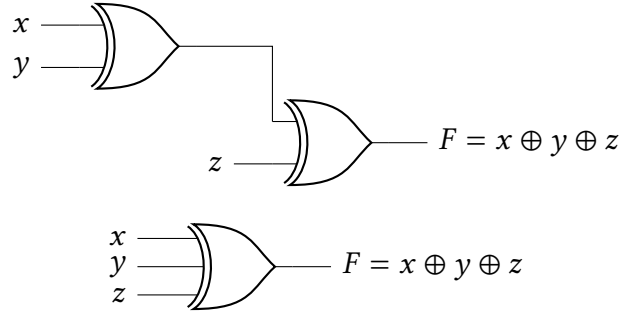
Ardı ardına bağlı NAND kapılarını yazarken doğru parantezlemeye dikkat etmek gerekir. Örneğin aşağıdaki devrede:



Şekil 2.22: Ardı ardına bağlı NAND kapılarıyla $F = [(ABC)'(DE)']' = ABC + DE$ gerçekleştirilmesi.

fonksiyon şöyle yazılmalıdır: $F = [(ABC)'(DE)']'$. De Morgan uygulanınca $F = ABC + DE$ elde edilir; yani bu devre, aynı fonksiyonu yalnızca NAND kapılarıyla gerçekleştirebiliyor (bu tür standart ifade biçimlerine ilerleyen bir bölümde ayrıca değineceğiz).

XOR ve XNOR hem değişmeli hem birleşmelidir, dolayısıyla çok girişe genişleyebilirler. Ama donanımda çok girişli XOR nadirdir; iki girişli hâli bile genelde başka kapı türleriyle kurulur. Üstelik çok değişkenli tanımı da değişir: XOR bir **tek sayı fonksiyonudur** (*odd function*): giriş değişkenlerindeki 1 sayısı tek olduğunda 1 üretir. Üç girişli XOR, iki girişli kapıların ardı ardına bağlanmasıyla ya da tek bir üç girişli kapı sembolüyle gösterilebilir:



Şekil 2.23: Üç girişli XOR'un iki eşdeğer gösterimi (üstte: ardı ardına bağlantı; altta: tek kapı sembolü).

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Tablo 2.10: Üç girişli XOR'un doğruluk tablosu.

Tablodan görüldüğü gibi $F = 1$, girişlerdeki 1 sayısı tek olduğunda üretilir (yani tam olarak bir tanesi ya da üçü de 1 olduğunda).

2.6.4 Pozitif ve Negatif Mantık

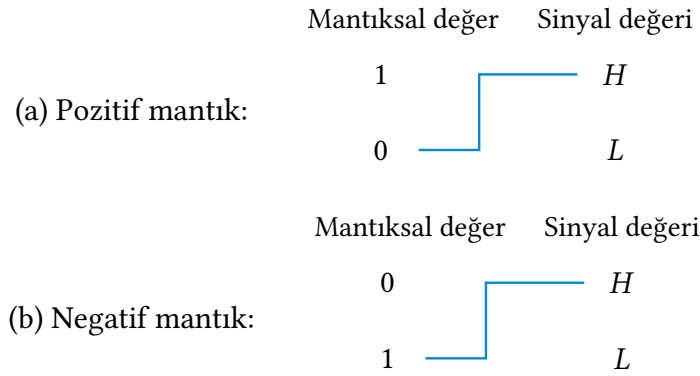
Bir kapının girişindeki ve çıkışındaki ikili sinyal (geçiş anları dışında) iki değerden birini alır: yüksek seviye (H) ve alçak seviye (L). İki sinyal değerine iki mantık değeri atanacağından, iki farklı atama mümkündür: yüksek seviyeyi (H) mantıksal 1 olarak seçmek **pozitif mantık**, alçak seviyeyi (L) mantıksal 1 olarak seçmek ise **negatif mantık** sistemi tanımlar. "Pozitif" ve "negatif" adları biraz yanıltıcıdır: iki sinyal de gerilim işareti bakımından pozitif ya da negatif olabilir; belirleyici olan sinyallerin gerçek değeri değil, iki seviyeye atanan görelî mantık değerleridir.

Donanım kapıları H ve L gibi sinyal değerleri cinsinden tanımlanır; pozitif ya da negatif mantık ataması kullanıcının tercihidir. Örneğin $H = 3\text{ V}$ ve $L = 0\text{ V}$ olan fiziksel bir kapıyı ele alalım:

Fiziksel (H/L)			Pozitif mantık ($H=1$)			Negatif mantık ($L=1$)		
x	y	F	x	y	F	x	y	F
L	L	L	0	0	0	1	1	1
L	H	L	0	1	0	1	0	1
H	L	L	1	0	0	0	1	1
H	H	H	1	1	1	0	0	0

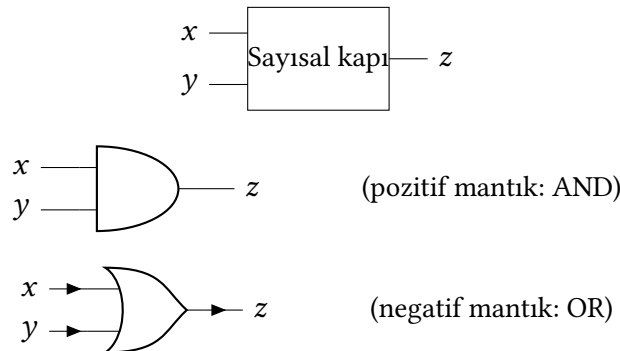
Tablo 2.11: Aynı fiziksel kapının pozitif mantıkta AND, negatif mantıkta OR olarak yorumlanması.

Pozitif mantık ataması ($H = 1, L = 0$) ile bu tablo AND işlemiyle birebir örtüşür. Şimdi aynı fiziksel kapıya negatif mantık ataması yapalım ($L = 1, H = 0$): sonuç, satırlar ters sırada olsa da, OR işlemiyle örtüşür. Yani **aynı fiziksel kapı**, pozitif mantıkta AND kapısı, negatif mantıkta ise OR kapısı olarak çalışır. Grafik sembollerde negatif mantık varsayıldığında uçlara küçük bir üçgen (kutupluluk göstergesi) eklenir. Önce mantıksal değerlerin sinyal değerine nasıl atandığını görselleştirelim:



Şekil 2.24: Mantıksal değerlerin sinyal değerine ataması.

Pozitif mantıkta $1 \rightarrow H, 0 \rightarrow L$ (üstte); negatif mantıkta $0 \rightarrow H, 1 \rightarrow L$ (altta). Basamak şeklindeki çizgi, fiziksel sinyalin düşük seviyeden (L) yüksek seviyeye (H) geçişini temsil eder. Şimdi aynı atamanın kapı sembollerine nasıl yansıdığını görelim:



Şekil 2.25: Aynı fiziksel kapının pozitif ve negatif mantıktaki sembolleri.

Aynı fiziksel kapı bloğu (üstte), pozitif mantıkta AND sembolüyle (ortada) ve negatif mantıkta uçlarındaki kutupluluk üçgenleriyle OR sembolüyle (altta) gösterilir.

Pozitif mantıktan negatif mantığa (ya da tersine) geçmek, girişlerin ve çıkışın hepsinde 1'leri 0'a, 0'ları 1'e çevirmek anlamına gelir; bu da fonksiyonun *dualini* almaktır (Bölüm 2.4). Bu yüzden kutup değişiminde tüm AND sembolleri OR'a, tüm OR sembolleri AND'e dönüşür.

Bilgi Notu 2.15: Bu Kitapta Hangi Mantık Kullanılıyor?

Negatif mantık pratikte nadiren tercih edilir ve kafa karıştırıcı olabilir. Bu kitap boyunca yalnızca **pozitif mantık** varsayılacak ($H = 1, L = 0$); negatif mantık kapıları bir daha kullanılmayacaktır. Burada tanıtılmasının amacı, aynı fiziksel donanımın atamaya bağlı olarak farklı yorumlanabileceğini görmenizdir.

2.7 Kanonik ve Standart Formlar

"Kanonik" sözcüğü Eski Yunanca *kanōn* (κανών) kökünden gelir; ilk anlamı düz bir çubuk ya da ölçüm aletiydi, zamanla mecazi olarak "kural, ölçüt, standart" anlamını kazandı. Latinceye *canonicus* olarak geçti, oradan da Türkçeye "kanon" (yasa, kural) ve "kanonik" (kurala uygun, standart) sözcükleri türedi. Matematikte bir **kanonik form**, bir nesneyi ifade etmenin standart ve **tek (unique)** yoludur.

Bu, Bölüm 2.6.2'de gördüğümüz sadeleştirmeden temelde farklıdır: sadeleştirme bir fonksiyonu *en az* kapıyla ifade etmeyi hedefler ve genelde tek bir "en sade" sonuca varmaz (izlenen algebratik yola göre farklı ama eşdeğer sonuçlar elde edilebilir). Kanonik form ise sadelik değil **teklik** arar: her Bool fonksiyonunun tam olarak bir kanonik minterm toplamı ve tam olarak bir kanonik maxterm çarpımı biçimi vardır. Bu bölümde bu iki standart biçimi tanıyacağız.

2.7.1 Minterm ve Maxterm

Bir ikili değişken ya normal (x) ya da tümlü (x') biçimde bulunabilir. İki değişken x, y AND ile birleştirildiğinde dört olası bileşim ortaya çıkar: $x'y', x'y, xy', xy$. Genel olarak n değişken 2^n farklı bileşim üretir.

Minterm: n ikili değişkenin her birinin (normal ya da tümlü biçimde) tam olarak bir kez AND ile çarpılmasından oluşan terime *minterm* (asgari terim, standart çarpım) denir. Değişken, ilgili bit 0 ise tümlü, 1 ise normal biçimde yazılır. n değişken için 2^n farklı minterm vardır; j 'inci minterm m_j ile gösterilir (j , ikili sayının onluk karşılığıdır).

Maxterm: n ikili değişkenin her birinin (normal ya da tümlü biçimde) tam olarak bir kez OR ile toplanmasından oluşan terime *maxterm* (azami terim, standart toplam) denir. Değişken, ilgili bit 0 ise normal, 1 ise tümlü biçimde yazılır — minterm'deki kuralın tam tersi. j 'inci maxterm M_j ile gösterilir.

Üç değişken için tüm minterm ve maxterm Tablo 2.12'de listelenmiştir:

x	y	z	Minterm	Simge	Maxterm	Simge
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Tablo 2.12: Üç ikili değişken için minterm ve maxtermeler.

Tablodan önemli bir ilişki görülür: j 'inci maxterm, j 'inci mintermin tümleyenidir ve tersi de doğrudur: $m_j' = M_j$. Örneğin $m_2 = x'yz'$ iken $M_2 = x + y' + z$; gerçekten de De Morgan ile $(x'yz')' = x + y' + z = M_2$.

2.7.2 Minterm Toplamı ve Maxterm Çarpımı Simgeleri

Bir fonksiyonun doğruluk tablosundan yola çıkarak, $F = 1$ üreten her satır için bir minterm yazıp bunları OR ile birleştirirsek, F 'i **minterm toplamı** biçiminde elde ederiz. Örnek olarak $F = x + y'z$ fonksiyonunun doğruluk tablosunu inceleyelim:

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Tablo 2.13: $F = x + y'z$ fonksiyonunun doğruluk tablosu.

$F = 1$ olan satırlar 1, 4, 5, 6, 7'dir; bu satırların mintermlerini OR ile birleştirirsek:

$$F = x'y'z + xy'z' + xy'z + xyz' + xyz = m_1 + m_4 + m_5 + m_6 + m_7$$

Bu, cebirsel olarak sadeleştirilebilir ($x + x'y'z = (x + x')(x + y'z) = x + y'z$, Postülat 4 ve 5 ile), ama kanonik biçimde bırakıyoruz çünkü amaç sadelik değil tekliktir. Bu toplamı kısaca yazmak için Σ (toplam) simgesi kullanılır: Σ , mintermlerin OR'lanmasını (toplanmasını) temsil eder ve şöyle yazılır:

$$F(x, y, z) = \Sigma(1, 4, 5, 6, 7)$$

Şimdi aynı fonksiyonun tümleyenini, $F = 0$ olan satırlardan (0, 2, 3) okuyalım:

$$F' = x'y'z' + x'yz' + x'yz = m_0 + m_2 + m_3 = \Sigma(0, 2, 3)$$

F' 'ye De Morgan uygularsak F 'i farklı bir kanonik biçimde elde ederiz:

$$\begin{aligned} F &= (F')' = (m_0 + m_2 + m_3)' \\ &= m'_0 \cdot m'_2 \cdot m'_3 \\ &= M_0 \cdot M_2 \cdot M_3 \end{aligned}$$

Son adım, bir önceki alt bölümde gördüğümüz $m'_j = M_j$ ilişkisinden gelir. Maxtermleri okuyup yerine koyarsak (bkz. Tablo 2.12):

$$\begin{aligned} F &= (x + y + z)(x + y' + z) \\ &\quad \cdot (x + y' + z') \\ &= M_0 \cdot M_2 \cdot M_3 \end{aligned}$$

Bu **maxterm çarpımı** biçimidir; kısaca yazmak için Π (çarpım) simgesi kullanılır: Π , maxtermlerin AND'lenmesini (çarpılmasını) temsil eder ve şöyle yazılır:

$$F(x, y, z) = \Pi(0, 2, 3)$$

Dikkat edilirse Σ ifadesindeki indisler (F 'in 1 olduğu satırlar) ile Π ifadesindeki indisler (F 'in 0 olduğu satırlar) birbirini tamamlar: üç değişkenli bir fonksiyon için toplam $2^3 = 8$ minterm/maxterm olduğundan, $\Sigma(1, 4, 5, 6, 7)$ ve $\Pi(0, 2, 3)$ aynı fonksiyonun iki farklı ama eşdeğer kanonik anlatımıdır. Özetle: Σ her zaman mintermlere (OR'lanan AND terimleri), Π her zaman maxtermlere (AND'lenen OR terimleri) karşılık gelir.

Bu gözlem genel bir kurala genişler: **bir kanonik formu diğerine çevirmek için Σ ile Π sembolleri yer değiştirilir ve orijinal formda yer almayan indisler listelenir.** n değişkenli bir fonksiyon için toplam minterm/maxterm sayısı 2^n 'dir.

2.7.3 Cebirsel İfadeyi Minterm Toplamına Genişletme

Bir Bool fonksiyonu elimize doğruluk tablosu olarak değil, cebirsel bir ifade olarak gelmişse, mintermlerini okumak için önce ifadeyi minterm toplamı biçimine **genişletmek** gerekir. Yöntem şöyledir: ifade önce çarpım terimlerinin toplamı hâline getirilir (gerekirse dağılma özelliğiyle parantezler açılır); sonra her terim incelenir, eğer bir değişken eksikse terim $(x + x') = 1$ ile çarpılıp o değişken hem normal hem tümlü biçimde eklenir.

Örnek 2.11: $F = x + y'z'$ İfadesini Minterm Toplamına Genişletme

Fonksiyonun üç değişkeni var: x, y, z . İlk terim x , iki değişken eksik:

$$\begin{aligned} x &= x(y + y') = xy + xy' \\ &= xy(z + z') + xy'(z + z') = xyz + xyz' + xy'z + xy'z' \end{aligned}$$

İkinci terim $y'z'$, bir değişken eksik:

$$y'z' = y'z'(x + x') = xy'z' + x'y'z'$$

Tüm terimleri birleştirirsek:

$$F = xyz + xyz' + xy'z + xy'z' + x'y'z'$$

$xy'z'$ iki kez görüldüğünden (Teorem 1: $x + x = x$ ile) bir tekrar silinir. İndis sırasına göre düzenlersek:

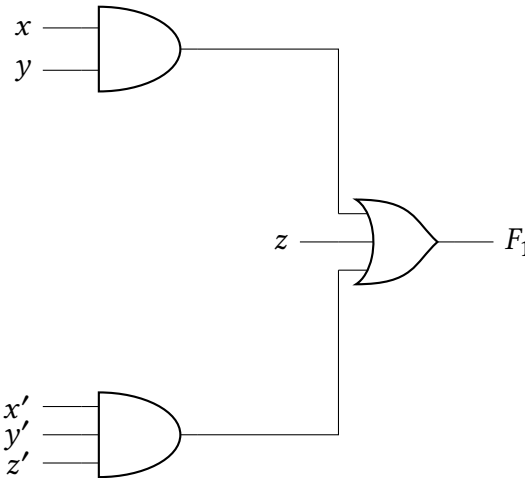
$$F = x'y'z' + xy'z' + xy'z + xyz' + xyz = m_0 + m_4 + m_5 + m_6 + m_7 = \Sigma(0, 4, 5, 6, 7)$$

2.7.4 Standart Formlar: Çarpımların Toplamı ve Toplamların Çarpımı

Kanonik formlar (minterm toplamı, maxterm çarpımı) genellikle **en az literalli** biçim değildir, çünkü tanım gereği her minterm/maxterm, değişkenlerin tümünü (normal ya da tümlü) içermek zorundadır. **Standart form** adı verilen başka bir gösterimde ise terimler bir, iki ya da istenilen sayıda literal içerebilir. İki standart form türü vardır: çarpımların toplamı ve toplamların çarpımı.

Çarpımların Toplamı (Sum of Products, SOP): Bir ya da daha çok literal içeren AND terimlerinin (*çarpım terimi*) OR ile birleştirilmesinden oluşan ifadeye çarpımların toplamı denir.

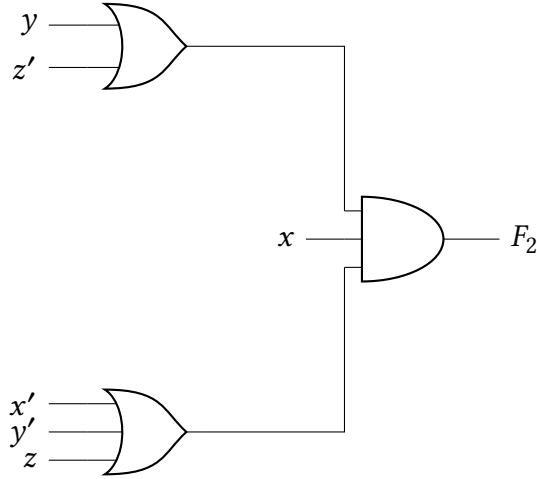
Örneğin $F_1 = z + xy + x'y'z'$ üç çarpım terimi içerir (bir, iki ve üç literalli). Devre şeması bir grup AND kapısının ardından tek bir OR kapısından oluşur; tek literalli terimler doğrudan OR kapısına bağlanır, ayrı bir AND kapısı gerektirmez. Bu yapıya **iki-seviyeli devre** denir, çünkü girişten çıkışa giden herhangi bir yolda en fazla iki kapı vardır.



Şekil 2.26: $F_1 = z + xy + x'y'z'$ ifadesinin çarpımların toplamı (SOP) biçiminde, iki-seviyeli gerçekleştirilmesi.

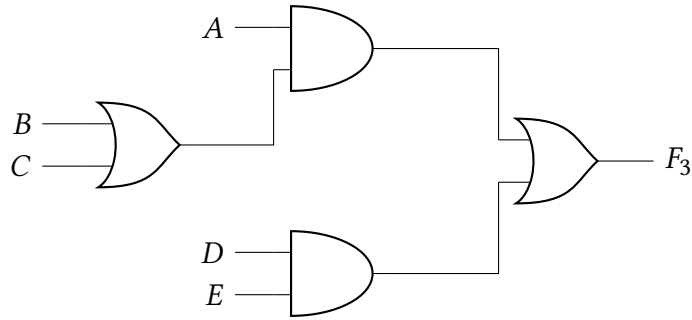
Toplamların Çarpımı (Product of Sums, POS): Bir ya da daha çok literal içeren OR terimlerinin (*toplam terimi*) AND ile birleştirilmesinden oluşan ifadeye toplamların çarpımı denir.

Örneğin $F_2 = x(y + z')(x' + y' + z)$ üç toplam terimi içerir. Devre şeması bir grup OR kapısının ardından tek bir AND kapısından oluşur; bu da iki-seviyeli bir devredir.



Şekil 2.27: $F_2 = x(y + z')(x' + y' + z)$ ifadesinin toplamaların çarpımı (POS) biçiminde, iki-seviyeli gerçekleştirilmesi.

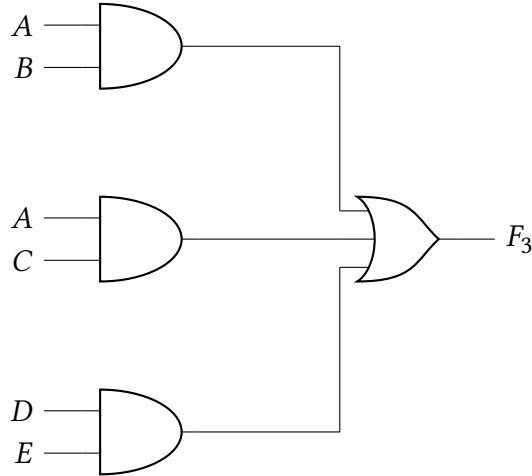
Her fonksiyon standart formda gelmeyebilir. Örneğin $F_3 = A(B + C) + DE$ ne SOP ne POS'tur; gerçekleştirilmesi üç seviyeli bir devre gerektirir. Girişten çıkışa en uzun yol OR, AND, OR sırasıyla üç kapıdan geçer:



Şekil 2.28: $F_3 = A(B + C) + DE$: standart olmayan, üç seviyeli devre.

Dağılma özelliğiyle parantezler açılırsa standart (SOP) forma indirgenir:

$$F_3 = A(B + C) + DE = AB + AC + DE$$



Şekil 2.29: $F_3 = AB + AC + DE$: aynı fonksiyonun standart (SOP), iki seviyeli gerçekleştirilmesi.

Genel olarak iki-seviyeli bir gerçekleştirme tercih edilir, çünkü girişten çıkışa sinyalin geçtiği kapı sayısı (ve dolayısıyla toplam yayılma gecikmesi) en azdır. Ancak bir kapının girebileceği giriş sayısının (fan-in) fiziksel bir sınırı olduğundan, çok sayıda terimli gerçek tasarımlarda çok seviyeli devrelerden kaçınılamayabilir.

2.8 Entegre Devreler

Şimdiye kadar mantık kapılarını soyut sembollerle ele aldık. Ama gerçekte bu kapılar, silikon bir yarı iletken kristal üzerine üretilen **entegre devre** (*integrated circuit*, IC) adı verilen bir *çip* üzerinde gerçekleşir. Çip üzerindeki kapılar istenen devreyi oluşturacak şekilde birbirine bağlanır; çip seramik ya da plastik bir kılıfa yerleştirilir ve dış bacaklara (pinlere) kaynaklanan bağlantılarla entegre devre tamamlanır. Bacak sayısı, küçük bir kılıftaki 14'ten büyük kılıflardaki binlerce bacağa kadar değişebilir; her IC, tanımlanması için üzerine basılmış sayısal bir koda sahiptir.

2.8.1 Entegrasyon Seviyeleri

Sayısal IC'ler, tek bir kılıftaki kapı sayısına göre sınıflandırılır:

- **SSI** (*small-scale integration*, küçük ölçekli entegrasyon): tek bir kılıfta birkaç bağımsız kapı bulunur; kapıların girişleri ve çıkışları doğrudan kılıfın bacaklarına bağlıdır. Kapı sayısı genellikle 10'dan azdır ve kılıftaki bacak sayısıyla sınırlıdır.
- **MSI** (*medium-scale integration*, orta ölçekli entegrasyon): tek bir kılıfta yaklaşık 10 ile 1000 arası kapı bulunur; genellikle kod çözücü, toplayıcı, çoklayıcı gibi temel sayısal işlemleri gerçekler (ilerleyen bölümlerde göreceğimiz yazmaç ve sayaçlar da bu sınıftandır).
- **LSI** (*large-scale integration*, büyük ölçekli entegrasyon): tek bir kılıfta binlerce kapı bulunur; işlemciler, bellek çipleri ve programlanabilir mantık aygıtları gibi sayısal sistemleri içerir.
- **VLSI** ve **ULSI** (*very/ultra large-scale integration*, çok/aşırı büyük ölçekli entegrasyon): tek bir kılıfta milyonlarca kapı bulunur; ULSI devreleri bir milyondan fazla transistör içerir. Büyük bellek dizileri ve karmaşık mikroişlemci çipleri örnektir. Küçük boyutları ve düşük maliyetleri sayesinde VLSI aygıtları, daha önce üretilmesi ekonomik olmayan

yapıların tasarlanabilmesini sağlayarak sayısal sistem tasarımını dönüştürmüştür.

2.8.2 Dijital Lojik Aileleri

Sayısal entegre devreler yalnızca karmaşıklıklarına göre değil, ait oldukları devre teknolojisine göre de sınıflandırılır; bu teknolojiye **lojik ailesi** denir. Her ailenin daha karmaşık devrelerin üzerine kurulduğu temel bir devresi vardır ve bu temel devre her zaman bir NAND, NOR ya da tersleyicidir. En yaygın dört aile şunlardır:

- **TTL** (*transistor-transistor logic*, transistör-transistör lojik): 50 yılı aşkın süredir kullanılan, standart kabul edilen bir aile.
- **ECL** (*emitter-coupled logic*, emiter-bağlaşımlı lojik): yüksek hız gerektiren sistemlerde avantajlıdır.
- **MOS** (*metal-oxide semiconductor*, metal-oksit yarı iletken): yüksek bileşen yoğunluğu gerektiren devrelerde tercih edilir.
- **CMOS** (*complementary metal-oxide semiconductor*, tümleyen metal-oksit yarı iletken): düşük güç tüketimi gerektiren sistemlerde (dijital kameralar, taşınabilir medya oynatıcılar gibi) tercih edilir. CMOS'un bir NAND kapısını PMOS/NMOS transistörleriyle nasıl gerçekleştirdiğini Detaylı Bilgi 2.3'te zaten görmüştük. Düşük güç tüketimi VLSI tasarımı için kritik olduğundan, CMOS günümüzde baskın lojik ailesi hâline gelmiştir; TTL ve ECL kullanımı giderek azalmaktadır.

Lojik aileleri birbirinden ayıran en önemli parametreler şunlardır:

- **Fan-out (çıkış yükü)**: bir kapının çıkışının, normal çalışmasını bozmadan sürebileceği standart yük sayısı. Standart yük, genellikle aynı ailedeki başka bir kapının girişinin çektiği akım olarak tanımlanır.
- **Fan-in (giriş sayısı)**: bir kapıdaki giriş sayısı.
- **Güç tüketimi**: kapının çalışması için güç kaynağından çekilmesi gereken güç.
- **Yayılma gecikmesi (propagation delay)**: bir sinyalin girişten çıkışa ulaşması için geçen ortalama süre. Örneğin bir tersleyicinin girişi 0'dan 1'e geçtiğinde, çıkış 1'den 0'a bir gecikmeyle geçer; çalışma hızı bu gecikmeyle ters orantılıdır.
- **Gürültü payı**: bir giriş sinyaline eklenen ve devrenin çıkışında istenmeyen bir değişikliğe yol açmayan en büyük dış gürültü gerilimi (bkz. Detaylı Bilgi 2.2).

Terimler Sözlüğü

Taban (Radix) — Pozisyonel bir sayı sisteminde kullanılan basamak sayısı; her basamağın ağırlığı bu sayının kuvvetleriyle belirlenir (ör. ikili için 2, onaltılık için 16).

Ardışık Bölme/Çarpma Yöntemi — Ondalık bir sayıyı başka bir tabana çevirmek için kullanılan yöntem; tam sayı kısmı ardışık bölme (kalanlar aşağıdan yukarı okunur), kesirli kısım ardışık çarpma (taşan basamaklar yukarıdan aşağı okunur) ile çevrilir.

Tümleyen (Complement) — Bir sayının çıkarma işlemini toplamaya çevirmek için kullanılan dönüştürülmüş hâli; $(r-1)$ 'in tümleyeni (her basamağı $r-1$ 'den çıkararak) ve r 'nin tümleyeni (ona 1 eklenerek) olmak üzere iki türü vardır.

Uçtan-Dolaşan Elde (End-Around Carry) — $(r-1)$ 'in tümleyeniyle çıkarma yapıldığında oluşan taşma bitinin atılmayıp sonucun en düşük anlamlı basamağına eklenmesi kuralı.

İşaret Biti — İşaretle bir ikili sayıda en soldaki bit; 0 pozitif, 1 negatif anlamına gelir.

Taşma (Overflow) — n bitlik iki sayının toplamının n bite sığmaması durumu; donanımda sabit bit genişliği nedeniyle sonucun bozulmasına yol açar.

Normalizasyon — Bir kayan noktalı sayıyı $1.xxxx \times 2^k$ biçiminde, baştaki basamak tam olarak 1 olacak şekilde yazma işlemi; ikilide baştaki 1 her zaman var olduğu için saklanmaz (örtük bit).

Bias — IEEE 754'te üs alanına eklenen sabit kaydırma değeri (tek duyarlılıkta 127); negatif üsleri de işaretsiz tam sayı olarak karşılaştırılabilir kılar.

Bool Değişkeni — Yalnızca 0 veya 1 değerini alabilen değişken.

Mantık Kapısı — Bir veya birden çok ikili girişten tek bir ikili çıkış üreten temel devre elemanı.

Gerilim Eşiği (Threshold) — Bir girişin ya da çıkışın Lojik 0 veya Lojik 1 sayılacağı sınır gerilim değeri; V_{IL} , V_{IH} , V_{OL} , V_{OH} simgeleriyle gösterilir.

Gürültü Payı (Noise Margin) — Bir giriş sinyaline eklenen ve devrenin çıkışında istenmeyen bir değişikliğe yol açmayan en büyük dış gürültü gerilimi.

Bool Cebiri (Boolean Algebra) — Kapalılık, birim eleman, değişme, dağılma, tümlleme ve farklılık postülatlarını sağlayan bir küme ve iki ikili işleminden oluşan cebirsel yapı.

Postülat (Postulate) — Bir cebirsel yapının ispatsız kabul edilen, diğer tüm teoremlerin üzerine kurulduğu temel özdeşliği.

Düallite İlkesi (Principle of Duality) — Bool cebirinde geçerli olan her özdeşliğin, AND ile OR'un ve 0 ile 1'in yerleri değiştirilerek elde edilen dualinin de geçerli olması ilkesi.

De Morgan Teoremi — $(x+y)' = x'y'$ ve $(xy)' = x' + y'$ özdeşlikleri; bir toplamın (çarpımın) tümleyeni, terimlerin tümleyenlerinin çarpımı (toplamı) olarak ifade eder.

NAND — AND işleminin tümleyeni: $F = (xy)'$.

NOR — OR işleminin tümleyeni: $F = (x + y)'$.

XOR (Özel-VEYA, Exclusive-OR) — $F = xy' + x'y = x \oplus y$; girişleri birbirinden farklı olduğunda 1 üreten kapı. Çok girişli hâli bir *tek sayı fonksiyonudur*: girişlerdeki 1 sayısı tek

olduğunda 1 üretir.

XNOR (Denklik, Equivalence) – XOR’un tümleyeni: $F = xy + x'y' = (x \oplus y)'$; girişleri birbirine eşit olduğunda 1 üreten kapı.

Fonksiyonel Tamlık (Functional Completeness) – Tek başına NAND ya da tek başına NOR kapısı kullanılarak herhangi bir Bool fonksiyonunun gerçekleştirilebilmesi özelliği.

Pozitif/Negatif Mantık – Yüksek sinyal seviyesinin (H) mantıksal 1 sayılması pozitif mantığı, alçak seviyenin (L) mantıksal 1 sayılması negatif mantığı tanımlar; aynı fiziksel kapı, atamaya bağlı olarak farklı bir işlemi gerçekleştiriyormuş gibi yorumlanabilir.

Minterm – n ikili değişkenin her birinin (normal ya da tümlü biçimde) tam olarak bir kez AND ile çarpılmasından oluşan terim; m_j ile gösterilir.

Maxterm – n ikili değişkenin her birinin (normal ya da tümlü biçimde) tam olarak bir kez OR ile toplanmasından oluşan terim; M_j ile gösterilir.

Kanonik Form (Canonical Form) – Bir Bool fonksiyonunu ifade etmenin standart ve tek (unique) yolu; minterm toplamı (Σ) ya da maxterm çarpımı (Π) biçiminde olur.

Entegre Devre (Integrated Circuit, IC) – Silikon bir yarı iletken kristal üzerine üretilen, birbirine bağlı mantık kapılarını içeren ve bir kılıfa yerleştirilen çip.

Lojik Ailesi (Logic Family) – Bir entegre devrenin ait olduğu, temel devresi her zaman bir NAND, NOR ya da tersleyici olan devre teknolojisi (ör. TTL, ECL, MOS, CMOS).

CMOS (Complementary Metal-Oxide Semiconductor) – PMOS ve NMOS transistör ağlarının birlikte kullanıldığı, düşük güç tüketimiyle bilinen ve VLSI tasarımında baskın olan lojik ailesi.

Fan-out (Çıkış Yüğü) – Bir kapının çıkışının, normal çalışmasını bozmadan sürebileceği standart yük sayısı.

Yayılma Gecikmesi (Propagation Delay) – Bir sinyalin girişten çıkışa ulaşması için geçen ortalama süre; çalışma hızı bu gecikmeyle ters orantılıdır.